MIF 1.5: Concepts of Programming Languages (Winter 2026/27)

BC George & Carsten Gips (HSBI)

16. Oktober 2026

Inhaltsverzeichnis

I. Syllabus 1

1. Syllabus 2
- 1.1. Kursbeschreibung . . . 2
- 1.2. Überblick Modulinhalte . . . 2
- 1.3. Team . . . 3
- 1.4. Kursformat . . . 3
- 1.5. Fahrplan . . . 3
- 1.6. Prüfungsform, Note und Credits . . . 5
  - 1.6.1. **Studienleistung**: "Portfolio" . . . 5
  - 1.6.2. **Gesamtnote**: Mündliche Prüfung (einzeln, ca. 45 Minuten) . . . 5
  - 1.6.3. Hinweise . . . 5
- 1.7. Materialien . . . 6
- 1.8. Förderungen und Kooperationen . . . 7
  - 1.8.1. Kooperation mit University of Alberta, Edmonton (Kanada) . . . 7
- 1.9. LICENSE . . . 7

II. Vorlesungsunterlagen 8

2. Überblick 9
- 2.1. Struktur eines Compilers . . . 9
  - 2.1.1. Sprachen verstehen, Texte transformieren . . . 9
  - 2.1.2. Compiler: Big Picture . . . 10
  - 2.1.3. Lexikalische Analyse: Wörter ("*Token*") erkennen . . . 12
  - 2.1.4. Syntaxanalyse: Sätze erkennen . . . 12
  - 2.1.5. Vorschau: Parser implementieren . . . 13
  - 2.1.6. Semantische Analyse: Bedeutung erkennen . . . 14
  - 2.1.7. Zwischencode generieren . . . 14
  - 2.1.8. Code optimieren . . . 15
  - 2.1.9. Code generieren . . . 15
  - 2.1.10. Probleme . . . 15
  - 2.1.11. Unbedingt lesenswert . . . 16
  - 2.1.12. Wrap-Up . . . 16
- 2.2. Bandbreite der Programmiersprachen . . . 16
  - 2.2.1. 99 Bottles of Beer . . . 17

2.2.2. Imperativ, Hardwarenah: C . . . 17
2.2.3. Imperativ, Objektorientiert: Java . . . 18
2.2.4. Logisch: Prolog . . . 18
2.2.5. Funktional: Haskell . . . 19
2.2.6. Brainfuck . . . 20
2.2.7. Programmiersprache Lox . . . 20
2.2.8. Wrap-Up . . . 21
2.3. Anwendungen . . . 21
2.3.1. Anwendung: Compiler . . . 22
2.3.2. Anwendung: Interpreter . . . 22
2.3.3. Anwendung: Virtuelle Maschinen . . . 22
2.3.4. Anwendung: C-Toolchain . . . 23
2.3.5. Anwendung: C++-Compiler . . . 23
2.3.6. Anwendung: Bugfinder . . . 23
2.3.7. Anwendung: Pandoc . . . 24
2.3.8. Was bringt mir das? . . . 25
2.3.9. Wrap-Up . . . 27

3. Lexikalische Analyse **29**
3.1. Reguläre Sprachen, Ausdrucksstärke . . . 29
3.1.1. Motivation . . . 29
3.1.2. Endliche Automaten . . . 29
3.1.3. Reguläre Sprachen . . . 30
3.1.4. Wrap-Up . . . 32
3.2. Lexer: Handcodierte Implementierung . . . 33
3.2.1. Lexer: Erzeugen eines Token-Stroms aus einem Zeichenstrom . . . 34
3.2.2. Manuelle Implementierung: Rekursiver Abstieg . . . 34
3.2.3. Read-Ahead: Unterscheiden von “<” und “<=” . . . 35
3.2.4. Puffern des Input-Stroms: Double Buffering . . . 35
3.2.5. Typische Muster für Erstellung von Token . . . 36
3.2.6. Fehler bei der Lexikalischen Analyse . . . 38
3.2.7. Wrap-Up . . . 39

4. Syntaktische Analyse **40**
4.1. CFG . . . 40
4.1.1. Wiederholung . . . 40
4.1.2. Motivation . . . 40
4.1.3. Kellerautomaten (Push-Down-Automata, PDAs) . . . 41
4.1.4. Kontextfreie Grammatiken und Sprachen . . . 42
4.1.5. Wrap-Up . . . 43
4.2. LL-Parser . . . 43
4.2.1. Wiederholung . . . 44
4.2.2. Motivation . . . 44
4.2.3. Syntaxanalyse . . . 44

4.2.4. Algorithmus: Rekursiver Abstieg . . . 45
4.2.5. Tabellengesteuerte Parser: LL(k)-Grammatiken . . . 45
4.2.6. Wrap-Up . . . 49
4.3. LL-Parser selbst implementiert . . . 50
4.3.1. Erinnerung Lexer: Zeichenstrom => Tokenstrom . . . 50
4.3.2. Grundidee LL-Parser . . . 51
4.3.3. Alternative Subregeln . . . 51
4.3.4. Optionale Subregeln: Eins oder keins . . . 52
4.3.5. Optionale Subregeln: Mindestens eins . . . 52
4.3.6. LL(1)-Parser . . . 52
4.3.7. Detail: *match()* und *consume()* . . . 53
4.3.8. Vorrangregeln . . . 53
4.3.9. Linksrekursion . . . 54
4.3.10. Assoziativität . . . 54
4.3.11. LL(k)-Parser . . . 55
4.3.12. LL(k)-Parser: Implementierung mit Ringpuffer . . . 55
4.3.13. Wrap-Up . . . 56
4.4. Syntaxanalyse: LR-Parser (LR(0), LALR) . . . 57
4.4.1. Wiederholung . . . 57
4.4.2. Motivation . . . 57
4.4.3. Von unten nach oben . . . 57
4.4.4. Theorie: LR(0) . . . 58
4.4.5. Und was gibt es noch? . . . 61
4.4.6. Mehrdeutige Grammatiken . . . 61
4.4.7. Hierarchie der kontextfreien Sprachen . . . 62
4.4.8. Wrap-Up . . . 62

5. Semantische Analyse **64**
5.1. SymbTab0: Überblick Symboltabellen . . . 64
5.1.1. Was passiert nach der Syntaxanalyse? . . . 64
5.1.2. Semantische Analyse und Symboltabellen . . . 66
5.1.3. Syntax und Semantik . . . 66
5.1.4. Aufgaben der semantischen Analyse . . . 67
5.1.5. Einfache Verwaltung von Variablen primitiven Typs . . . 68
5.1.6. Was kann jetzt weiter passieren? . . . 70
5.1.7. Wo werden Verweise in Symboltabellen gebraucht? . . . 71
5.1.8. Grenzen der semantischen Analyse . . . 71
5.1.9. Wrap-Up . . . 71
5.2. SymbTab1: Nested Scopes . . . 72
5.2.1. Scopes und Name Spaces . . . 72
5.2.2. Symbole und (nested) Scopes . . . 72
5.2.3. Nested Scopes: Symbole und Scopes . . . 74
5.2.4. Nested Scopes: Definieren und Auflösen von Namen . . . 75
5.2.5. Nested Scopes: Listener . . . 75

5.2.6. Löschen von Symboltabellen . . . 77
5.2.7. Wrap-Up . . . 77
5.3. SymbTab2: Funktionen . . . 78
5.3.1. Funktionen und Scopes . . . 78
5.3.2. Erweiterung des Klassendiagramms für Funktions-Scopes . . . 79
5.3.3. Funktionen sind Symbole *und* Scopes . . . 79
5.3.4. Funktionen: Listener . . . 80
5.3.5. Wrap-Up . . . 81
5.4. SymbTab3: Strukturen und Klassen . . . 82
5.4.1. Strukturen . . . 82
5.4.2. Strukturen: Erweiterung der Symbole und Scopes . . . 83
5.4.3. Strukturen: Auflösen von Namen . . . 83
5.4.4. Klassen . . . 83
5.4.5. Klassen: Erweiterung der Symbole und Scopes . . . 85
5.4.6. Klassen: Auflösen von Namen . . . 85
5.4.7. Wrap-Up . . . 86
5.5. Typen, Type Checking und Attributierte Grammatiken . . . 88
5.5.1. Motivation . . . 88
5.5.2. Semantische Analyse . . . 88
5.5.3. Analyse von Datentypen . . . 89
5.5.4. Attributierte Grammatiken . . . 90
5.5.5. Syntax-gesteuerte Übersetzung: Attribute und Aktionen . . . 91
5.5.6. S-Attributgrammatiken und L-Attributgrammatiken . . . 92
5.5.7. Syntax-gesteuerte Übersetzung (SDT) . . . 96
5.5.8. Implementierung im rekursiven Abstieg . . . 97
5.5.9. Wrap-Up . . . 97

6. Interpreter **99**
6.1. AST-basierte Interpreter: Basics . . . 99
6.1.1. Aufgaben im Interpreter . . . 100
6.1.2. AST-basierte Interpreter: Visitor-Dispatcher . . . 100
6.1.3. Auswertung von Literalen und Ausdrücken . . . 101
6.1.4. Kontrollstrukturen . . . 102
6.1.5. Zustände: Auswerten von Anweisungen . . . 102
6.1.6. Detail: Felder im Interpreter . . . 103
6.1.7. Ausführen einer Variablendeklaration . . . 104
6.1.8. Ausführen einer Zuweisung . . . 104
6.1.9. Blöcke: Umgang mit verschachtelten Environments . . . 105
6.1.10. Wrap-Up . . . 105
6.2. AST-basierte Interpreter: Funktionen und Klassen . . . 106
6.2.1. Funktionen . . . 107
6.2.2. Ausführen einer Funktionsdeklaration . . . 107
6.2.3. Ausführen eines Funktionsaufrufs . . . 108
6.2.4. Funktionsaufruf: Rückgabewerte . . . 108

6.2.5. Native Funktionen . . . 109
6.2.6. Klassen und Instanzen I . . . 110
6.2.7. Klassen und Instanzen II . . . 111
6.2.8. Zugriff auf Methoden (und Attribute) . . . 111
6.2.9. Methoden und *this* oder *self* . . . 111
6.2.10. Wrap-Up . . . 112
6.3. Syntaxgesteuerte Interpreter . . . 113
6.3.1. Überblick Interpreter . . . 114
6.3.2. Syntaxgesteuerte Interpreter: Attributierte Grammatiken . . . 115
6.3.3. Eingebettete Aktionen in ANTLR I . . . 115
6.3.4. Eingebettete Aktionen in ANTLR II . . . 116
6.3.5. ANTLR: Traversierung des AST und Auslesen von Kontext-Objekten . . . 116
6.3.6. Wrap-Up . . . 121

7. Optimierung **122**
7.1. Optimierung und Datenflussanalyse . . . 122
7.1.1. Motivation . . . 122
7.1.2. Thema für heute: Optimierungen . . . 122
7.1.3. Algebraische Optimierung . . . 124
7.1.4. Maschinenunabhängige Optimierung . . . 125
7.1.5. Lokale Optimierung . . . 125
7.1.6. Globale Optimierung . . . 127
7.1.7. Maschinenabhängige Optimierung . . . 129
7.1.8. Optimierung zur Reduzierung des Energieverbrauchs . . . 130
7.1.9. Wrap-Up . . . 131

III. Praktikum **132**

8. Seminaristischer Unterricht: Vortragsthemen Kurzvortrag/Fachvortrag **134**
8.1. Kurzvortrag “PL Feature” (ca. 20 Minuten, DE) . . . 134
8.1.1. Verfügbare Themen . . . 134
8.1.2. Weitere mögliche Themen (nach Absprache) . . . 134
8.1.3. Hinweise . . . 134
8.2. Fachvortrag “Compiler” (ca. 60 Minuten + 10 Minuten Q&A, DE) . . . 135
8.2.1. Verfügbare Themen . . . 135
8.2.2. Weitere mögliche Themen (nach Absprache) . . . 136
8.2.3. Hinweise . . . 136

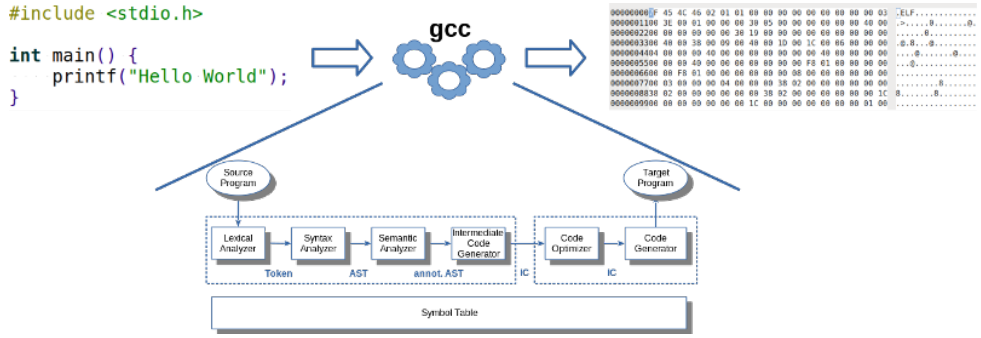
9. DSL-Projekt: Sprachen und Compiler **137**
9.1. Ziel und Rahmen . . . 137
9.2. Passende Themen (nicht abschließend) . . . 137
9.3. Projektanregungen . . . 138
9.3.1. Escape-Rooms mit MiniJava-Auswertung . . . 138
9.3.2. Einsteiger-IDE für Escape-Rooms . . . 139

9.3.3. Visualisierung von Programmabläufen 139
9.3.4. Quest-DSL 140
9.3.5. Compiler Bootstrapping 140
9.3.6. LaTeX Equation Language 141
9.3.7. MLIR/E-Graphs-Pipeline 141

Teil I.

Syllabus

1. Syllabus



1.1. Kursbeschreibung

Der Compiler ist das wichtigste Werkzeug in der Informatik. In der Königsdisziplin der Informatik schließt sich der Kreis, hier kommen die unterschiedlichen Algorithmen und Datenstrukturen und Programmiersprachenkonzepte zur Anwendung.

In diesem Modul geht es um ein fortgeschrittenes Verständnis für interessante Konzepte im Compilerbau sowie um grundlegende Konzepte von Programmiersprachen und -paradigmen. Wir schauen uns dazu relevante aktuelle Tools und Frameworks an und setzen diese bei der Erstellung eines Bytecode-Compilers für unterschiedliche Programmiersprachen für die Java-VM oder WASM ein.

1.2. Überblick Modulinhalte

1. Lexikalische Analyse: Scanner/Lexer
   - Reguläre Sprachen
   - Manuelle Implementierung, Parsergeneratoren (ANTLR, Flex, …)
2. Syntaxanalyse: Parser
   - Kontextfreie Grammatiken (CFG), Chomsky
   - LL-Parser (Top-Down-Parser)
   - LR-Parser (Bottom-Up-Parser)
   - Manuelle Implementierung, Parsergeneratoren (ANTLR, Bison, …)
3. Semantische Analyse und Optimierungen

    - Symboltabellen
    - Typen, Typ-Inferenz, Type Checking
    - Datenfluss- und Kontrollfluss-Analyse
    - Optimierungen: Peephole u.a.
4. Zwischencode: Intermediate Representation (IR), LLVM-IR
5. Interpreter: AST-Traversierung vs. Bytecode/VM, Garbage Collection
6. Code-Generierung
7. Programmiersprachen-Konzepte: OOP, FP, LP, CP u.a. und die Auswirkungen auf Compiler/Interpreter und Laufzeitumgebung

1.3. Team

- BC George
- Carsten Gips (Sprechstunde nach Vereinbarung)

1.4. Kursformat

Seminaristischer Unterricht (2 SWS)	Praktikum (3 SWS)
Fr, 08:45 - 10:15 Uhr (Zoom)	Fr, 10:30 - 12:45 Uhr (Zoom)

Durchführung des seminaristischen Unterrichts als *Flipped Classroom* (Carsten) bzw. als *reguläre Vorlesung* (BC).
Zugangsdaten Zoom siehe ILIAS.

1.5. Fahrplan

Monat	Woche (Fr)	Seminaristischer Unterricht	Praktikum	Edmonton/Minden-Meetings
Oktober	16.10.	Orga || Überblick | Sprachen | Anwendungen	-	
	23.10.	Reguläre Sprachen	CFG	
	30.10.	LL-Parser (Theorie)	Lexer (Implementierung) | LL-Parser (Implementierung)	

Monat	Woche (Fr)	Seminaristischer Unterricht	Praktikum	Edmonton/Minden-Meetings
November	06.11.	LR-Parser	**Vortrag**: Parsergeneratoren (ANTLR, Treesitter, Flex&Bison, …)	**Di, 03.11., 17:00 - 18:00 Uhr (online): ANTLR + Live-Coding**
	13.11.	Semantische Analyse: Intro | Scopes | Funktionen | Klassen	**Vortrag**: LALR, PEG, Pratt, Combinators	
	20.11.	**Vortrag**: Type Checking, Hindley-Milner	**Pitch DSL-Projekt**	
	27.11.	**Kurzvortrag**: OOP (Gabbrielli & Martini, Kap. 10)	**Kurzvortrag**: FP (Gabbrielli & Martini, Kap. 11)	
Dezember	04.12.	Interpreter 1 | Interpreter 2	-	**Mo, 30.11., 17:00 - 18:00 Uhr (online): Minden Presentations**: **Vorstellung "DSL-Projekt"**
	11.12.	**Vortrag**: VM & Bytecode	**Kurzvortrag**: LP (Gabbrielli & Martini, Kap. 12)	**Mo, 07.12., 17:00 - 18:00 Uhr (online): Edmonton Presentations**
	18.12.	Optimierung und Datenfluss- und Kontrollflussanalyse	**Kurzvortrag**: CP (Gabbrielli & Martini, Kap. 13)	
	25.12.	***Weihnachtspause***	-	
	01.01.	***Weihnachtspause***	-	
Januar	08.01.	**Vortrag**: Garbage Collection	**Vortrag**: JIT	
	15.01.	**Kurzvortrag**: Borrow Checking und Lifetimes (Rust)	**Kurzvortrag**: Dependent Type Systems (Idris)	
	22.01.	*Sprechstunde*	*Freies Arbeiten*	
	29.01.	**Vorträge DSL-Projekt**	**Vorträge DSL-Projekt**	

- Zu den Talks …
- Zum DSL-Projekt …

1.6. Prüfungsform, Note und Credits

Mündliche Prüfung plus Studienleistung (Portfolio), 10 ECTS

1.6.1. Studienleistung: “Portfolio”

Die Studienleistung ist eine unbenotete Leistung und setzt sich aus mehreren Komponenten zusammen:

1. **Projekt-Pitch** (Vorstellung der Konzepte für das DSL-Projekt): Freitag, 20.11., ca. 20 Minuten (pro Team); **Exposé** (s.u.) bis zum 19.11.
2. Teilnahme an **mind. zwei Edmonton/Minden-Terminen** mit aktiver Beteiligung, pro Team ist am zweiten Treffen ein Vortrag zum DSL-Projekt ca. 45 Minuten zu halten (Englisch!)
    - Termin 1: Dienstag, 03.11., 17:00 - 18:00 Uhr (online)
    - **Termin 2**: Montag, 30.11., 17:00 - 18:00 Uhr (online): **Vorstellung der DSL-Projekte** (Projektvortrag 1), ca. 40-45 Minuten pro Team (**Englisch**)
    - Termin 3: Montag, 07.12., 17:00 - 18:00 Uhr (online)
3. **Kurzvortrag** “PL Features” ca. 20 Minuten (pro Team) plus Diskussionsleitung
4. **Fachvortrag** “Compiler” ca. 60 Minuten (pro Team)
5. **Abschlusspräsentation** zum DSL-Projekt (Projektvortrag 2) am Semesterende (Freitag, 29.01.) ca. 30 Minuten (pro Team)

Zu diesen Leistungen soll ein **Lerntagebuch** (s.u.) geführt und abgegeben werden (**jede Person individuell**).

Das Exposé, die Slides (Edmonton-Talk, Kurzvortrag, Fachvortrag, Abschlusspräsentation), das Projekt (via GitHub-Link im Lerntagebuch) und das Lerntagebuch sind im **ILIAS** als PDF abzugeben. Bitte beachtet die jeweiligen Abgabefristen im ILIAS!

1.6.2. Gesamtnote: Mündliche Prüfung (einzeln, ca. 45 Minuten)

Die Modul-Note ergibt sich aus der Leistung in der mündlichen Prüfung.

Sie können die Prüfung in der ersten oder in der zweiten Prüfungsphase ablegen. Die mündliche Prüfung wird über Zoom durchgeführt und dauert ca. 45 Minuten.

1.6.3. Hinweise

- Die mündliche Prüfung und die Studienleistung sind zwei unabhängige Prüfungsleistungen und werden separat im CAT angemeldet. Die Studienleistung schaltet die Credits für das Modul frei, die mündliche Prüfung ergibt die Modulnote. Die Studienleistung ist **keine** Vorleistung für die mündliche Prüfung.
- Die Bearbeitung der Leistungen erfolgt im Team.
- Ein Team umfasst 3 Personen.
- Es gibt keine Aufgabenblätter. Stattdessen haben wir verschiedene Vorträge und das DSL-Projekt.
- Das Lerntagebuch ist individuell zu erstellen und abzugeben.

- “Aktive Beteiligung” umfasst Anwesenheit und sachbezogene Beiträge; Anwesenheit/Beteiligung werden dokumentiert.

- **Exposé** (pro Team)

  - Umfang: 150-400 Wörter
  - Struktur:

    1. Was wollt ihr machen?
    2. Warum ist das spannend?
    3. Wie werdet ihr das umsetzen?
    4. Wie könnt ihr euren Erfolg empirisch bewerten?
    5. Wer ist alles im Team?

  Bitte beschreiben Sie die einzelnen Punkte so ausführlich wie nötig, um nachvollziehbar zu sein.

  Abgabe als PDF im ILIAS, spätestens einen Tag vor der internen Projektvorstellung.

- **Lerntagebuch**: Jede Person beschreibt individuell(!) die Bearbeitung der Studienleistung (auch die Teilnahme an den Edmonton/Minden-Meetings) zurückblickend mit mind. 750 bis max. 2000 Wörtern (Nutzlast! Überschriften und Links zählen nicht mit). Gehen Sie dabei aussagekräftig und nachvollziehbar auf folgende Punkte ein:

  1. **Zusammenfassung**: Was wurde gemacht bzw. was wurde auf dem Meeting besprochen?
  2. **Details**: Kurze Beschreibung besonders interessanter Aspekte.
  3. **Reflexion**: Was war der schwierigste Teil? Wie haben Sie dieses Problem gelöst?
  4. **Reflexion**: Was haben Sie gelernt oder (besser) verstanden?
  5. **Team**: Mit wem haben Sie zusammengearbeitet?
  6. **Link zu Ihrem Repo** mit den relevanten Artefakten (Lösung, Slides für den Vortrag, …).

  Für die Edmonton/Minden-Meetings passen Sie bitte die Punkte (1) bis (4) und (5) entsprechend inhaltlich an, (6) entfällt.

  Das Lerntagebuch geben Sie bitte pro Person bis spätestens zur letzten gemeinsamen Sitzung im ILIAS ab.

1.7. Materialien

1. “**Compilers: Principles, Techniques, and Tools**”. Aho, A. V. und Lam, M. S. und Sethi, R. und Ullman, J. D. and Bansal, S., Pearson India, 2023. ISBN 978-9-3570-5488-1. Online über die O’Reilly-Lernplattform.
2. “**Crafting Interpreters**”. Nystrom, R., Genever Benning, 2021. ISBN 978-0-9905829-3-9. Online.
3. “**Engineering a Compiler**”. Torczon, L. und Cooper, K., Morgan Kaufmann, 2012. ISBN 978-0-1208-8478-0. Online über die O’Reilly-Lernplattform.
4. “Introduction to Compilers and Language Design”. Thain, D., 2023. ISBN 979-8-655-18026-0. Online.
5. “Writing a C Compiler”. Sandler, N., No Starch Press, 2024. ISBN 978-1-0981-8222-9. Online über die O’Reilly-Lernplattform.
6. “**Seven Languages in Seven Weeks**”. Tate, B.A., Pragmatic Bookshelf, 2010. ISBN 978-1-93435-659-3. Online über die O’Reilly-Lernplattform.

1.8. Förderungen und Kooperationen

1.8.1. Kooperation mit University of Alberta, Edmonton (Kanada)

Über das Projekt "We CAN virtuOWL" der Fachhochschule Bielefeld ist im Frühjahr 2021 eine Kooperation mit der University of Alberta (Edmonton/Alberta, Kanada) im Modul "Compilerbau" gestartet.

Wir freuen uns, auch in diesem Semester wieder drei gemeinsame Sitzungen für beide Hochschulen anbieten zu können. (Diese Termine werden in englischer Sprache durchgeführt.)

1.9. LICENSE



Unless otherwise noted, this work by BC George, Carsten Gips and contributors is licensed under CC BY-SA 4.0. See the credits for a detailed list of contributing projects.

Teil II.

Vorlesungsunterlagen

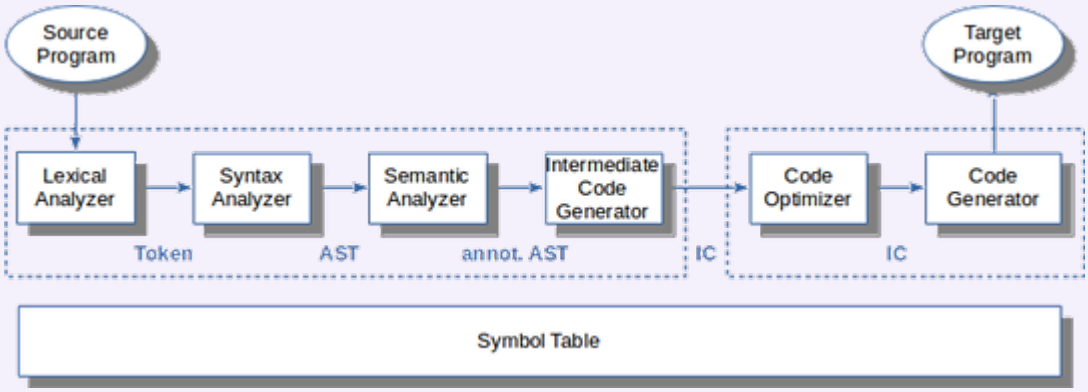
2. Überblick

Was ist ein Compiler? Welche Bausteine lassen sich identifizieren, welche Aufgaben haben diese?

2.1. Struktur eines Compilers

TL;DR

Compiler übersetzen (formalen) Text in ein anderes Format.
Typischerweise kann man diesen Prozess in verschiedene Stufen/Phasen einteilen. Dabei verarbeitet jede Phase den Output der vorangegangenen Phase und erzeugt ein (kompakteres) Ergebnis, welches an die nächste Phase weitergereicht wird. Dabei nimmt die Abstraktion von Stufe zu Stufe zu: Der ursprüngliche Input ist ein Strom von Zeichen, daraus wird ein Strom von Wörtern (Token), daraus ein Baum (Parse Tree), Zwischencode (IC), …



Die gezeigten Phasen werden traditionell unterschieden. Je nach Aufgabe können verschiedene Stufen zusammengefasst werden oder sogar gar nicht auftreten.

Videos

Vorlesung [YT], [HSBI]

2.1.1. Sprachen verstehen, Texte transformieren

> The cat runs quickly.

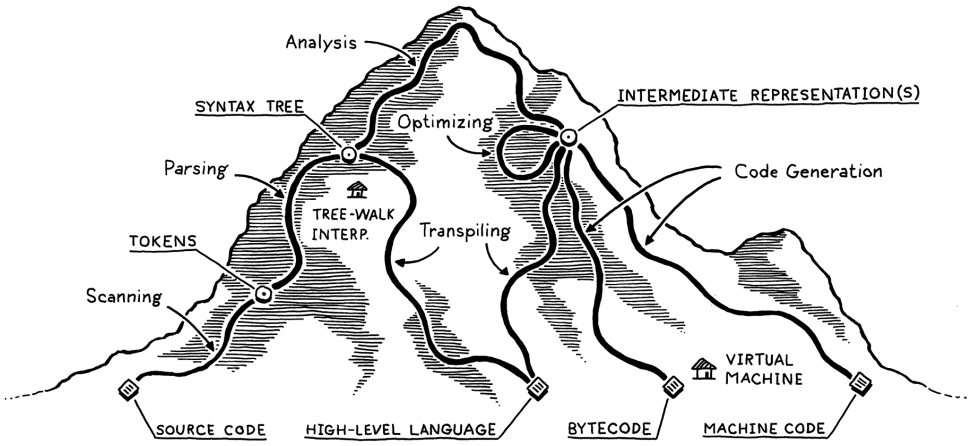
=> Struktur? Bedeutung?

Wir können hier (mit steigender Abstraktionsstufe) unterscheiden:

- Sequenz von Zeichen
- Wörter: Zeichenketten mit bestimmten Buchstaben, getrennt durch bestimmte andere Zeichen; Wörter könnten im Wörterbuch nachgeschlagen werden
- Sätze: Anordnung von Wörtern nach einer bestimmten Grammatik, Grenze: Satzzeichen

  Hier (vereinfacht): Ein Satz besteht aus Subjekt und Prädikat. Das Subjekt besteht aus einem oder keinen Artikel und einem Substantiv. Das Prädikat besteht aus einem Verb und einem oder keinem Adverb.
- Sprache: Die Menge der in einer Grammatik erlaubten Sätze

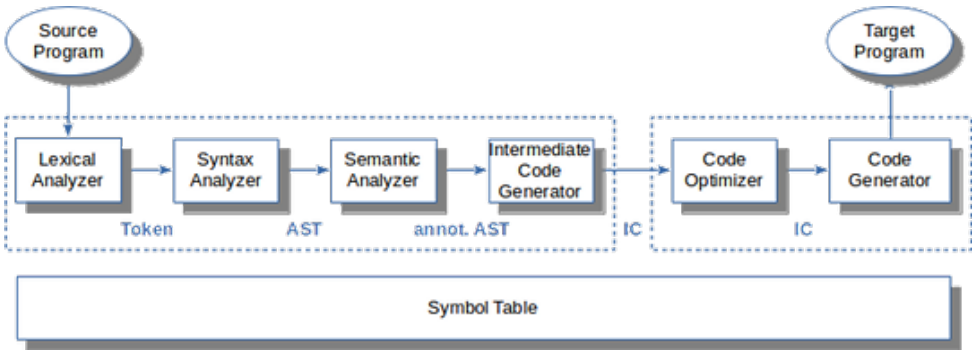
2.1.2. Compiler: Big Picture



Quelle: A Map of the Territory (mountain.png) by Bob Nystrom on Github.com (MIT)

Begriffe und Phasen

Die obige Bergsteige-Metapher kann man in ein nüchternes Ablaufdiagramm mit verschiedenen Stufen und den zwischen den Stufen ausgetauschten Artefakten übersetzen:



2.1.2.1. Frontend, Analyse

Die ersten Stufen eines Compilers, die mit der **Analyse** des Inputs beschäftigt sind. Dies sind in der Regel der Scanner, der Parser und die semantische Analyse.

- Scanner, Lexer, Tokenizer, Lexikalische Analyse

  Zerteilt den Zeichenstrom in eine Folge von Wörtern. Mit regulären Ausdrücken kann definiert werden, was Klassen gültiger Wörter (“Token”) sind. Ein Token hat i.d.R. einen Namen und einen Wert.

- Parser, Syntaxanalyse

  Der Parser erhält als Eingabe die Folge der Token und versucht mit Hilfe einer Grammatik zu bestimmen, ob es sich bei der Tokensequenz um gültige Sätze im Sinne der Grammatik handelt. Hier gibt es viele Algorithmen, die im Wesentlichen in die Klassen “top-down” und “bottom-up” fallen.

- Semantische Analyse, Kontexthandling

  In den vorigen Stufen wurde eher lokal gearbeitet. Hier wird über den gesamten Baum und die Symboltabelle hinweg geprüft, ob beispielsweise Typen korrekt verwendet wurden, in welchen Scope ein Name gehört etc. Mit diesen Informationen wird der AST angereichert.

- Symboltabellen

  Datenstrukturen, um Namen, Werte, Scopes und weitere Informationen zu speichern. Die Symboltabellen werden vor allem beim Parsen befüllt und bei der semantischen Analyse gelesen, aber auch der Lexer benötigt u.U. diese Informationen.

2.1.2.2. Backend, Synthese

Die hinteren Stufen eines Compilers, die mit der **Synthese** der Ausgabe beschäftigt sind. Dies sind in der Regel verschiedene Optimierungen und letztlich die Code-Generierung

- Codegenerierung

  Erzeugung des Zielprogramms aus der (optimierten) Zwischendarstellung. Dies ist oft Maschinencode, kann aber auch C-Code oder eine andere Ziel-Sprache sein.

- Optimierung

  Diverse Maßnahmen, um den resultierenden Code kleiner und/oder schneller zu gestalten.

- Symboltabellen

  Datenstrukturen, um Namen, Werte, Scopes und weitere Informationen zu speichern. Die Symboltabellen werden vor allem beim Parsen befüllt und bei der semantischen Analyse gelesen, aber auch der Lexer benötigt u.U. diese Informationen.

2.1.2.3. Weitere Begriffe

- Parse Tree, Concrete Syntax Tree

Repräsentiert die Struktur eines Satzes, wobei jeder Knoten dem Namen einer Regel der Grammatik entspricht. Die Blätter bestehen aus den Token samt ihren Werten.

- AST, (Abstract) Syntax Tree

  Vereinfachte Form des Parse Tree, wobei der Bezug auf die Element der Grammatik (mehr oder weniger) weggelassen wird.

- Annotierter AST

  Anmerkungen am AST, die für spätere Verarbeitungsstufen interessant sein könnten: Typ-Informationen, Optimierungsinformationen, …

- Zwischen-Code, IC

  Zwischensprache, die abstrakter ist als die dem AST zugrunde liegenden Konstrukte der Ausgangssprache. Beispielsweise könnten `while`-Schleifen durch entsprechende Label und Sprünge ersetzt werden. Wie genau dieser Zwischen-Code aussieht, muss der Compilerdesigner entscheiden. Oft findet man den Assembler-ähnlichen "3-Adressen-Code".

- Sprache

  Eine Sprache ist eine Menge gültiger Sätze. Die Sätze werden aus Wörtern gebildet, diese wiederum aus Zeichenfolgen.

- Grammatik

  Eine Grammatik beschreibt formal die Syntaxregeln für eine Sprache. Jede Regel in der Grammatik beschreibt dabei die Struktur eines Satzes oder einer Phrase.

2.1.3. Lexikalische Analyse: Wörter ("*Token*") erkennen

Die lexikalische Analyse (auch *Scanner* oder *Lexer* oder *Tokenizer* genannt) zerteilt den Zeichenstrom in eine Folge von Wörtern ("*Token*"). Die geschieht i.d.R. mit Hilfe von *regulären Ausdrücken*.

Dabei müssen unsinnige/nicht erlaubte Wörter erkannt werden.

Überflüssige Zeichen (etwa Leerzeichen) werden i.d.R. entfernt.

```
1  sp = 100;
```

```
1  <ID, sp>, <OP, =>, <INT, 100>, <SEM>
```

Anmerkung: In der obigen Darstellung werden die Werte der Token ("*Lexeme*") zusammen mit den Token "gespeichert". Alternativ können die Werte der Token auch direkt in der Symboltabelle gespeichert werden und in den Token nur der Verweis auf den jeweiligen Eintrag in der Tabelle.

2.1.4. Syntaxanalyse: Sätze erkennen

In der Syntaxanalyse (auch *Parser* genannt) wird die Tokensequenz in gültige Sätze unterteilt. Dazu werden in der Regel *kontextfreie Grammatiken* und unterschiedliche Parsing-Methoden (*top-down*, *bottom-up*) genutzt.

Dabei müssen nicht erlaubte Sätze erkannt werden.

```
<ID, sp>, <OP, =>, <INT, 100>, <SEM>
```

```
statement : assign SEM ;
assign : ID OP INT ;
```

```
                    statement                    =
                    /       \                  / \
                assign      SEM              sp  100
              /   |   \      |
            ID    OP  INT    ;
            |     |    |
            sp    =   100
```

Mit Hilfe der Produktionsregeln der Grammatik wird versucht, die Tokensequenz zu erzeugen. Wenn dies gelingt, ist der Satz (also die Tokensequenz) ein gültiger Satz im Sinne der Grammatik. Dabei sind die Token aus der lexikalischen Analyse die hier betrachteten Wörter!

Dabei entsteht ein sogenannter *Parse-Tree* (oder auch "*Syntax Tree*"; in der obigen Darstellung der linke Baum). In diesen Bäumen spiegeln sich die Regeln der Grammatik wider, d.h. zu einem Satz kann es durchaus verschiedene Parse-Trees geben.

Beim *AST* ("*Abstract Syntax Tree*") werden die Knoten um alle später nicht mehr benötigten Informationen bereinigt (in der obigen Darstellung der rechte Baum).

Anmerkung: Die Begriffe werden oft nicht eindeutig verwendet. Je nach Anwendung ist das Ergebnis des Parsers ein AST oder ein Parse-Tree.

Anmerkung: Man könnte statt `OP` auch etwa ein `ASSIGN` nutzen und müsste dann das "=" nicht extra als Inhalt speichern, d.h. man würde die Information im Token-Typ kodieren.

2.1.5. Vorschau: Parser implementieren

```
stat : assign | ifstat | ... ;
assign : ID '=' expr ';' ;
```

```
void stat() {
    switch (<<current token>>) {
        case ID : assign(); break;
        case IF : ifstat(); break;
        ...
        default : <<raise exception>>
    }
}
void assign() {
    match(ID);
    match('=');
    expr();
    match(';');
}
```

Der gezeigte Parser ist ein sogenannter “LL(1)”-Parser und geht von oben nach unten vor, d.h. ist ein Top-Down-Parser.

Nach dem Betrachten des aktuellen Tokens wird entschieden, welche Alternative vorliegt und in die jeweilige Methode gesprungen.

Die `match()`-Methode entspricht dabei dem Erzeugen von Blättern, d.h. hier werden letztlich die Token der Grammatik erkannt.

2.1.6. Semantische Analyse: Bedeutung erkennen

In der semantischen Analyse (auch *Context Handling* genannt) wird der AST zusammen mit der Symboltabelle geprüft. Dabei spielen Probleme wie Scopes, Namen und Typen eine wichtige Rolle.

Die semantische Analyse ist direkt vom Programmierparadigma der zu übersetzenden Sprache abhängig, d.h. müssen wir beispielsweise das Konzept von Klassen verstehen?

Als Ergebnis dieser Phase entsteht typischerweise ein *annotierter AST*.

```
{
    int x = 42;
    {
        int x = 7;
        x += 3;    // ???
    }
}
```

```
                                                    = {type: real, loc: tmp1}
sp = 100;                                          / \
                                                  /   \
                                               sp     inttofloat
                                        {type: real,       |
                                         loc: var b}      100
```

2.1.7. Zwischencode generieren

Aus dem annotierten AST wird in der Regel ein Zwischencode (“*Intermediate Code*”, auch “IC”) generiert. oft findet man hier den Assembler-ähnlichen “3-Adressen-Code”, in manchen Compilern wird als IC aber auch der AST selbst genutzt.

```
                  = {type: real, loc: tmp1}
                 / \
                /   \
             sp     inttofloat
      {type: real,       |
       loc: var b}      100
```

=> `t1 = inttofloat(100)`

2.1.8. Code optimieren

An dieser Stelle verlassen wir das Compiler-Frontend und begeben uns in das sogenannte *Backend*. Die Optimierung des Codes kann sehr unterschiedlich ausfallen, beispielsweise kann man den Zwischencode selbst optimieren, dann nach sogenanntem "Targetcode" übersetzen und diesen weiter optimieren, bevor das Ergebnis im letzten Schritt in Maschinencode übersetzt wird.

Die Optimierungsphase ist sehr stark abhängig von der Zielhardware. Hier kommen fortgeschrittene Mengen- und Graphalgorithmen zur Anwendung. Die Optimierung stellt den wichtigsten Teil aktueller Compiler dar.

Aus zeitlichen und didaktischen Gründen werden wir in dieser Veranstaltung den Fokus auf die Frontend-Phasen legen und die Optimierung nur grob streifen.

```
t1 = inttofloat(100) => t1 = 100.0
```

```
x = y*0; => x = 0;
```

2.1.9. Code generieren

- Maschinencode:

```
1  STD  t1, 100.0
```

- Andere Sprache:
  - Bytecode
  - C
  - …

2.1.10. Probleme

```
1  5*4+3
```

AST?

Problem: Vorrang von Operatoren

- Variante 1: `+(*(5, 4), 3)`
- Variante 2: `*(5, +(4, 3))`

```
1  stat : expr ';'
2       | ID '(' ')' ';'
3       ;
4  expr : ID '(' ')'
5       | INT
6       ;
```

2.1.11. Unbedingt lesenswert

Sie sollten diese beiden Paper unbedingt als Einstieg in das Modul lesen:

1. Programming Language Semantics
2. An Incremental Approach to Compiler Construction

2.1.12. Wrap-Up

- Compiler übersetzen Text in ein anderes Format

- Typische Phasen:
  1. Lexikalische Analyse
  2. Syntaxanalyse
  3. Semantische Analyse
  4. Generierung von Zwischencode
  5. Optimierung des (Zwischen-) Codes
  6. Codegenerierung

Zum Nachlesen

- Aho u. a. (2023): Kapitel 1 Introduction
- Grune u. a. (2012): Kapitel 1 Introduction

Lernziele

- k2: Ich kann die Struktur eines Compilers und die verschiedenen Phasen und deren Aufgaben erklären

2.2. Bandbreite der Programmiersprachen

TL;DR

Am Beispiel des Abzählreims “99 Bottles of Beer” werden (ganz kurz) verschiedene Programmiersprachen betrachtet. Jede der Sprachen hat ihr eigenes Sprachkonzept (Programmierparadigma) und auch ein eigenes Typ-System sowie ihre eigene Strategie zur Speicherverwaltung und Abarbeitung.
Auch wenn die Darstellung längst nicht vollständig ist, macht sie doch deutlich, dass Compiler teilweise sehr unterschiedliche Konzepte “verstehen” müssen.

Videos

Vorlesung [YT], [HSBI]

2.2.1. 99 Bottles of Beer

> 99 bottles of beer on the wall, 99 bottles of beer. Take one down and pass it around, 98 bottles of beer on the wall.
>
> 98 bottles of beer on the wall, 98 bottles of beer. Take one down and pass it around, 97 bottles of beer on the wall.
>
> [...]
>
> 2 bottles of beer on the wall, 2 bottles of beer. Take one down and pass it around, 1 bottle of beer on the wall.
>
> 1 bottle of beer on the wall, 1 bottle of beer. Take one down and pass it around, no more bottles of beer on the wall.
>
> No more bottles of beer on the wall, no more bottles of beer. Go to the store and buy some more, 99 bottles of beer on the wall.

Quelle: Abzählreim "99 Bottles of Beer" nach "Lyrics of the song 99 Bottles of Beer" on 99-bottles-of-beer.net

2.2.2. Imperativ, Hardwarenah: C

```c
#define MAXBEER (99)
void chug(int beers);
main() {
   register beers;
   for(beers = MAXBEER; beers; chug(beers--))  puts("");
   puts("\nTime to buy more beer!\n");
}
void chug(register beers) {
   char howmany[8], *s;
   s = beers != 1 ? "s" : "";
   printf("%d bottle%s of beer on the wall,\n", beers, s);
   printf("%d bottle%s of beeeeer . . . ,\n", beers, s);
   printf("Take one down, pass it around,\n");
   if(--beers) sprintf(howmany, "%d", beers); else strcpy(howmany, "No more");
   s = beers != 1 ? "s" : "";
   printf("%s bottle%s of beer on the wall.\n", howmany, s);
}
```

Quelle: "Language C" by Bill Wein on 99-bottles-of-beer.net

- Imperativ
- Procedural
- Statisches Typsystem
- Resourcenschonend, aber "unsicher": Programmierer muss wissen, was er tut
- Relativ hardwarenah
- Einsatz: Betriebssysteme, Systemprogrammierung

2.2.3. Imperativ, Objektorientiert: Java

```java
class bottles {
    public static void main(String args[]) {
        String s = "s";
        for (int beers=99; beers>-1;) {
            System.out.print(beers + " bottle" + s + " of beer on the wall, ");
            System.out.println(beers + " bottle" + s + " of beer, ");
            if (beers==0) {
                System.out.print("Go to the store, buy some more, ");
                System.out.println("99 bottles of beer on the wall.\n");
                System.exit(0);
            } else
                System.out.print("Take one down, pass it around, ");
            s = (--beers == 1)?"":"s";
            System.out.println(beers + " bottle" + s + " of beer on the wall.\n
                ");
        }
    }
}
```

Quelle: "Language Java" by Sean Russell on 99-bottles-of-beer.net

- Imperativ
- Objektorientiert
- Multi-Threading
- Basiert auf C/C++
- Statisches Typsystem
- Automatische Garbage Collection
- "Sichere" Architektur: Laufzeitumgebung fängt viele Probleme ab
- Architekturneutral: Nutzt Bytecode und eine JVM
- Einsatz: High-Level All-Purpose Language

2.2.4. Logisch: Prolog

```prolog
bottles :-
    bottles(99).

bottles(1) :-
    write('1 bottle of beer on the wall, 1 bottle of beer,'), nl,
    write('Take one down, and pass it around,'), nl,
    write('Now they are all gone.'), nl,!.
bottles(X) :-
    write(X), write(' bottles of beer on the wall,'), nl,
    write(X), write(' bottles of beer,'), nl,
    write('Take one down and pass it around,'), nl,
    NX is X - 1,
```

```
13      write(NX), write(' bottles of beer on the wall.'), nl, nl,
14      bottles(NX).
```

Quelle: "Language Prolog" by M@ on 99-bottles-of-beer.net

- Deklarativ
- Logisch: Definition von Fakten und Regeln; eingebautes Beweissystem
- Einsatz: Theorem-Beweisen, Natural Language Programming (NLP), Expertensysteme, …

2.2.5. Funktional: Haskell

```
1  bottles 0 = "no more bottles"
2  bottles 1 = "1 bottle"
3  bottles n = show n ++ " bottles"
4
5  verse 0   = "No more bottles of beer on the wall, no more bottles of beer.\n"
6            ++ "Go to the store and buy some more, 99 bottles of beer on the wall.
               "
7
8  verse n   = bottles n ++ " of beer on the wall, " ++ bottles n ++ " of beer.\n"
9            ++ "Take one down and pass it around, " ++ bottles (n-1)
10                                                   ++ " of beer on the wall.\n"
11
12 main      = mapM (putStrLn . verse) [99,98..0]
```

Quelle: "Language Haskell" by lavor on 99-bottles-of-beer.net

- Deklarativ
- Funktional
- Lazy, pure
- Statisches Typsystem
- Typinferenz
- Algebraische Datentypen, Patternmatching
- Einsatz: Compiler, DSL, Forschung

2.2.6. Brainfuck



Quelle: Screenshot of "Language Brainfuck" by Michal Wojciech Tarnowski on 99-bottles-of-beer.net

- Imperativ
- Feldbasiert (analog zum Band der Turingmaschine)
- 8 Befehle: Zeiger und Zellen inkrementieren/dekrementieren, Aus- und Eingabe, Sprungbefehle

2.2.7. Programmiersprache Lox

```
fun fib(x) {
    if (x == 0) {
        return 0;
    } else {
        if (x == 1) {
            return 1;
        } else {
            fib(x - 1) + fib(x - 2);
        }
    }
}

var wuppie = fib;
wuppie(4);
```

- Die Sprache “Lox” finden Sie hier: craftinginterpreters.com/the-lox-language.html
- C-ähnliche Syntax
- Imperativ, objektorientiert, Funktionen als *First Class Citizens*, Closures

- Dynamisch typisiert
- Garbage Collector
- Statements und Expressions
- (Kleine) Standardbibliothek eingebaut

Die Sprache ähnelt stark anderen modernen Sprachen und ist gut geeignet, um an ihrem Beispiel Themen wie Scanner/Parser/AST, Interpreter, Object Code und VM zu studieren :)

2.2.8. Wrap-Up

- Compiler übersetzen formalen Text in ein anderes Format

- Berücksichtigung von unterschiedlichen
  - Sprachkonzepten (Programmierparadigmen)
  - Typ-Systemen
  - Speicherverwaltungsstrategien
  - Abarbeitungsstrategien

Zum Nachlesen

- Aho u. a. (2023): Kapitel 1 Introduction
- Grune u. a. (2012): Kapitel 1 Introduction

Lernziele

- k1: Ich kenne verschiedene Beispiele für verschiedene Programmiersprachen und Paradigmen

2.3. Anwendungen

TL;DR

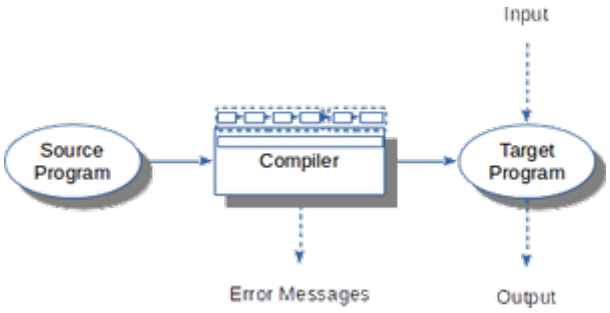
Es gibt verschiedene Anwendungsmöglichkeiten für Compiler. Je nach Bedarf wird dabei die komplette Toolchain durchlaufen oder es werden Stufen ausgelassen. Häufig genutzte Varianten sind dabei:

- “Echte” Compiler: Übersetzen Sourcecode nach ausführbarem Maschinencode
- Interpreter: Interaktive Ausführung von Sourcecode
- Virtuelle Maschinen als Zwischending zwischen Compiler und Interpreter
- Transpiler: Übersetzen formalen Text nach formalem Text
- Analysetools: Parsen den Sourcecode, werten die Strukturen aus

Videos

Vorlesung [YT], [HSBI]

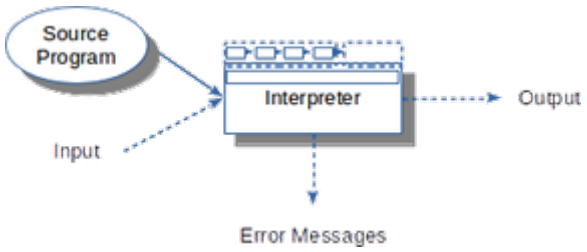
2.3.1. Anwendung: Compiler



Wie oben diskutiert: Der Sourcecode durchläuft alle Phasen des Compilers, am Ende fällt ein ausführbares Programm heraus. Dieses kann man starten und ggf. mit Inputdaten versehen und erhält den entsprechenden Output. Das erzeugte Programm läuft i.d.R. nur auf einer bestimmten Plattform.

Beispiele: gcc, clang, …

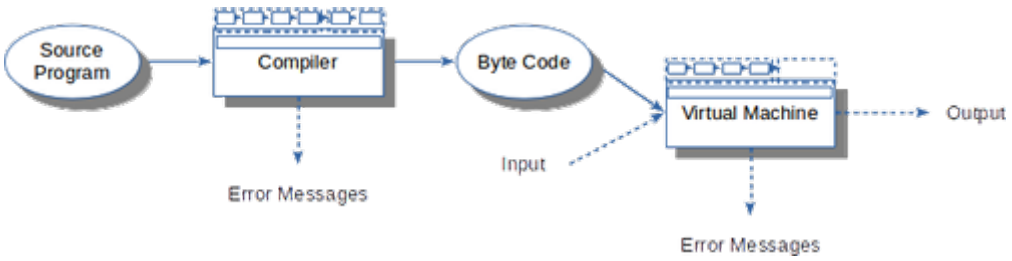
2.3.2. Anwendung: Interpreter



Beim Interpreter durchläuft der Sourcecode nur das Frontend, also die Analyse. Es wird kein Code erzeugt, stattdessen führt der Interpreter die Anweisungen im AST bzw. IC aus. Dazu muss der Interpreter mit den Eingabedaten beschickt werden. Typischerweise hat man hier eine “Read-Eval-Print-Loop” (*REPL*).

Beispiele: Python

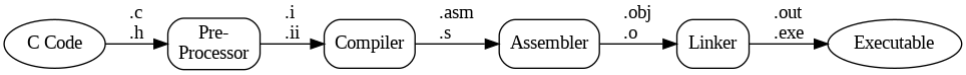
2.3.3. Anwendung: Virtuelle Maschinen



Hier liegt eine Art Mischform aus Compiler und Interpreter vor: Der Compiler übersetzt den Quellcode in ein maschinenunabhängiges Zwischenformat (“Byte-Code”). Dieser wird von der virtuellen Maschine (“VM”) gelesen und ausgeführt. Die VM kann also als Interpreter für Byte-Code betrachtet werden.

Beispiel: Java mit seiner JVM

2.3.4. Anwendung: C-Toolchain

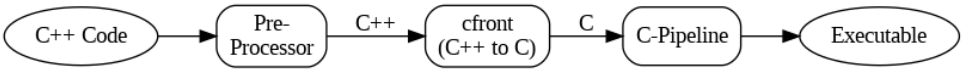


Erinnern Sie sich an die LV “Systemprogrammierung” im dritten Semester :-)

Auch wenn es so aussieht, als würde der C-Compiler aus dem Quelltext direkt das ausführbare Programm erzeugen, finden hier dennoch verschiedene Stufen statt. Zuerst läuft ein Präprozessor über den Quelltext und ersetzt alle `#include` und `#define` etc., danach arbeitet der C-Compiler, dessen Ausgabe wiederum durch einen Assembler zu ausführbarem Maschinencode transformiert wird.

Beispiele: gcc, clang, …

2.3.5. Anwendung: C++-Compiler



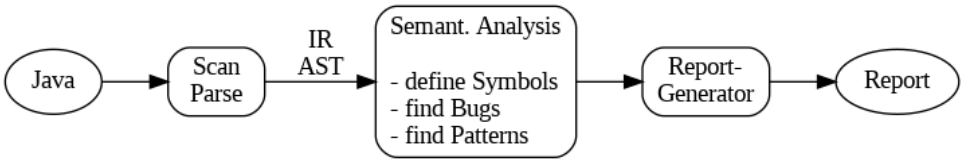
C++ hat meist keinen eigenen (vollständigen) Compiler :-)

In der Regel werden die C++-Konstrukte durch `cfront` nach C übersetzt, so dass man anschließend auf die etablierten Tools zurückgreifen kann.

Dieses Vorgehen werden Sie relativ häufig finden. Vielleicht sogar in Ihrem Projekt …

Beispiel: g++

2.3.6. Anwendung: Bugfinder



Tools wie FindBugs analysieren den (Java-) Quellcode und suchen nach bekannten Fehlermustern. Dazu benötigen sie nur den Analyse-Teil eines Compilers!

Auf dem AST kann dann nach vorab definierten Fehlermustern gesucht werden (Stichwort “Graphmatching”). Dazu fällt die semantische Analyse entsprechend umfangreicher aus als normal.

Zusätzlich wird noch eine Reporting-Komponente benötigt, da die normalen durch die Analysekette erzeugten Fehlermeldungen nicht helfen (bzw. sofern der Quellcode wohlgeformter Code ist, würden ja keine Fehlermeldungen durch die Analyseeinheit generiert).

Beispiele: SpotBugs, Checkstyle, ESLint, …

2.3.7. Anwendung: Pandoc

Pandoc ist ein universeller und modular aufgebauter Textkonverter, der mit Hilfe verschiedener *Reader* unterschiedliche Textformate einlesen und in ein Zwischenformat (hier JSON) transformieren kann. Über verschiedene *Writer* können aus dem Zwischenformat dann Dokumente in den gewünschten Zielformaten erzeugt werden.

Die Reader entsprechen der Analyse-Phase und die Writer der Synthese-Phase eines Compilers. Anstelle eines ausführbaren Programms (Maschinencode) wird ein anderes Textformat erstellt/ausgegeben.

Beispielsweise wird aus diesem Markdown-Schnipsel …

```
Dies ist ein Satz mit
*  einem Stichpunkt, und
*  einem zweiten Stichpunkt.
```

… dieses Zwischenformat erzeugt, …

```
{"blocks":[{"t":"Para","c":[{"t":"Str","c":"Dies"},{"t":"Space"},
           {"t":"Str","c":"ist"},{"t":"Space"},{"t":"Str","c":"ein"},
           {"t":"Space"},{"t":"Str","c":"Satz"},{"t":"Space"},
           {"t":"Str","c":"mit"}]},
           {"t":"BulletList","c":[[{"t":"Plain","c":[{"t":"Str","c":"einem"},{"
               t":"Space"},{"t":"Str","c":"Stichpunkt,"},{"t":"Space"},{"t":"Str
               ","c":"und"}]}],[{"t":"Plain","c":[{"t":"Str","c":"einem"},{"t":"
               Space"},{"t":"Str","c":"zweiten"},{"t":"Space"},{"t":"Str","c":"
               Stichpunkt."}]}]]}],"pandoc-api-version":[1,17,0,4],"meta":{}}
```

… und daraus schließlich dieser TeX-Code.

```
Dies ist ein Satz mit
\begin{itemize}
\tightlist
\item einem Stichpunkt, und
\item einem zweiten Stichpunkt.
\end{itemize}
```

Im Prinzip ist Pandoc damit ein Beispiel für Compiler, die aus einem formalen Text nicht ein ausführbares Programm erzeugen (Maschinencode), sondern einen anderen formalen Text. Dieser werden häufig auch “Transpiler” genannt.

Weitere Beispiele:

- Lexer-/Parser-Generatoren: ANTLR, Flex, Bison, …: formale Grammatik nach Sourcecode

- CoffeeScript: CoffeeScript (eine Art “JavaScript light”) nach JavaScript
- Emscripten: C/C++ nach LLVM nach WebAssembly (tatsächlich kann LLVM-IR auch direkt als Input verwendet werden)
- Fitnesse: Word/Wiki nach ausführbare Unit-Tests

2.3.8. Was bringt mir das?

Beschäftigung mit dem schönsten Thema in der Informatik ;-)

2.3.8.1. Auswahl einiger Gründe für den Besuch des Moduls “Compilerbau”

- Erstellung eigener kleiner Interpreter/Compiler
  - Einlesen von komplexen Daten
  - DSL als Brücke zwischen Stakeholdern
  - DSL zum schnelleren Programmieren (denken Sie etwa an CoffeeScript …)
- Wie funktionieren FindBugs, Lint und ähnliche Tools?
  - Statische Codeanalyse: Dead code elimination
- Language-theoretic Security: LangSec
- Verständnis für bestimmte Sprachkonstrukte und -konzepte (etwa `virtual` in C++)
- Vertiefung durch Besuch “echter” Compilerbau-Veranstaltungen an Uni möglich :-)
- Wie funktioniert:
  - ein Python-Interpreter?
  - das Syntaxhighlighting in einem Editor oder in Doxygen?
  - ein Hardwarecompiler (etwa VHDL)?
  - ein Text-Formatierer (TeX, LaTeX, …)?
  - CoffeeScript oder Emscripten?
- Wie kann man einen eigenen Compiler/Interpreter basteln, etwa für
  - MiniJava (mit C-Backend)
  - Brainfuck
  - Übersetzung von JSON nach XML
- Um eine profundes Kenntnis von Programmiersprachen zu erlangen, ist eine Beschäftigung mit ihrer Implementierung unerlässlich.
- Viele Grundtechniken der Informatik und elementare Datenstrukturen wie Keller, Listen, Abbildungen, Bäume, Graphen, Automaten etc. finden im Compilerbau Anwendung. Dadurch schließt sich in gewisser Weise der Kreis in der Informatikausbildung …
- Aufgrund seiner Reife gibt es hervorragende Beispiele von formaler Spezifikation im Compilerbau.
- Mit dem Gebiet der formalen Sprachen berührt der Compilerbau interessante Aspekte moderner Linguistik. Damit ergibt sich letztlich eine Verbindung zur KI …
- Die Unterscheidung von Syntax und Semantik ist eine grundlegende Technik in fast allen formalen Systeme.

2.3.8.2. Parser-Generatoren (Auswahl)

Diese Tools könnte man beispielsweise nutzen, um seine eigene Sprache zu basteln.

- ANTLR (ANother Tool for Language Recognition) is a powerful parser generator for reading, processing, executing, or translating structured text or binary files: github.com/antlr/antlr4
- Grammars written for ANTLR v4; expectation that the grammars are free of actions: github.com/antlr/grammars-v4
- An incremental parsing system for programmings tools: github.com/tree-sitter/tree-sitter
- Flex, the Fast Lexical Analyzer - scanner generator for lexing in C and C++: github.com/westes/flex
- Bison is a general-purpose parser generator that converts an annotated context-free grammar into a deterministic LR or generalized LR (GLR) parser employing LALR(1) parser tables: gnu.org/software/bison
- Parser combinators for binary formats, in C: github.com/UpstandingHackers/hammer
- Eclipse Xtext is a language development framework: github.com/eclipse/xtext

2.3.8.3. Statische Analyse, Type-Checking und Linter

Als Startpunkt für eigene Ideen. Oder Verbessern/Erweitern der Projekte …

- Pluggable type-checking for Java: github.com/typetools/checker-framework
- SpotBugs is FindBugs' successor. A tool for static analysis to look for bugs in Java code: github.com/spotbugs/spotbugs
- An extensible cross-language static code analyzer: github.com/pmd/pmd
- Checkstyle is a development tool to help programmers write Java code that adheres to a coding standard: github.com/checkstyle/checkstyle
- JaCoCo - Java Code Coverage Library: github.com/jacoco/jacoco
- Sanitizers: memory error detector: github.com/google/sanitizers
- JSHint is a tool that helps to detect errors and potential problems in your JavaScript code: github.com/jshint/jshint
- Haskell source code suggestions: github.com/ndmitchell/hlint
- Syntax checking hacks for vim: github.com/vim-syntastic/syntastic

2.3.8.4. DSL (Domain Specific Language)

- NVIDIA Material Definition Language SDK: github.com/NVIDIA/MDL-SDK
- FitNesse – The Acceptance Test Wiki: github.com/unclebob/fitnesse

Hier noch ein Framework, welches auf das Erstellen von DSL spezialisiert ist:

- Eclipse Xtext is a language development framework: github.com/eclipse/xtext

2.3.8.5. Konverter von X nach Y

- Emscripten: An LLVM-to-JavaScript Compiler: github.com/kripken/emscripten
- "Unfancy JavaScript": github.com/jashkenas/coffeescript

- Universal markup converter: github.com/jgm/pandoc
- Übersetzung von JSON nach XML

2.3.8.6. Odds and Ends

- How to write your own compiler: staff.polito.it/silvano.rivoira/HowToWriteYourOwnCompiler.htm
- Building a modern functional compiler from first principles: github.com/sdiehl/write-you-a-haskell
- Language-theoretic Security: LangSec
- Generierung von automatisierten Tests mit Esprima: heise.de/-4129726
- Eigener kleiner Compiler/Interpreter, etwa für
  - MiniJava mit C-Backend oder sogar LLVM-Backend
  - Brainfuck

2.3.8.7. Als weitere Anregung: Themen der Mini-Projekte im W17

- Java2UMLet
- JavaDoc-to-Markdown
- Validierung und Übersetzung von Google Protocol Buffers v3 nach JSON
- svg2tikz
- SwaggerLang – Schreiben wie im Tagebuch
- Markdown zu LaTeX
- JavaDocToLaTeX
- MySQL2REDIS-Parser

2.3.9. Wrap-Up

- Compiler übersetzen formalen Text in ein anderes Format

- Nicht alle Stufen kommen immer vor => unterschiedliche Anwendungen
  - “Echte” Compiler: Sourcecode nach Maschinencode
  - Interpreter: Interaktive Ausführung
  - Virtuelle Maschinen als Zwischending zwischen Compiler und Interpreter
  - Transpiler: formaler Text nach formalem Text
  - Analysetools: Parsen den Sourcecode, werten die Strukturen aus

Zum Nachlesen

- Aho u. a. (2023): Kapitel 1 Introduction
- Grune u. a. (2012): Kapitel 1 Introduction

Lernziele

- k1: Ich kenne verschiedene Anwendungen für Compiler durch Einsatz bestimmter Stufen der Compiler-Pipeline

3. Lexikalische Analyse

In der lexikalischen Analyse soll ein Lexer (auch “Scanner”) den Zeichenstrom in eine Folge von Token zerlegen. Zur Spezifikation der Token werden in der Regel reguläre Ausdrücke verwendet.

3.1. Reguläre Sprachen, Ausdrucksstärke

Weitere Unterlagen

- Annotierte Folien: Reguläre Sprachen, Ausdrucksstärke

3.1.1. Motivation

3.1.1.1. Was muss ein Compiler wohl als erstes tun?

Hier entsteht ein Tafelbild.

3.1.1.2. Themen für heute

- Endliche Automaten
- Reguläre Sprachen

3.1.2. Endliche Automaten

3.1.2.1. Deterministische endliche Automaten

Def.: Ein **deterministischer endlicher Automat** (DFA) ist ein 5-Tupel $A = (Q, \Sigma, \delta, q_0, F)$ mit

- Q : endliche Menge von **Zuständen**
- Σ : Alphabet von **Eingabesymbolen**
- δ : die (eventuell partielle) **Übergangsfunktion** $(Q \times \Sigma) \to Q$, δ kann partiell sein
- $q_0 \in Q$: der **Startzustand**
- $F \subseteq Q$: die Menge der **Endzustände**

3.1.2.2. Nichtdeterministische endliche Automaten

Def.: Ein **nichtdeterministischer endlicher Automat** (NFA) ist ein 5-Tupel $A = (Q, \Sigma, \delta, q_0, F)$ mit

- Q : endliche Menge von **Zuständen**
- Σ : Alphabet von **Eingabesymbolen**
- δ : die (eventuell partielle) Übergangsfunktion $(Q \times \Sigma) \to Q$
- $q_0 \in Q$: der **Startzustand**
- $F \subseteq Q$: die Menge der **Endzustände**

3.1.2.3. Akzeptierte Sprachen

Def.: Sei A ein DFA oder ein NFA. Dann ist **L(A)** die von A akzeptierte Sprache, d. h.

$L(A) = \{\text{Wörter } w \mid \delta^*(q_0, w) \in F\}$

3.1.2.4. Wozu NFAs im Compilerbau?

Pattern Matching (Erkennung von Schlüsselwörtern, Bezeichnern, …) geht mit NFAs.

NFAs sind so nicht zu programmieren, aber:

Satz: Eine Sprache L wird von einem NFA akzeptiert $\Leftrightarrow$ L wird von einem DFA akzeptiert.

D. h. es existieren Algorithmen zur

- Umwandlung von NFAs in DFAS
- Minimierung von DFAs

3.1.3. Reguläre Sprachen

3.1.3.1. Reguläre Ausdrücke

Def.: Induktive Definition von **regulären Ausdrücken** (regex) und der von ihnen repräsentierten Sprache **L**:

- Basis:
  - ϵ und $\emptyset$ sind reguläre Ausdrücke mit $L(\epsilon) = \{\epsilon\}$, $L(\emptyset) = \emptyset$
  - Sei a ein Symbol $\Rightarrow$ a ist ein regex mit $L(a) = \{a\}$
- Induktion: Seien $E,\ F$ reguläre Ausdrücke. Dann gilt:
  - $E + F$ ist ein regex und bezeichnet die Vereinigung $L(E + F) = L(E) \cup L(F)$
  - EF ist ein regex und bezeichnet die Konkatenation $L(EF) = L(E)L(F)$
  - E^* ist ein regex und bezeichnet die Kleene-Hülle $L(E^*) = (L(E))^*$
  - (E) ist ein regex mit $L((E)) = L(E)$

Vorrangregeln der Operatoren für reguläre Ausdrücke: *, Konkatenation, +

3.1.3.2. Formale Grammatiken

Def.: Eine *formale Grammatik* ist ein 4-Tupel $G = (N, T, P, S)$ aus

- N: endliche Menge von **Nichtterminalen**
- T: endliche Menge von **Terminalen**, $N \cap T = \emptyset$
- $S \in N$: **Startsymbol**
- P: endliche Menge von **Produktionen** der Form

$X \to Y$ mit $X \in (N \cup T)^* N (N \cup T)^*, Y \in (N \cup T)^*$

3.1.3.3. Ableitungen

Def.: Sei $G = (N, T, P, S)$ eine Grammatik, sei $\alpha A \beta$ eine Zeichenkette über $(N \cup T)^*$ und sei $A \to \gamma$ eine Produktion von G.

Wir schreiben: $\alpha A \beta \Rightarrow \alpha \gamma \beta$ ($\alpha A \beta$ leitet $\alpha \gamma \beta$ ab).

Def.: Wir definieren die Relation $\overset{*}{\Rightarrow}$ induktiv wie folgt:

- Basis: $\forall \alpha \in (N \cup T)^* \alpha \overset{*}{\Rightarrow} \alpha$ (Jede Zeichenkette leitet sich selbst ab.)
- Induktion: Wenn $\alpha \overset{*}{\Rightarrow} \beta$ und $\beta \Rightarrow \gamma$ dann $\alpha \overset{*}{\Rightarrow} \gamma$

Def.: Sei $G = (N, T, P, S)$ eine formale Grammatik. Dann ist $L(G) = \{\text{Wörter } w \text{ über } T \mid S \overset{*}{\Rightarrow} w\}$ die von G erzeugte Sprache.

3.1.3.4. Reguläre Grammatiken

Def.: Eine **reguläre (oder type-3-) Grammatik** ist eine formale Grammatik mit den folgenden Einschränkungen:

- Alle Produktionen sind entweder von der Form
  - $X \to aY$ mit $X \in N, a \in T, Y \in N$ (*rechtsreguläre* Grammatik) oder
  - $X \to Ya$ mit $X \in N, a \in T, Y \in N$ (*linksreguläre* Grammatik)
- $X \to \epsilon$ ist erlaubt

3.1.3.5. Reguläre Sprachen und ihre Grenzen

Satz: Die von endlichen Automaten akzeptiert Sprachklasse, die von regulären Ausdrücken beschriebene Sprachklasse und die von regulären Grammatiken erzeugte Sprachklasse sind identisch und heißen **reguläre Sprachen**.

Reguläre Sprachen

- einfache Struktur
- Matchen von Symbolen (z. B. Klammern) nicht möglich, da die fixe Anzahl von Zuständen eines DFAs die Erkennung solcher Sprachen verhindert.

3.1.3.6. Wozu reguläre Sprachen im Compilerbau?

- Reguläre Ausdrücke
  - definieren Schlüsselwörter und alle weiteren Symbole einer Programmiersprache, z. B. den Aufbau von Gleitkommazahlen
  - werden (oft von einem Generator) in DFAs umgewandelt
  - sind die Basis des *Scanners* oder *Lexers*

3.1.3.7. Ein Lexer ist mehr als ein DFA

- Ein **Lexer**
  - wandelt mittels DFAs aus regulären Ausdrücken die Folge von Zeichen der Quelldatei in eine Folge von sog. Token um
  - bekommt als Input eine Liste von Paaren aus regulären Ausdrücken und Tokennamen, z. B. (“while”, WHILE)
  - Kommentare und Strings müssen richtig erkannt werden. (Schachtelungen)
  - liefert Paare von Token und deren Werte, sofern benötigt, z. B. (WHILE, _), oder (IDENTIFIER, “radius”) oder (INTEGERZAHL, “334”)

3.1.3.8. Wie geht es weiter?

- Ein **Parser**
  - führt mit Hilfe des Tokenstreams vom Lexer die Syntaxanalyse durch
  - basiert auf einer sog. kontextfreien Grammatik, deren Terminale die Token sind
  - liefert die syntaktische Struktur in Form eines Ableitungsbaums (**syntax tree**, **parse tree**), bzw. einen **AST** (abstract syntax tree) ohne redundante Informationen im Ableitungsbaum (z. B. Semikolons)
  - liefert evtl. Fehlermeldungen

3.1.4. Wrap-Up

3.1.4.1. Wrap-Up

- Definition und Aufgaben von Lexern
- DFAs und NFAs

- Reguläre Ausdrücke
- Reguläre Grammatiken
- Zusammenhänge zwischen diesen Mechanismen und Lexern, bzw. Lexergeneratoren

Zum Nachlesen

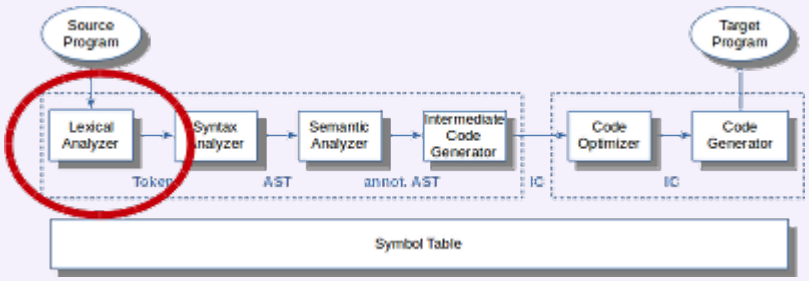
- Aho u. a. (2023): Abschnitt 2.6 und Kapitel 3
- Torczon und Cooper (2012): Kapitel 2
- Parr (2014)

Lernziele

- k1: Ich kenne DFAs
- k1: Ich kenne NFAs
- k1: Ich kenne reguläre Ausdrücke
- k1: Ich kenne reguläre Grammatiken
- k2: Ich kann die Zusammenhänge und Gesetzmäßigkeiten bzgl. der oben genannten Konstrukte an einem Beispiel erklären
- k3: Ich kann für eine Fragestellung DFAs, NFAs, reguläre Ausdrücke, reguläre Grammatiken entwickeln
- k3: Ich kann herausfinden, ob eine Sprache regulär ist
- k3: Ich kann einen DFA entwickeln, der alle Schlüsselwörter, Namen und weitere Symbole einer Programmiersprache akzeptiert

3.2. Lexer: Handcodierte Implementierung

TL;DR



Der Lexer (auch “Scanner”) soll den Zeichenstrom in eine Folge von Token zerlegen. Zur Spezifikation der Token werden reguläre Ausdrücke verwendet.
Von Hand implementierte Lexer arbeiten üblicherweise rekursiv und verarbeiten immer das nächste Zeichen im Eingabestrom. Die Arbeitsweise erinnert an LL-Parser (vgl. LL-Parser).
Lexer müssen sehr effizient sein, da sie noch direkt auf der niedrigsten Abstraktionsstufe arbeiten und u.U. oft durchlaufen werden. Deshalb setzt man hier gern spezielle Techniken wie Puffern von Zeichen über einen Doppel-Puffer ein.
Die Palette an Fehlerbehandlungsstrategien im Lexer reichen von “aufgeben” über den “Panic Mode” (“gobbeln” von Zeichen, bis wieder eines passt) und Ein-Schritt-Transformationen bis hin zu speziellen

> Lexer-Regeln, die beispielsweise besonders häufige Typos abfangen.

Videos

Vorlesung [YT], [HSBI]

Aus dem Eingabe(-quell-)text

```
/* demo */
a= [5   , 6]     ;
```

erstellt der Lexer (oder auch Scanner genannt) eine Sequenz von Token:

```
<ID, "a"> <ASSIGN> <LBRACK> <NUM, 5> <COMMA> <NUM, 6> <RBRACK> <SEMICOL>
```

3.2.2. Manuelle Implementierung: Rekursiver Abstieg

```
def nextToken():
    while (peek != EOF):  # globale Variable, über consume()
        switch (peek):
            case ' ': case '\t': case '\n': WS(); continue
            case '[': consume(); return Token(LBRACK, '[')
            ...
            default:
                if isLetter(peek): return NAME()
                raise Error("invalid character: "+peek)
    return Token(EOF_Type, "<EOF>")

def WS():
    while (peek == ' ' || peek == '\t' || ...): consume()

def NAME():
    buf = StringBuilder()
    do { buf.append(peek); consume(); } while (isLetter(peek))
    return Token(NAME, buf.toString())
```

Die manuelle Implementierung “denkt” nicht in den Zuständen des DFA, sondern orientiert sich immer am aktuellen Zeichen “`peek`”. Abhängig von dessen Ausprägung wird entweder direkt ein Token erzeugt und das Zeichen aus dem Eingabestrom entfernt sowie das nächste Zeichen eingelesen (mittels der Funktion `consume()`, nicht dargestellt im Beispiel), oder man ruft weitere Funktionen auf, die das Gewünschte erledigen, beispielsweise um White-Spaces zu entfernen oder um einen Namen einzulesen: Nach einem Buchstaben werden alle folgenden Buchstaben dem Namen (Bezeichner) hinzugefügt. Sobald ein anderes Zeichen im Eingabestrom erscheint, wird das Namen-Token erzeugt.

Die Funktion `consume()` “verbraucht” das aktuelle Zeichen “`peek`” und holt das nächste Zeichen aus dem Eingabestrom.

Anmerkung: Häufig findet man im Lexer keinen “schönen” objektorientierten Ansatz. Dies ist i.d.R. Geschwindigkeitsgründen geschuldet …

3.2.3. Read-Ahead: Unterscheiden von “<” und “<=”

```
def nextToken():
    while (peek != EOF):  # globale Variable
        switch (peek):
            case '<':
                if match('='): consume(); return Token(LE, "<=")
                else: consume(); return Token(LESS, '<')
            ...
    return Token(EOF_Type, "<EOF>")

def match(c):   # Lookahead: Ein Zeichen
    consume()
    if (peek == c): return True
    else: rollBack(); return False
```

Um die Token “<” und “<=” unterscheiden zu können, müssen wir ein Zeichen vorausschauen: Wenn nach dem “<” noch ein “=” kommt, ist es “<=”, sonst “<”.

Erinnerung: Die Funktion `consume()` liest das nächste Zeichen aus dem Eingabestrom und speichert den Wert in der globalen Variable `peek`.

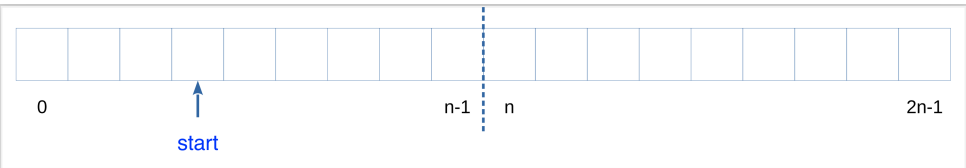
Für das Read-Ahead wird die Funktion `match()` definiert, die zunächst das bereits bekannte Zeichen, in diesem Fall das “<” durch das nächste Zeichen im Eingabestrom ersetzt (Aufruf von `consume()`). Falls der Vergleich des Lookahead-Zeichens mit dem gesuchten Zeichen erfolgreich ist, liegt das “größere” Token vor, also “<=”. Dann wird noch das “=” durch das nächste Zeichen ersetzt und das Token `LE` gebildet. Anderenfalls muss das zuviel gelesene Zeichen wieder in den Eingabestrom zurückgelegt werden (`rollBack()`).

3.2.4. Puffern des Input-Stroms: Double Buffering

Das Einlesen einzelner Zeichen führt zwar zu eleganten algorithmischen Lösungen, ist aber zur Laufzeit deutlich “teurer” als das Einlesen mit gepufferten I/O-Operationen, die eine ganze Folge von Zeichen einlesen (typischerweise einen ganzen Disk-Block, beispielsweise 4096 Zeichen).

Dazu kann man einen Ringpuffer nutzen, den man mit Hilfe von zwei gleich großen **`char`**-Puffern mit jeweils der Länge N simulieren kann. (N sollte dann der Länge eines Disk-Blocks entsprechen.)

Vergleiche auch Wikipedia: “Circular Buffer”.




```
start = 0; end = 0; fill(buffer[0:n])

def consume():
    peek = buffer[start]
    start = (start+1) mod 2n
```

```
6       if (start mod n == 0):
7           fill(buffer[start:start+n-1])
8           end = (start+n) mod 2n
9
10  def rollBack():
11      if (start == end): raise Error("roll back error")
12      start = (start-1) mod 2n
```

Zunächst wird nur der vordere Pufferteil durch einen passenden Systemaufruf gefüllt.

Beim Weiterschalten im simulierten DFA oder im manuell kodierten Lexer (Funktionsaufruf von `consume()`) wird das nächste Zeichen aus dem vorderen Pufferteil zurückgeliefert. Über die Modulo-Operation bleibt der Pointer `start` immer im Speicherbereich der beiden Puffer.

Wenn man das Ende des vorderen Puffers erreicht, wird der hintere Puffer mit einem Systemaufruf gefüllt. Gleichzeitig wird ein Hilfspointer `end` auf den Anfang des vorderen Puffers gesetzt, um Fehler beim Roll-Back zu erkennen.

Wenn man das Ende des hinteren Puffers erreicht, wird der vordere Puffer nachgeladen und der Hilfspointer auf den Anfang des hinteren Puffers gesetzt.

Im Grunde ist also immer ein Puffer der "Arbeitspuffer" und der andere enthält die bereits gelesene (verarbeitete) Zeichenkette. Wenn beim Nachladen weniger als N Zeichen gelesen werden, liefert der Systemaufruf als letztes "Zeichen" ein `EOF`. Beim Verarbeiten wird `peek` entsprechend diesen Wert bekommen und der Lexer muss diesen Wert abfragen und berücksichtigen.

Für das Roll-Back wird der `start`-Pointer einfach dekrementiert (und mit einer Modulo-Operation auf den Speicherbereich der beiden Puffer begrenzt). Falls dabei der `end`-Pointer "eingeholt" wird, ist der `start`-Pointer durch beide Puffer zurückgelaufen und es gibt keinen früheren Input mehr. In diesem Fall wird entsprechend ein Fehler gemeldet.

Anmerkung: In der Regel sind die Lexeme kurz und man muss man nur ein bis zwei Zeichen im Voraus lesen. Dann ist eine Puffergröße von 4096 Zeichen mehr als ausreichend groß und man sollte nicht in Probleme laufen. Wenn der nötige Look-Ahead aber beliebig groß werden kann, etwa bei Sprachen ohne reservierte Schlüsselwörtern oder bei Kontext-sensitiven Lexer-Grammatiken (denken Sie etwa an die Einrücktiefe bei Python), muss man andere Strategien verwenden. ANTLR beispielsweise vergrößert in diesem Fall den Puffer dynamisch, alternativ könnte man die Auflösung zwischen Schlüsselwörtern und Bezeichnern dem Parser überlassen.

3.2.5. Typische Muster für Erstellung von Token

1. Schlüsselwörter

   - Ein eigenes Token (RE/DFA) für jedes Schlüsselwort, oder
   - Erkennung als Name (`ID`) und nachträglich Vergleich mit Wörterbuch sowie Korrektur des Tokentyps

   Wenn Schlüsselwörter über je ein eigenes Token abgebildet werden, benötigt man für jedes Schlüsselwort einen eigenen RE bzw. DFA. Die Erkennung als Bezeichner und das Nachschlagen in einem Wörterbuch (geeignete Hashtabelle) sowie die entsprechende nachträgliche Korrektur des Tokentyps kann die Anzahl der Zustände im Lexer signifikant reduzieren!

2. Operatoren
   - Ein eigenes Token für jeden Operator, oder
   - Gemeinsames Token für jede Operatoren-Klasse
3. Bezeichner: Ein gemeinsames Token für alle Namen
4. Zahlen: Ein gemeinsames Token für alle numerischen Konstante (ggf. Integer und Float unterscheiden)

   Für Zahlen führt man oft ein Token "`NUM`" ein. Als Attribut speichert man das Lexem i.d.R. als String. Alternativ kann man (zusätzlich) das Lexem in eine Zahl konvertieren und als (zusätzliches) Attribut speichern. Dies kann in späteren Stufen viel Arbeit sparen.
5. String-Literale: Ein gemeinsames Token
6. Komma, Semikolon, Klammern, ...: Je ein eigenes Token
7. Regeln für White-Space und Kommentare etc. ...

   Normalerweise benötigt man Kommentare und White-Spaces in den folgenden Stufen nicht und entfernt diese deshalb aus dem Eingabestrom. Dabei könnte man etwa White-Spaces in den Pattern der restlichen Token berücksichtigen, was die Pattern aber sehr komplex macht. Die Alternative sind zusätzliche Pattern, die auf die White-Space und anderen nicht benötigten Inhalt matchen und diesen "geräuschlos" entfernen. Mit diesen Pattern werden keine Token erzeugt, d.h. der Parser und die anderen Stufen bemerken nichts von diesem Inhalt.

   Gelegentlich benötigt man aber auch Informationen über White-Spaces, beispielsweise in Python. Dann müssen diese Token wie normale Token an den Parser weitergereicht werden.

Jedes Token hat i.d.R. ein Attribut, in dem das Lexem gespeichert wird. Bei eindeutigen Token (etwa bei eigenen Token je Schlüsselwort oder bei den Interpunktions-Token) kann man sich das Attribut auch sparen, da das Lexem durch den Tokennamen eindeutig rekonstruierbar ist.

Token	Beschreibung	Beispiel-Lexeme
`if`	Zeichen `i` und `f`	`if`
`relop`	< oder > oder <= oder >= oder == oder !=	<, <=
`id`	Buchstabe, gefolgt von Buchstaben oder Ziffern	`pi`, `count`, `x3`
`num`	Numerische Konstante	`42`, `3.14159`, `0`
`literal`	Alle Zeichen außer ", in " eingeschlossen	`"core dumped"`

Anmerkung: Wenn es mehrere matchende REs gibt, wird in der Regel das längste Lexem bevorzugt. Wenn es mehrere gleich lange Alternativen gibt, muss man mit Vorrangregeln bzgl. der Token arbeiten.

3.2.6. Fehler bei der Lexikalischen Analyse

Problem: Eingabestrom sieht so aus: `fi (a==42){ ... }`

Der Lexer kann nicht erkennen, ob es sich bei `fi` um ein vertipptes Schlüsselwort handelt oder um einen Bezeichner: Es könnte sich um einen Funktionsaufruf der Funktion `fi()` handeln … Dieses Problem kann erst in der nächsten Stufe sinnvoll erkannt und behoben werden.

=> Was tun, wenn keines der Pattern auf den Anfang des Eingabestroms passt?

Optionen:

- Aufgeben …

  Eventuell vielleicht sogar die beste und einfachste Variante :-)

- "Panic Mode": Entferne so lange Zeichen, bis ein Pattern passt.

  Das verwirrt u.U. den Parser, kann aber insbesondere in interaktiven Umgebungen hilfreich sein. Ggf. kann man dem Parser auch signalisieren, dass hier ein Problem vorlag.

- Ein-Schritt-Transformationen:
  - Füge fehlendes Zeichen in Eingabestrom ein.
  - Entferne ein Zeichen aus Eingabestrom.
  - Vertausche ein Zeichen:
    * Ersetze ein Zeichen durch ein anderes.
    * Vertausche zwei benachbarte Zeichen.

  Diese Transformationen versuchen, den Input in einem Schritt zu reparieren. Das ist durchaus sinnvoll, da in der Praxis die meisten Fehler in dieser Stufe durch ein einzelnes Zeichen hervorgerufen werden: Es fehlt ein Zeichen oder es ist eines zuviel im Input. Es liegt ein falsches Zeichen vor (Tippfehler) oder zwei benachbarte Zeichen wurden verdreht …
  Im Prinzip könnte man auch eine allgemeinere Strategie versuchen, indem man diejenige Transformation mit der *kleinsten Anzahl von Schritten* zur Fehlerbehebung bestimmt. Beispiele dafür finden sich im Bereich Natural Language Processing (*NLP*), etwa die Levenshtein-Distanz oder der SoundEx-Algorithmus oder sogar Hidden-Markov-Modelle. Allerdings muss man sich in Erinnerung rufen, dass gerade in dieser ersten Phase eines Compilers die Geschwindigkeit stark im Fokus steht und eine ausgefeilte Fehlerkorrekturstrategie die vielen kleinen Optimierungen schnell wieder zunichte machen kann.

- Fehler-Regeln: Matche typische Typos

  Gelegentlich findet man in den Grammatiken für den Lexer extra Regeln, die häufige bzw. typische Typos matchen und dann passend darauf reagieren.

3.2.7. Wrap-Up

- Zusammenhang DFA, RE und Lexer
- Implementierungsansatz: Manuell codiert (rekursiver Abstieg)
- Read-Ahead
- Puffern mit Doppel-Puffer-Strategie
- Typische Fehler beim Scannen

Zum Nachlesen

- Nystrom (2021): Kapitel 4
- Parr (2010): Kapitel 2 (“Pattern 2: LL(1) Recursive-Decent Lexer”)

Lernziele

- k1: Ich kenne die Aufgaben eines Lexers
- k2: Ich kann die manuelle Implementierung eines Lexers mit Hilfe von rekursivem Abstieg erklären
- k2: Ich kann den Umgang mit dem Doppel-Puffer erklären
- k2: Ich kann die Varianten bei der Erkennung von Schlüsselwörtern an einem Beispiel verdeutlichen
- k2: Ich kann typische Fehler und Lösungsansätze in der lexikalischen Analyse erklären
- k3: Ich kann für ein Problem eine typische Einteilung von Token vornehmen
- k3: Ich kann einen Top-Down-Lexer mit Read-Ahead und intelligenter Pufferung implementieren

Challenges

Manuell implementierter Lexer

Betrachten Sie die folgende einfache Sprache:

```
1 a = 10 - 5      # Zuweisung des Ausdruckes 10-5 (Integer-Wert 5) an Variable
      a
2 b = a + 2 * 3   # Zuweisung von 16 an Variable b
3 c = a != b      # Zuweisung eines boolschen Werts an c
```

Es gibt nur Statements und Expressions:

- Statement: Zuweisung; jedes Statement endet mit einem NL
- Expression: Zahl, Variable, Addition, Subtraktion, Multiplikation (mit üblichem Vorrang), Vergleich

Aufgaben:

- Geben Sie eine ANTLR-Grammatik für diese Sprache an.
- Implementieren Sie analog zum Vorgehen in der Vorlesung einen Lexer für diese Sprache. (Nur den Lexer, den Parser besprechen wir in einer anderen Sitzung.)

4. Syntaktische Analyse

In der syntaktischen Analyse arbeitet ein Parser mit dem Tokenstrom, der vom Lexer kommt. Mit Hilfe einer Grammatik wird geprüft, ob gültige Sätze im Sinne der Sprache/Grammatik gebildet wurden. Der Parser erzeugt dabei den Parse-Tree. Man kann verschiedene Parser unterscheiden, beispielsweise die LL- und die LR-Parser.

4.1. CFG

Weitere Unterlagen

- Annotierte Folien: CFG, LL-Parser

4.1.1. Wiederholung

4.1.1.1. Endliche Automaten, reguläre Ausdrücke, reguläre Grammatiken, reguläre Sprachen

- Wie sind DFAs und NFAs definiert?
- Was sind reguläre Ausdrücke?
- Was sind formale und reguläre Grammatiken?
- In welchem Zusammenhang stehen all diese Begriffe?
- Wie werden DFAs und reguläre Ausdrücke im Compilerbau eingesetzt?

4.1.2. Motivation

4.1.2.1. Wofür reichen reguläre Sprachen nicht?

Für z. B. alle Sprachen, in deren Wörtern Zeichen über eine Konstante hinaus gezählt werden müssen. Diese Sprachen lassen sich oft mit Variablen im Exponenten beschreiben, die unendlich viele Werte annehmen können.

- $a^i b^{2*i}$ ist nicht regulär
- $a^i b^{2*i}$ für $0 \leq i \leq 3$ ist regulär
- Wo finden sich die oben genannten VAriablen bei einem DFA wieder?
- Warum ist die erste Sprache oben nicht regulär, die zweite aber?

4.1.2.2. Themen für heute

- PDAs: mächtiger als DFAs, NFAs
- kontextfreie Grammatiken und Sprachen: mächtiger als reguläre Grammatiken und Sprachen
- DPDAs und deterministisch kontextfreie Grammatiken: die Grundlage der Syntaxanalyse im Compilerbau

4.1.3. Kellerautomaten (Push-Down-Automata, PDAs)

4.1.3.1. Kellerautomaten (Push-Down-Automata, PDAs)

Einordnung: Erweiterung der Automatenklasse DFA um einen Stack

Def.: Ein **Kellerautomat** (PDA) $P = (Q,\ \Sigma,\ \Gamma,\ \delta,\ q_0,\ \bot,\ F)$ ist ein Septupel aus

Q:	eine endliche Menge von Zuständen
Σ:	eine endliche Menge von Eingabesymbolen
Γ:	ein endliches Kelleralphabet
δ:	die Übergangsfunktion $\delta : \mathrm{Q} \times \Sigma \cup \{\epsilon\} \times \Gamma \to 2^{Q \times \Gamma^*}$
q_0:	der Startzustand
$\bot \in \Gamma$:	der anfängliche Kellerinhalt, symbolisiert den leeren Keller ($\bot$ = bottom)
$F \subseteq Q$:	die Menge von Endzuständen

Abbildung 4.1.: Definition eines PDAs

Ein PDA ist per Definition nichtdeterministisch und kann spontane Zustandsübergänge durchführen.

4.1.3.2. Was kann man damit akzeptieren?

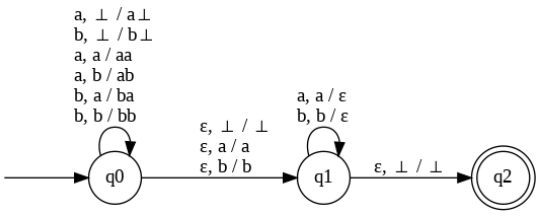
Strukturen mit paarweise zu matchenden Symbolen.

Bei jedem Zustandsübergang wird ein Zeichen (oder ϵ) aus der Eingabe gelesen, ein Symbol von Keller genommen. Diese und das Eingabezeichen bestimmen den Folgezustand und eine Zeichenfolge, die auf den Stack gepackt wird. Dabei wird ein Symbol, das später mit einem Eingabesymbol zu matchen ist, auf den Stack gepackt.

Soll das automatisch vom Stack genommene Symbol auf dem Stack bleiben, muss es wieder gepusht werden.

4.1.3.3. Beispiel

Ein PDA für $L = \{ww^R \mid w \in \{a, b\}^*\}$:



4.1.3.4. Deterministische PDAs

Def. Ein PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, \bot, F)$ ist *deterministisch* :⇔

- $\delta(q, a, X)$ hat höchstens ein Element für jedes $q \in Q, a \in \Sigma$ oder $(a = \epsilon$ und $X \in \Gamma)$.
- Wenn $\delta(q, a, x)$ nicht leer ist für ein $a \in \Sigma$, dann muss $\delta(q, \epsilon, x)$ leer sein.

Deterministische PDAs werden auch *DPDAs* genannt.

4.1.3.5. Der kleine Unterschied

Satz: Die von DPDAs akzeptierten Sprachen sind eine echte Teilmenge der von PDAs akzeptierten Sprachen.

Reguläre Sprachen sind eine echte Teilmenge der von DPDAs akzeptierten Sprachen.

4.1.4. Kontextfreie Grammatiken und Sprachen

4.1.4.1. Kontextfreie Grammatiken

Def. Eine *kontextfreie (cf-)* Grammatik ist ein 4-Tupel $G = (N, T, P, S)$ mit *N, T, S* wie in (formalen) Grammatiken und *P* ist eine endliche Menge von Produktionen der Form:

$X \to Y$ mit $X \in N, Y \in (N \cup T)^*$.

$\Rightarrow, \overset{*}{\Rightarrow}$ sind definiert wie bei regulären Sprachen.

4.1.4.2. Nicht jede kontextfreie Grammatik ist eindeutig

Def.: Gibt es in einer von einer kontextfreien Grammatik erzeugten Sprache ein Wort, für das mehr als ein Ableitungsbaum existiert, so heißt diese Grammatik *mehrdeutig*. Anderenfalls heißt sie *eindeutig*.

Satz: Es ist nicht entscheidbar, ob eine gegebene kontextfreie Grammatik eindeutig ist.

Satz: Es gibt kontextfreie Sprachen, für die keine eindeutige Grammatik existiert.

4.1.4.3. Kontextfreie Grammatiken und PDAs

Satz: Die kontextfreien Sprachen und die Sprachen, die von PDAs akzeptiert werden, sind dieselbe Sprachklasse.

Satz: Eine von einem DPDA akzeptierte Sprache hat eine eindeutige Grammatik.

Def.: Die Klasse der Sprachen, die von einem DPDA akzeptiert werden, heißt Klasse der *deterministisch kontextfreien (oder LR(k)-) Sprachen*.

Vorgehensweise im Compilerbau: Eine Grammatik für die gewünschte Sprache definieren und schauen, ob sich daraus ein DPDA generieren lässt (automatisch).

4.1.5. Wrap-Up

4.1.5.1. Das sollen Sie mitnehmen

- Die Struktur von gängigen Programmiersprachen lässt sich nicht mit regulären Ausdrücken beschreiben und damit nicht mit DFAs akzeptieren.
- Das Automatenmodell der DFAs wird um einen endlosen Stack erweitert, das ergibt PDAs.
- Kontextfreie Grammatiken (CFGs) erweitern die regulären Grammatiken.
- Deterministisch parsebare Sprachen haben eine eindeutige kontextfreie Grammatik.
- Es ist nicht entscheidbar, ob eine gegebene kontextfreie Grammatik eindeutig ist.
- Von DPDAs akzeptierte Sprachen haben eindeutige Grammatiken.

Zum Nachlesen

- Aho u. a. (2023)
- Hopcroft u. a. (2003)

Lernziele

- k1: Ich kenne PDAs
- k1: Ich kenne deterministische PDAs
- k1: Ich kenne kontextfreie Grammatiken
- k1: Ich kenne deterministisch kontextfreie Grammatiken
- k2: Ich kann den Zusammenhang zwischen PDAs und kontextfreien Grammatiken an einem Beispiel erklären

4.2. LL-Parser

Weitere Unterlagen

- Annotierte Folien: LL-Parser (Theorie)

4.2.1. Wiederholung

4.2.1.1. PDAs und kontextfreie Grammatiken

- Warum reichen uns DFAs nicht zum Matchen von Eingabezeichen?
- Wie könnnen wir sie minimal erweitern?
- Sind PDAs deterministisch?
- Wie sind kontextfreie Grammatiken definiert?
- Sind kontextfreie Grammatiken eindeutig?

4.2.2. Motivation

4.2.2.1. Was brauchen wir für die Syntaxanalyse von Programmen?

- einen Grammatiktypen, aus dem sich manuell oder automatisiert ein Programm zur deterministischen Syntaxanalyse erstellen lässt
- einen Algorithmus zum sog. Parsen von Programmen mit Hilfe einer solchen Grammatik

4.2.2.2. Themen für heute

- Syntaxanalyse
- Top-down-Analyse
- rekursiver Abstieg
- LL(k)-Analyse

4.2.3. Syntaxanalyse

4.2.3.1. Ziele der Syntaxanalyse

- Auffinden von Syntaxfehlern mit möglichst genauer Fehlerangabe
- evtl. Vorschläge zur Fehlerbehebung
- Erstellen eines AST (abstract parse trees) zur semantischen Analyse = Ableitungsbaum ohne strukturell überflüssige Token (Semikolons, geschweifte Klammern, …)

4.2.3.2. Arten der Syntaxanalyse

Die Syntax bezieht sich auf die Struktur der zu analysierenden Eingabe, z. B. einem Computerprogramm in einer Hochsprache. Diese Struktur wird mit formalen Grammatiken beschrieben. Einsetzbar sind Grammatiken, die deterministisch kontextfreie Sprachen erzeugen.

- Top-Down-Analyse: Aufbau des Parse trees von oben nach unten
  - Parsen durch rekursiven Abstieg

  - tabellengesteuertes LL-Parsing
- Bottom-Up-Analyse: LR-Parsing

4.2.3.3. Bevor wir richtig anfangen...

Def.: Ein Nichtterminal *A* einer kontextfreien Grammatik *G* heißt *unerreichbar*, falls es kein $a, b \in (N \cup T)^*$ gibt mit $S \stackrel{*}{\Rightarrow} aAb$. Ein Nichtterminal *A* einer Grammatik *G* heißt *nutzlos*, wenn es kein Wort $w \in T^*$ gibt mit $A \stackrel{*}{\Rightarrow} w$.

Def.: Eine kontextfreie Grammatik $G = (N, T, P, S)$ heißt *reduziert*, wenn es keine nutzlosen oder unerreichbaren Nichtterminale in *N* gibt.

Bevor mit einer Grammatik weitergearbeitet wird, müssen erst alle nutzlosen und dann alle unerreichbaren Symbole eliminiert werden. Wir betrachten ab jetzt nur reduzierte Grammatiken.

4.2.4. Algorithmus: Rekursiver Abstieg

4.2.4.1. Algorithmus: Rekursiver Abstieg

Hier ist ein einfacher Algorithmus, der (indeterministisch) top-down Ableitungen vom Nonterminal *X* aufbaut:

Eingabe: Ein Nichtterminal X und das nächste zu verarbeitende Eingabezeichen a.

```
RecursiveDescent(X, a) //X ∈ N, a ∈ T
    for a production, X → Y_1 Y_2 ... Y_n
        for i = 1 to n
            if Y_i ∈ N
                RecursiveDescent(Y_i, a)
            else if Y_i ≠ a
                error
```

Abbildung 4.2.: Recursive Descent-Algorithmus

4.2.4.2. Wann scheitert das Verfahren?

Hier entsteht eine Tafelskizze ...

4.2.5. Tabellengesteuerte Parser: LL(k)-Grammatiken

4.2.5.1. First-Mengen

$S \to A \mid B \mid C$

Welche Produktion nehmen?

Wir brauchen die “terminalen k-Anfänge” von Ableitungen von Nichtterminalen, um eindeutig die nächste zu benutzende Produktion festzulegen. k ist dabei die Anzahl der sog. *Vorschautoken*.

Def.: Wir definieren $First$ - Mengen einer Grammatik wie folgt:

- $a \in T^*, |a| \leq k : First_k(a) = \{a\}$
- $a \in T^*, |a| > k : First_k(a) = \{v \in T^* \mid a = vw, |v| = k\}$
- $\alpha \in (N \cup T)^* \backslash T^* : First_k(\alpha) = \{v \in T^* \mid \alpha \overset{*}{\Rightarrow} w, \text{mit } w \in T^*, First_k(w) = \{v\}\}$

4.2.5.2. Linksableitungen

Def.: Bei einer kontextfreien Grammatik G ist die *Linksableitung* von $\alpha \in (N \cup T)^*$ die Ableitung, die man erhält, wenn in jedem Schritt das am weitesten links stehende Nichtterminal in α abgeleitet wird.

Man schreibt $\alpha \overset{*}{\Rightarrow}_l \beta$.

4.2.5.3. Follow-Mengen

Manchmal müssen wir wissen, welche terminalen Zeichen hinter einem Nichtterminal stehen können.

Def. Wir definieren *Follow* - Mengen einer Grammatik wie folgt:

$\forall \beta \in (N \cup T)^* :$

$$Follow_k(\beta) = \{w \in T^* \mid \exists \alpha, \gamma \in (N \cup T)^* \text{ mit } S \overset{*}{\Rightarrow}_l \alpha\beta\gamma \text{ und } w \in First_k(\gamma)\}$$

4.2.5.4. LL(k)-Grammatiken

Def.: Eine kontextfreie Grammatik *G = (N, T, P, S)* ist genau dann eine *LL(k)*-Grammatik, wenn für alle Linksableitungen der Form:

$S \overset{*}{\Rightarrow}_l \ wA\gamma \Rightarrow_l w\alpha\gamma \overset{*}{\Rightarrow}_l wx$

und

$S \overset{*}{\Rightarrow}_l wA\gamma \Rightarrow_l w\beta\gamma \overset{*}{\Rightarrow}_l wy$

mit $(w, x, y \in T^*, \alpha, \beta, \gamma \in (N \cup T)^*, A \in N)$ und $First_k(x) = First_k(y)$ gilt:

$\alpha = \beta$

4.2.5.5. LL(1)-Grammatiken

Das hilft manchmal:

Für $k = 1$: G ist $LL(1) : \forall A \to \alpha, A \to \beta \in P, \alpha \neq \beta$ gilt:

1. $\neg\exists a \in T : \alpha \overset{*}{\Rightarrow}_l a\alpha_1$ und $\beta \overset{*}{\Rightarrow}_l a\beta_1$
2. $((\alpha \overset{*}{\Rightarrow}_l \epsilon) \Rightarrow (\neg(\beta \overset{*}{\Rightarrow}_l \epsilon)))$ und $((\beta \overset{*}{\Rightarrow}_l \epsilon) \Rightarrow (\neg(\alpha \overset{*}{\Rightarrow}_l \epsilon)))$
3. $((\beta \overset{*}{\Rightarrow}_l \epsilon)$ und $(\alpha \overset{*}{\Rightarrow}_l a\alpha_1)) \Rightarrow a \notin Follow(A)$
4. $((\alpha \overset{*}{\Rightarrow}_l \epsilon)$ und $(\beta \overset{*}{\Rightarrow}_l a\beta_1)) \Rightarrow a \notin Follow(A)$

Die ersten beiden Zeilen bedeuten:

α und β können nicht beide ϵ ableiten, $First_1(\alpha) \cap First_1(\beta) = \emptyset$

Die dritte und vierte Zeile bedeuten:

$(\epsilon \in First_1(\beta)) \Rightarrow (First_1(\alpha) \cap Follow_1(A) = \emptyset)$

$(\epsilon \in First_1(\alpha)) \Rightarrow (First_1(\beta) \cap Follow_1(A) = \emptyset)$

4.2.5.6. LL(k)-Sprachen

Die von *LL(k)*-Grammatiken erzeugten Sprachen sind eine echte Teilmenge der deterministisch parsbaren Sprachen.

Die von *LL(k)*-Grammatiken erzeugten Sprachen sind eine echte Teilmenge der von *LL(k+1)*-Grammatiken erzeugten Sprachen.

Für eine kontextfreie Grammatik *G* ist nicht entscheidbar, ob es eine *LL(1)* - Grammatik *G'* gibt mit $L(G) = L(G')$.

In der Praxis reichen $LL(1)$ - Grammatiken oft. Hier gibt es effiziente Parsergeneratoren, deren Eingabe eine LL(k)- (meist LL(1)-) Grammatik ist, und die als Ausgabe den Quellcode eines (effizienten) tabellengesteuerten Parsers generieren.

4.2.5.7. Algorithmus: Konstruktion einer LL-Parsertabelle

Eingabe: Eine Grammatik G = (N, T, P, S) mit $\perp \in T$ als Endezeichen

Ausgabe: Eine Parsertabelle *P*

```
for each production X → α
    for each a ∈ First(α)
        add X → α to P[X, a]

    if ϵ ∈ First(α)
        for each b ∈ Follow(α)
            add X → α to P[X, b]
        if ⊥ ∈ Follow(X)
            add X → α to P[X, ⊥]
```

Abbildung 4.3.: Algorithmus zur Generierung einer LL-Parsertabelle

Statt $First_1(\alpha)$ und $Follow_1(\alpha)$ wird oft nur $First(\alpha)$ und $Follow(\alpha)$ geschrieben.

4.2.5.8. LL-Parsertabellen

Rekursive Programmierung bedeutet, dass das Laufzeitsystem einen Stack benutzt (bei einem Recursive-Descent-Parser, aber auch bei der Parsertabelle). Diesen Stack kann man auch “selbst programmieren”, d. h. einen PDA implementieren. Dabei wird ebenfalls die oben genannte Tabelle zur Bestimmung der nächsten anzuwendenden Produktion benutzt. Der Stack enthält die zu erwartenden Eingabezeichen, wenn immer eine Linksableitung gebildet wird. Diese Zeichen im Stack werden mit dem Input gematcht.

4.2.5.9. Algorithmus: Tabellengesteuertes LL-Parsen mit einem PDA

Eingabe: Eine Grammatik G = (N, T, P, S), eine Parsertabelle *P* mit $w \perp$ als initialem Kellerinhalt

Ausgabe: Wenn $w \in L(G)$, eine Linksableitung von w, Fehler sonst

```
a = next_token()
X = top of stack  // entfernt X vom Stack

while X ≠⊥

   if X = a
      a = next_token()

   else if X ∈ T
      error

   else if P[X, a] leer
      error

   else if P[X, a] = X → Y1Y2 ... Yk
      process_production(X → Y1Y2 ... Yk)
      push(Y1Y2 ... Yk) //Y1 = top of stack

   X = top of stack
```

Abbildung 4.4.: Algorithmus zum tabellengesteuerten LL-Parsen

4.2.6. Wrap-Up

4.2.6.1. Wrap-Up

- Syntaxanalyse wird mit deterministisch kontextfreien Grammatiken durchgeführt.
- Eine Teilmenge der dazu gehörigen Sprachen lässt sich top-down parsen.
- Ein einfacher Recursive-Descent-Parser arbeitet mit Backtracking.
- Ein effizienter LL(k)-Parser realisiert einen DPDA und kann automatisch aus einer LL(k)-Grammatik generiert werden.
- Der Parser liefert in der Regel einen abstrakten Syntaxbaum (AST).

Zum Nachlesen

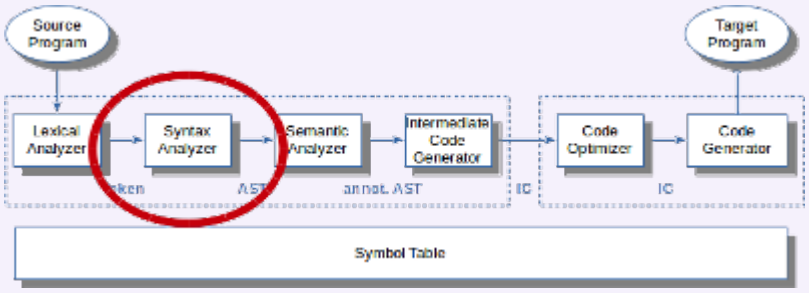
- Aho u. a. (2023)
- Hopcroft u. a. (2003)

Lernziele

- k1: Ich kenne die Top-Down-Analyse
- k1: Ich kenne Recursive-Descent-Parser
- k1: Ich kenne First- und Follow-Mengen
- k1: Ich kenne LL-Parser
- k2: Ich kann den Zusammenhang zwischen PDAs und kontextfreien Grammatiken an einem Beispiel erklären
- k2: Ich kann den algorithmischen Ablauf von LL-Parsern an einem Beispiel erklären

4.3. LL-Parser selbst implementiert

TL;DR



LL-Parser können über einen “rekursiven Abstieg” direkt aus einer Grammatik implementiert werden:

- Zu jeder Produktionsregel erstellt man eine gleichnamige Funktion.
- Wenn in der Produktionsregel andere Regeln “aufgerufen” werden, erfolgt in der Funktion an dieser Stelle der entsprechende Funktionsaufruf.
- Bei Terminalsymbolen wird das erwartete Token geprüft.

Dabei findet man wie bereits im Lexer die Funktionen `match` und `consume`, die sich hier aber auf den Tokenstrom beziehen. LL(1)-Parser schauen dabei immer das nächste Token an, LL(k) haben ein entsprechendes Look-Ahead von k Token. Dies kann man mit einem Ringpuffer für die Token realisieren.
Zur Beachtung der Vorrang- und Assoziativitätsregeln muss die Grammatik entsprechend umgebaut werden. LL-Parser haben durch die Betrachtung des aktuellen Vorschau-Tokens ein Problem mit Linksrekursion in der Grammatik, diese muss zunächst beseitigt werden. (ANTLR bietet hier gewisse Vereinfachungen an, kann aber mit indirekter Linksrekursion auch nicht umgehen.)
Eine gute Darstellung, die sich am zeichen- bzw. tokenweisen Einlesen orientiert, finden Sie in (Parr 2010) (Kapitel 2). Eine weitere sehr gut lesbare Darstellung ist in (Nystrom 2021) (Kapitel 4, 5 und 6) zu finden. Dort wird die Eingabe komplett eingelesen und daraus eine Liste aller Token erzeugt, auf der der Parser operiert. Der Beispiel-Code in dieser Vorlesung orientiert sich an der zeichenweisen bzw. tokenweisen Verarbeitung.

Videos

Vorlesung [YT], [HSBI]

4.3.1. Erinnerung Lexer: Zeichenstrom => Tokenstrom

```
def nextToken():
    while (peek != EOF):  # globale Variable, über consume()
        switch (peek):
            case ' ': case '\t': case '\n': WS(); continue
            case '[': consume(); return Token(LBRACK, '[')
            ...
            default: raise Error("invalid character: "+peek)
```

```
8       return Token(EOF_Type, "<EOF>")
9
10  def match(c):   # Lookahead: Ein Zeichen
11      consume()
12      if (peek == c): return True
13      else: rollBack(); return False
14
15  def consume():
16      peek = buffer[start]
17      start = (start+1) mod 2n
18      if (start mod n == 0):
19          fill(buffer[start:start+n-1])
20          end = (start+n) mod 2n
```

Erinnerung: Der Lexer arbeitet direkt auf dem Zeichenstrom und versucht über längste Matches daraus einen Tokenstrom zu erzeugen. Dabei wird immer das nächste Zeichen angeschaut (Funktion `match`) und mit `consume` das aktuelle Zeichen "verbraucht" und das nächste Zeichen geladen. Hier kann man über die Doppel-Puffer-Strategie das Einlesen einzelner Zeichen aus einer Datei vermeiden und immer blockweise in den Puffer einlesen.

4.3.2. Grundidee LL-Parser

Grammatik:

```
1  r : X s ;
```

LL-Implementierung:

```
1  def r():
2      match(X)
3      s()
```

- Für jede Regel in der Grammatik wird eine Methode/Funktion mit dem selben Namen definiert
- Referenzen auf ein Token `T` werden durch den Aufruf der Methode `match(T)` aufgelöst
  - `match(T)` "konsumiert" das aktuelle Token, falls dieses mit `T` übereinstimmt
  - Anderenfalls löst `match()` eine Exception aus
- Referenzen auf Regeln `s` werden durch Methodenaufrufe `s()` aufgelöst

Erinnerung: In ANTLR werden Parser-Regeln (non-Terminale) mit einem kleinen und Lexer-Regeln (Terminale/Token) mit einem großen Anfangsbuchstaben geschrieben.

4.3.3. Alternative Subregeln

```
1  a | b | c
```

kann zu einem **switch**-Konstrukt aufgelöst werden:

```
switch (lookahead):
    case predicting_a:
        a(); break;
    case predicting_b:
        b(); break;
    ...
    default: raise Error()
```

Dabei ist `lookahead` eine globale Variable, die das zu betrachtende Token enthält (vergleichbar mit `peek` beim Lexer).

Das Prädikat `predicting_a` soll andeuten, dass man mit dem aktuellen Token eine Vorhersage für die Regel `a` versucht (hier kommen die FIRST- und FOLLOW-Mengen ins Spiel …). Wenn das der Fall ist, springt man entsprechend in die Funktion bzw. Methode `a()`.

4.3.4. Optionale Subregeln: Eins oder keins

```
(T)?
```

wird zu einem **if** ohne **else**-Teil:

```
if lookahead.predicting_T: match(T)
```

4.3.5. Optionale Subregeln: Mindestens eins

```
(T)+
```

wird zu einer **do**–**while**-Schleife (mind. ein Durchlauf):

```
while True:
    match(T)
    if not lookahead.predicting_T: break
```

4.3.6. LL(1)-Parser

Beispiel: Parsen von Listen, also Sequenzen wie `[1,2,3,4]`:

```
list     : '[' elements ']' ;
elements : INT (',' INT)* ;

INT      : ('0'..'9')+ ;
```

Formal berechnet man die Lookahead-Mengen mit `FIRST` und `FOLLOW`, um eine Entscheidung für die nächste Regel zu treffen. Praktisch betrachtet kann man sich fragen, welche(s) Token eine Phrase in der aktuellen Alternative starten können.

Für LL(1)-Parser betrachtet man immer das **aktuelle** Token (**genau *EIN* Lookahead-Token**), um eine Entscheidung zu treffen.

```
def list():
    match(LBRACK); elements(); match(RBRACK);

def elements():
    match(INT)
    while lookahead == COMMA:  # globale Variable, über consume()
        match(COMMA); match(INT)
```

4.3.7. Detail: *match()* und *consume()*

```
def match(x):
    if lookahead == x: consume()
    else: raise Exception()

def consume():
    lookahead = lexer.nextToken()
```

Quelle: Eigener Code basierend auf einer Idee nach (Parr 2010, p. 43)

Dabei setzt man in der Klasse `Parser` zwei Attribute voraus:

```
class Parser:
    Lexer lexer
    Token lookahead
```

Starten würde man den Parser nach dem Erzeugen einer Instanz (dabei wird ein Lexer mit durchgereicht) über den Aufruf der Start-Regel, also beispielsweise `parser.list()`.

Anmerkung: Mit dem generierten Parse-Tree bzw. *AST* beschäftigen wir uns später (=> AST-basierte Interpreter).

4.3.8. Vorrangregeln

```
1+2*3 == 1+(2*3) != (1+2)*3
```

Die Eingabe `1+2*3` muss als `1+(2*3)` interpretiert werden, da `*` Vorrang vor `+` hat.

Dies formuliert man üblicherweise in der Grammatik:

```
expr : expr '+' term
     | term
     ;
term : term '*' INT
```

```
5       | INT
6       ;
```

ANTLR nutzt die Strategie des "*precedence climbing*" und löst nach der *Reihenfolge der Alternativen* in einer Regel auf. Entsprechend könnte man die obige Grammatik unter Beibehaltung der Vorrangregeln so in ANTLR (v4) formulieren:

```
1  expr : expr '*' expr
2       | expr '+' expr
3       | INT
4       ;
```

4.3.9. Linksrekursion

Normalerweise sind linksrekursive Grammatiken nicht mit einem LL-Parser behandelbar. Man muss die Linksrekursion manuell auflösen und die Grammatik umschreiben.

Beispiel:

```
1  expr : expr '*' expr | expr '+' expr | INT ;
```

Diese linksrekursive Grammatik könnte man (unter Beachtung der Vorrangregeln) etwa so umformulieren:

```
1  expr     : addExpr ;
2  addExpr  : multExpr ('+' multExpr)* ;
3  multExpr : INT ('*' INT)* ;
```

ANTLR (v4) kann Grammatiken mit *direkter* Linksrekursion auflösen. Für frühere Versionen von ANTLR muss man die Rekursion manuell beseitigen.

Vergleiche "ALL(*)" bzw. "Adaptive LL(*)".

Achtung: Mit *indirekter* Linksrekursion kann ANTLR (v4) *nicht* umgehen:

```
1  expr : expM | ... ;
2  expM : expr '*' expr ;
```

=> *Nicht* erlaubt!

4.3.10. Assoziativität

Die Eingabe `2^3^4` sollte als `2^(3^4)` geparst werden. Analog sollte `a=b=c` in C als `a=(b=c)` verstanden werden.

Per Default werden Operatoren wie `+` in ANTLR *links-assoziativ* behandelt, d.h. die Eingabe `1+2+3` wird als `(1+2)+3` gelesen. Für *rechts-assoziative* Operatoren muss man ANTLR dies in der Grammatik mitteilen:

```
expr : expr '^'<assoc=right> expr
     | INT
     ;
```

Anmerkung: Laut Doku gilt die Angabe `<assoc=right>` immer für die jeweilige Alternative und muss seit Version 4.2 an den Alternativen-Operator | geschrieben werden. In der Übergangsphase sei die Annotation an Tokenreferenzen noch zulässig, würde aber ignoriert?!

4.3.11. LL(k)-Parser

```
expr : ID '++'    // x++
     | ID '--'    // x--
     ;
```

Die obige Regel ist für einen LL(1)-Parser nicht deterministisch behandelbar, da die Alternativen mit dem selben Token beginnen (die Lookahead-Mengen überlappen sich). Entweder benötigt man zwei Lookahead-Tokens, also einen LL(2)-Parser, oder man muss die Regel in eine äquivalente LL(1)-Grammatik umschreiben:

```
expr : ID ('++' | '--') ;    // x++ oder x--
```

4.3.12. LL(k)-Parser: Implementierung mit Ringpuffer

Für einen größeren Lookahead benötigt man einen Puffer für die Token. Für einen Lookahead von k Token (also einen LL(k)-Parser) würde man einen Puffer mit k Plätzen anlegen und diesen wie einen Ringpuffer benutzen. Dabei ist `start` der Index des aktuellen Lookahead-Tokens. Über die neue Funktion `lookahead(1)` ist dieses aktuelle Lookahead-Token abrufbar.

```
class Parser:
    Lexer lexer
    k = 3               # Lookahead: 3 Token => LL(3)
    start = 0           # aktuelle Tokenposition im Ringpuffer
    Token[k] lookahead  # Ringpuffer mit k Plätzen (vorbefüllt via Konstruktor)
```

```
def match(x):
    if lookahead(1) == x: consume()
    else: raise Exception()

def consume():
    lookahead[start] = lexer.nextToken()
    start = (start+1) % k

def lookahead(i):
    return lookahead[(start+i-1) % k]  # i==1: start
```

Quelle: Eigener Code basierend auf einer Idee nach (Parr 2010, p. 47)

4.3.13. Wrap-Up

- LL(1) und LL(k) mit festem Lookahead
- Implementierung von Vorrang- und Assoziativitätsregeln
- Beachtung und Auflösung von Linksrekursion

Zum Nachlesen

- Nystrom (2021): Kapitel 5 und 6
- Parr (2010): Kapitel 2 (“Pattern 3: LL(1) Recursive-Decent Parser”)

Lernziele

- k2: Ich kann den prinzipiellen Aufbau von LL-Parsern am Beispiel erklären
- k3: Ich kann LL(1)- und LL(k)-Parser implementieren
- k3: Ich kann Vorrang und Assoziativität bei der Implementierung korrekt umsetzen
- k3: Ich kann mit Linksrekursion umgehen und diese ggf. auflösen

Challenges

Quizzfragen:

- Wie kann man aus einer LL(1)-Grammatik einen LL(1)-Parser mit rekursivem Abstieg implementieren? Wie “übersetzt” man dabei Token und Regeln?
- Wie geht man mit Alternativen um? Wie mit optionalen Subregeln?
- Warum ist Linksrekursion i.A. bei LL-Parsern nicht erlaubt? Wie kann man Linksrekursion beseitigen?
- Wie kann man Vorrangregeln implementieren?
- Wann braucht man mehr als ein Token Lookahead? Geben Sie ein Beispiel an.

Manuell implementierter Parser
Betrachten Sie erneut die folgende einfache Sprache:

```
1 a = 10 - 5     # Zuweisung des Ausdruckes 10-5 (Integer-Wert 5) an Variable
      a
2 b = a + 2 * 3  # Zuweisung von 16 an Variable b
3 c = a != b     # Zuweisung eines boolschen Werts an c
```

Es gibt nur Statements und Expressions:

- Statement: Zuweisung; jedes Statement endet mit einem NL
- Expression: Zahl, Variable, Addition, Subtraktion, Multiplikation (mit üblichem Vorrang), Vergleich

Aufgaben:
In den Challenges von LL Lexer haben Sie eine Grammatik definiert und einen Lexer implementiert.

- Geben Sie nun geeignete Datenstrukturen für den AST an.
- Implementieren Sie analog zum Vorgehen in der Vorlesung einen Parser mit *recursive descent* für diese Sprache.
- Was müssten Sie anpassen bzw. ergänzen, wenn Sie beispielsweise weitere Statements wie eine `if`-Abfrage oder eine `while`-Schleife mit einbauen wollten?

4.4. Syntaxanalyse: LR-Parser (LR(0), LALR)

Weitere Unterlagen

- Annotierte Folien: LR-Parser

4.4.1. Wiederholung

4.4.1.1. Top-Down-Analyse

- Baumaufbau von oben nach unten
- eine Möglichkeit: recursive-descent parser
- alternativ: tabellengesteuerter Parser
- *First*- und *Follow*-Mengen bestimmen Wahl der Ableitungen
- nicht mehr rekursiv, sondern mit PDA

4.4.2. Motivation

4.4.2.1. *LL* ist nicht alles

Die Menge der *LL*-Sprachen ist eine echte Teilmenge der deterministisch kontextfreien Sprachen.

Bei LL-Sprachen muss man nach den ersten k Eingabezeichen entscheiden, welche Ableitung ganz oben im Baum als erste durchgeführt wird, also eine, die im Baum ganz weit weg ist von den Terminalen, die die Entscheidung bestimmen. Das ist nicht bei allen deterministisch parsebaren Grammatiken möglich und erschwert die Fehlerbehandlung.

4.4.3. Von unten nach oben

4.4.3.1. Von unten nach oben

Bei der Bottom-Up-Analyse wird der Parse Tree wird von unten nach oben aufgebaut, von links nach rechts. Dabei entsteht eine *Rechtsableitung*.

Def.: Bei einer kontextfreien Grammatik G ist die *Rechtsableitung* von $\alpha \in (N \cup T)^*$ die Ableitung, die man erhält, wenn das am weitesten rechts stehende Nichtterminal in α abgeleitet wird. Man schreibt $\alpha \overset{*}{\Rightarrow}_r \beta$.

Mit Hilfe der Produktionen und der Vorschautoken werden die Ableitungen “rückwärts” angewandt und “Reduktionen” genannt.

4.4.3.2. Versuchen wir es einmal

Hier entsteht ein Tafelbild.

4.4.3.3. Kann ein Stack helfen?

Hier entsteht ein Tafelbild.

4.4.3.4. So geht es vielleicht

Hier entsteht ein Tafelbild.

4.4.3.5. Da wollen wir hin

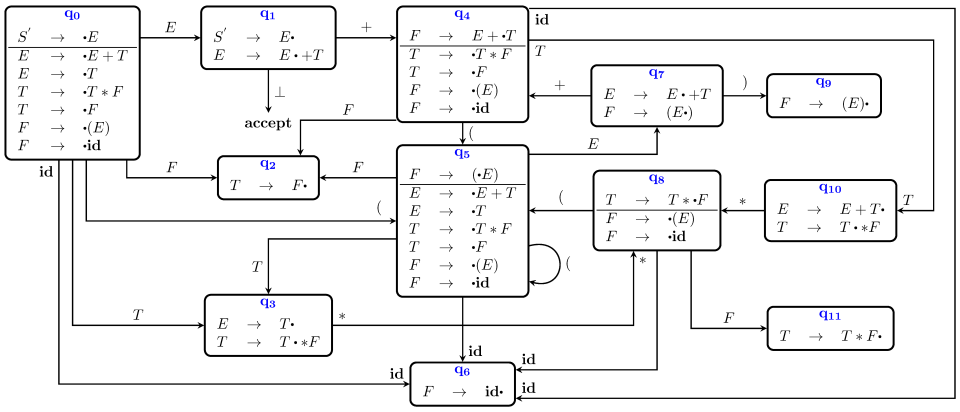


Abbildung 4.5.: Parser-Automat

4.4.4. Theorie: LR(0)

4.4.4.1. Arbeitsweise

Im Stack stehen nur Zustandsnummern, am Anfang die Nummer des Startzustandes (+ Bottomzeichen, oft auch $).

- Lesen des obersten Stackelements ergibt Zustand q
- Lesen des nächsten Eingabezeichens ergibt Zeichen a
- Nachschlagen der Reaktion auf (q, a) in der Parse Table
- Durchführung der Reaktion

4.4.4.2. Mögliche “Actions” ohne Berücksichtigung von Vorschautoken

- Shift: Schiebe logisch das nächste Eingabesymbol auf den Stack (in Wirklichkeit Zustandsnummern)

- Reduce: (Identifiziere ein Handle oben auf dem Stack und ersetze es durch das Nichtterminal der dazugehörigen Produktion.) Das ist gleichbedeutend mit: Entferne so viele Zustände vom Stack wie die rechte Seite der zu reduzierenden Regel Elemente hat, und schreibe den Zustand, der im Goto-Teil für (q, a) steht, auf den Stack.
- Accept: Beende das Parsen erfolgreich
- Reagiere auf einen Syntaxfehler

4.4.4.3. Berechnung der Zustände: Items

Def.: Ein *(dotted) Item* einer Grammatik G ist eine Produktion von G mit einem Punkt auf der rechten Seite der Regel vor, zwischen oder nach den Elementen.

Bsp.:

Zu der Produktion $A \to BC$ gehören die Items:

$[A \to \cdot BC]$

$[A \to B \cdot C]$

$[A \to BC\cdot]$

Das zu $A \to \epsilon$ gehörende Item ist $[A \to \cdot]$

4.4.4.4. Berechnung der *Closure_0* von einer Menge *I* von Items

1. füge I zu $CLOSURE_0(I)$ hinzu
2. gibt es ein Item $[A \to \alpha \cdot B\beta]$ aus $CLOSURE_0(I)$ und eine Produktion $(B \to \gamma)$, füge $[B \to \cdot\gamma]$ zu $CLOSURE_0(I)$ hinzu

4.4.4.5. Berechnung der *GOTO_0*-Sprungmarken

$GOTO_0(I, X) = CLOSURE_0(\{[A \to \alpha X \cdot \beta] \mid [A \to \alpha \cdot X\beta] \in I\})$

für eine Itemmenge I und $X \in N \cup T, A \in N, \alpha, \beta \in (N \cup T)^*$.

4.4.4.6. Konstruktion des $LR(0)$ - Automaten

1. Bilde die Hülle von $S' \to S$ und mache sie zum ersten Zustand.
2. Für jedes noch nicht betrachtete $\cdot X, X \in (N \cup T)$ in einem Zustand q des Automaten berechne $GOTO_0(q, X)$ und mache $GOTO_0(q, X)$ zu einem neuen Zustand r. Verbinde q mit einem Pfeil mit r und schreibe X an den Pfeil. Ist ein zu r identischer Zustand schon vorhanden, wird p mit diesem verbunden und kein neuer erzeugt.

4.4.4.7. Konstruktion der Parse Table

1. Erstelle eine leere Tabelle mit den Zuständen als Zeilenüberschriften. Für den Aktionstabellenteil überschreibe die Spalten mit den Terminalen, für den Sprungtabellenteil mit den Nonterminals.
2. Shift: Für jeden mit einem Terminal beschrifteten Pfeil aus einem Zustand erstelle in der Aktionstabelle die Aktion shift mit der Nummer des Zustands, auf den der Pfeil zeigt. Für Pfeile mit Nonterminals schreibe in die Sprungtabelle nur die Nummer des Folgezustands.
3. Schreibe beim Zustand $[S' \to S\cdot]$ ein $accept$ bei dem Symbol $\perp$.
4. Für jedes Item mit $[A \to \beta\cdot]$ aus allen Zuständen schreibe für alle Terminals $reduce$ und die Nummer der entsprechenden Grammatikregel in die Tabelle.

4.4.4.8. Ein Beispiel zum Nachvollziehen

(0) $S' \to S$

(1) $S \to aAbScS$

(2) $S \to aAbS$

(3) $S \to d$

(4) $A \to e$

4.4.4.9. Der LR(0)-Automat zu G1

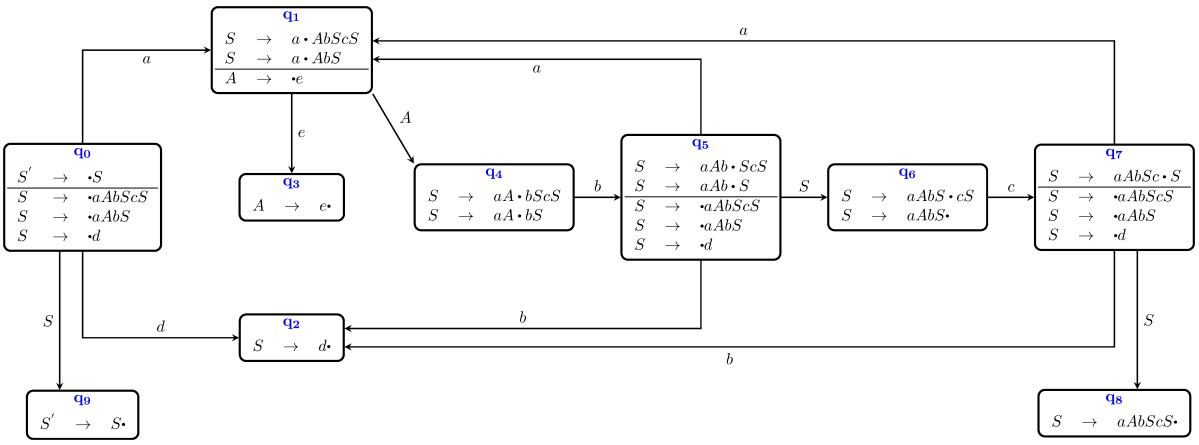


Abbildung 4.6.: LR(0)-Automat

4.4.4.10. Die LR(0)-Parsertabelle zu G1

	Aktion						Sprung	
Zustand	a	b	c	d	e	$\perp$	S	A
q_0	s1			s2			q_9	
q_1					s3			q_4
q_2	r3	r3	r3	r3	r3	r3		
q_3	r4	r4	r4	r4	r4	r4		
q_4		s5						
q_5	s1	s2		s2			q_6	
q_6	r2	r2	s7,r2	r2	r2	r2		
q_7	s1	s2					q_8	
q_8	r1	r1	r1	r1	r1	r1		
q_9						acc		

Abbildung 4.7.: LR(0)-Parsertabelle

4.4.5. Und was gibt es noch?

4.4.5.1. Wenn LR(0) nicht reicht

Zunächst: Zu jeder LR(k)-Sprache gibt es eine LR(1)-Grammatik.

Ist eine Grammatik nicht LR(0), müssen nichtdeterminsistische Tabelleneinträge verhindert werden:

- SLR(1)-Parsing ($A \to \beta$ wird nur reduziert, wenn das Vorschautoken in der $FOLLOW$-Menge von A ist.)
- (kanonisches) LR(1)-Parsing (wie LR(0) mit einem Vorschautoken)
- LALR(1)-Parsing (Zusammenfassung aller LR(1)-Zustände, die sich nur in den LOOKAHEAD-Mengen unterscheiden)

4.4.6. Mehrdeutige Grammatiken

4.4.6.1. Es gibt auch Auswege

Mehrdeutige Grammatiken sind oft leichter zu lesen und kleiner als die Grammatiken, die man erhält, wenn man die Mehrdeutigkeit auflöst, sofern möglich.

Folgendes kann bei Mehrdeutigkeiten helfen:

- Angabe von Vorrangregeln
- Angabe von Assoziativität
- Voreinstellung des Parsergenearators: z. B. Shiften bei Shift-Reduce-Konflikten

- Voreinstellung des Parsergenearators: z. B. Reduzieren nach der Regel, die in der Grammatik zuerst kommt bei Reduce-Reduce-Konflikten

4.4.7. Hierarchie der kontextfreien Sprachen

4.4.7.1. Hierarchie der kontextfreien Sprachen

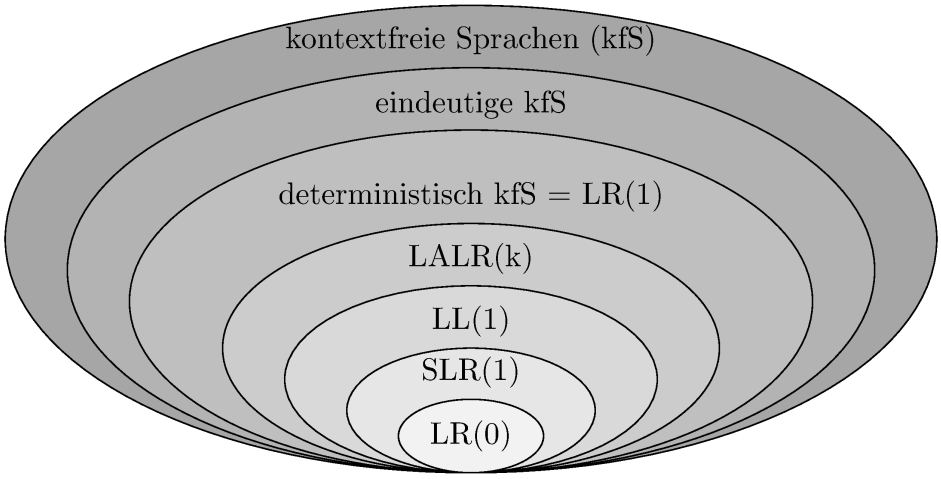


Abbildung 4.8.: Sprachenhierarchie

4.4.8. Wrap-Up

4.4.8.1. Wrap-Up

- LR-Analyse baut den Ableitungbaum von unten nach oben auf
- es wird ein DFA benutzt zusammen mit einem Stack, der Zustände speichert
- eine Parse-Tabelle steuert über Aktions- und Sprungbefehle das Verhalten des Parsers
- die Tabelle wird mit (dotted) Items und Closures konstruiert
- mit Bottom-Up-Parsing LR(1) kann man alle deterministisch kontextfreien Sprachen parsen
- LR(0)-, SLR- und LALR- Parsing sind vereinfachte Verfahren für Teilmengen der LR-Sprachen

Zum Nachlesen

- Aho u. a. (2023): Kapitel 4.5 bis 4.8
- Aho u. a. (2023)
- Hopcroft u. a. (2003)
- Kunert (2018)
- Wagenknecht und Hielscher (2014)

Lernziele

- k1: Ich kenne die Prinzipien der Bottom-Up-Analyse
- k1: Ich kenne die Begriffe Item, Closure und Parse Table
- k2: Ich kann LR(0)-Parsing an einem Beispiel erklären
- k3: Ich kann die Konstruktion der Parse Tables durchführen
- k3: Ich kann das Parsen mit einem LR(0)-Parser durchführen

5. Semantische Analyse

Auf die lexikalische Analyse und die Syntaxanalyse folgt die semantische Analyse. Nach dem Parsen steht fest, dass ein Programm syntaktisch korrekt ist. Nun muss geprüft werden, ob es auch semantisch korrekt ist. Dazu gehören u.a. die Identifikation und Sammlung von Bezeichnern und die Zuordnung zur richtigen Ebene (Scopes) sowie die die Typ-Prüfung und -Inferenz.

In dieser Phase zeigen sich die Eigenschaften der zu verarbeitenden Sprache sehr deutlich, beispielsweise müssen Bezeichner deklariert sein vor der ersten Benutzung, welche Art von Scopes soll es geben, gibt es Klassen und Vererbung …

Da hier der Kontext der Symbole eine Rolle spielt, wird diese Phase oft auch “Context Handling” oder “Kontext Analyse” bezeichnet. Neben attributierten Grammatiken sind die Symboltabellen wichtige Werkzeuge.

5.1. SymbTab0: Überblick Symboltabellen

TL;DR

Auf die lexikalische Analyse und die Syntaxanalyse folgt die semantische Analyse. Nach dem Parsen steht fest, dass ein Programm syntaktisch korrekt ist. Nun muss geprüft werden, ob es auch semantisch korrekt ist. Dies umfasst in der Regel die Identifikation und Sammlung von Bezeichnern und die Zuordnung zur richtigen Ebene (Scopes). Außerdem muss die Nutzung von Symbolen validiert werden: Je nach Sprache müssen beispielsweise Variablen und Funktionen vor ihrer Benutzung zumindest deklariert sein; Funktionen sollten sich nicht wie Variablen benutzen lassen, …
Als Werkzeug werden (hierarchische) Tabellen eingesetzt, um die verschiedenen Symbole und Informationen darüber zu verwalten. Dabei werden die Symboltabelleneinträge oft an verschiedenen Stellen im Compiler generiert und benutzt.

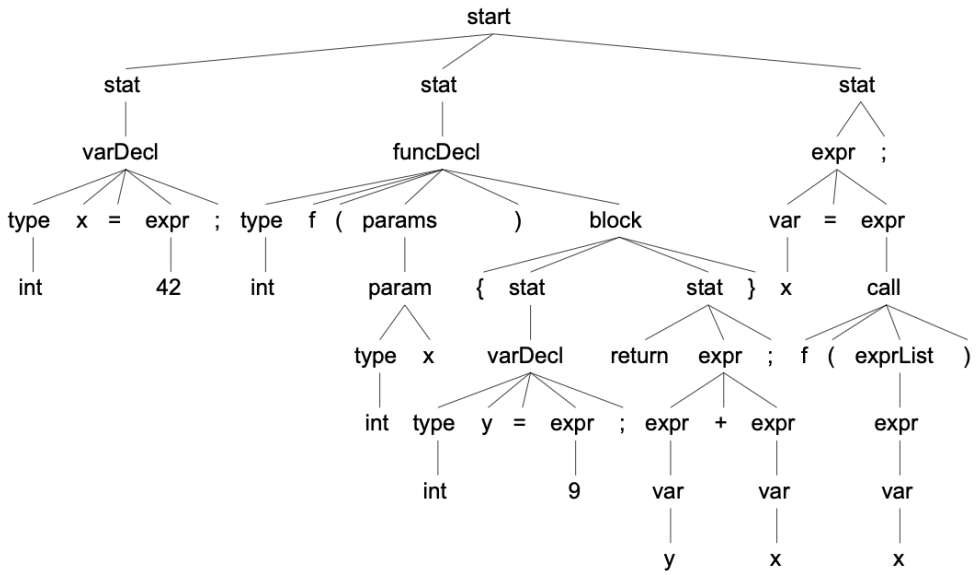
Videos

Vorlesung [YT], [HSBI]

5.1.1. Was passiert nach der Syntaxanalyse?

```
1  int x = 42;
2  int f(int x) {
3      int y = 9;
4      return y+x;
```

```
5  }
6
7  x = f(x);
```



Nach der Syntaxanalyse braucht der Compiler für die darauf folgenden Phasen **semantische Analyse**, Optimierung und Codegenerierung **Informationen über Bezeichner**, z.B.

- Welcher Bezeichner ist gemeint?
- Welchen Typ hat ein Bezeichner?

Auf dem Weg zum Interpreter/Compiler müssen die Symbole im AST korrekt zugeordnet werden. Dies geschieht über Symboltabellen. Im Folgenden werden wir verschiedene Aspekte von Symboltabellen betrachten und eine mögliche Implementierung erarbeiten, bevor wir uns (in Interpreter) um die Auswertung (Interpretation) des AST kümmern können.

5.1.1.1. Logische Compilierungsphasen

- Die lexikalische Analyse generiert eine Folge von Token.
- Die Syntaxanalyse generiert einen Parse Tree.

- Die semantische Analyse macht folgendes:
  - Der Parse Tree wird in einen abstrakten Syntaxbaum (AST) umgewandelt.
  - Dieser wird häufig mit Attributen annotiert.
  - Dabei sind oft mehrere Baumdurchläufe nötig (z.B. wegen der Abhängigkeiten der Attribute).

- Nachfolgende Stufen:
  - Der AST wird in einen Zwischencode umgewandelt mit Registern und virtuellen Adressen.

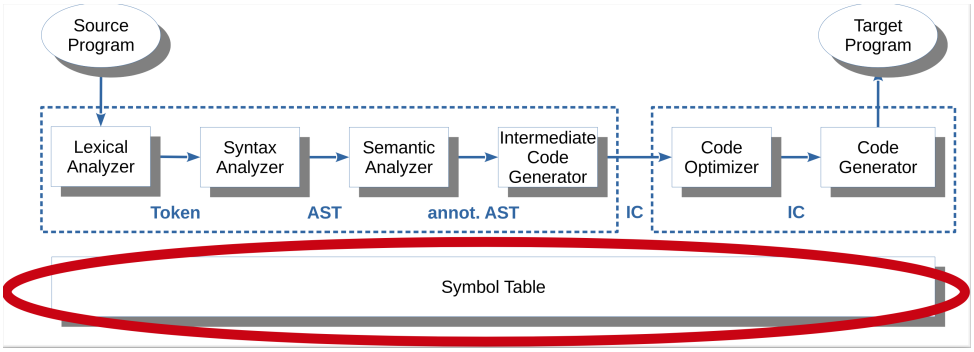
- Der Zwischencode wird optimiert.
- Aus dem optimierten Zwischencode wird der endgültige Code, aber immer noch mit virtuellen Adressen, generiert.
- Der generierte Code wird nachoptimiert.
- Der Linker ersetzt die virtuellen Adressen durch reale Adressen.

5.1.1.2. Abgrenzung der Phasen

Diese Phasen sind oft nicht klar unterscheidbar. Schon allein zur Verbesserung der Laufzeit baut der Parser oft schon den abstrakten Syntaxbaum auf, der Lexer trägt schon Bezeichner in Symboltabellen ein, der Parser berechnet beim Baumaufbau schon Attribute, …

Oft werden gar nicht alle Phasen und alle Zwischendarstellungen benötigt.

5.1.2. Semantische Analyse und Symboltabellen



5.1.3. Syntax und Semantik

- **Syntaxregeln**: Formaler Aufbau eines Programms
- **Semantik**: Bedeutung eines (syntaktisch korrekten) Programms

=> Keine Codegenerierung für syntaktisch/semantisch inkorrekte Programme!

Zur Erinnerung: Die *Syntaxregeln* einer Programmiersprache bestimmen den formalen Aufbau eines zu übersetzenden Programms. Die *Semantik* gibt die Bedeutung eines syntaktisch richtigen Programms an.

Lexikalische und syntaktische Analyse können formalisiert mit regulären Ausdrücken und endlichen Automaten, sowie mit CFG und Parsern durchgeführt werden.

Die Durchführung der semantischen Analyse ist stark von den Eigenschaften der zu übersetzenden Sprache, sowie der Zielsprache abhängig und kann hier nur beispielhaft für einige Eigenschaften erklärt werden.

Es darf kein lauffähiges Programm erstellt werden können, dass nicht syntaktisch und semantisch korrekt ist. Ein lauffähiges Programm muss syntaktisch und semantisch korrekt sein!

5.1.4. Aufgaben der semantischen Analyse

- Identifikation und Sammlung der Bezeichner
- Zuordnung zur richtigen Ebene (Scopes)

- Typ-Inferenz
- Typkonsistenz (Ausdrücke, Funktionsaufrufe, …)

- Validieren der Nutzung von Symbolen
  - Vermeidung von Mehrfachdefinition
  - Zugriff auf nicht definierte Bezeichner
  - (Lesender) Zugriff auf nicht initialisierte Bezeichner
  - Funktionen werden nicht als Variablen genutzt
  - …

Die semantische Analyse überprüft die Gültigkeit eines syntaktisch korrekten Programms bzgl. statischer semantischer Eigenschaften und liefert die Grundlage für die (Zwischen-) Codeerzeugung und -optimierung. Insbesondere wird hier die Typkonsistenz (in Ausdrücken, von Parametern, …) überprüft, und implizite Typumwandlungen werden vorgenommen. Oft müssen Typen automatisch bestimmt werden (z.B. bei Polymorphie, Typinferenz). Damit Typen bestimmt oder angepasst werden können, müssen Bezeichner zunächst identifiziert werden, d.h. bei namensgleichen Bezeichnern der richtige Bezug bestimmt werden.

Zu Annotationen/Attributen, Typen und Type-Checks siehe VL Typprüfungen, Attributgrammatiken

=> Ein wichtiges Hilfsmittel dazu sind **Symboltabellen**

5.1.4.1. Identifizierung von Objekten

Beim Compiliervorgang müssen Namen immer wieder den dazugehörigen Definitionen zugeordnet, ihre Eigenschaften gesammelt und geprüft und darauf zugegriffen werden. Symboltabellen werden im Compiler fast überall gebraucht (siehe Abbildung unter “Einordnung”).

Welche Informationen zu einem Bezeichner gespeichert und ermittelt werden, ist dann abhängig von der Klasse des Bezeichners.

5.1.4.2. Validieren der Nutzung von Symbolen

Hier sind unendlich viele Möglichkeiten denkbar. Dies reicht von den unten aufgeführten Basisprüfungen bis hin zum Prüfen der Typkompatibilität bei arithmetischen Operationen. Dabei müssen für alle Ausdrücke die Ergebnistypen berechnet werden und ggf. automatische Konvertierungen vorgenommen werden, etwa bei `3+4.1`…

- Zugriff auf Variablen: Müssen sichtbar sein
- Zugriff auf Funktionen: Vorwärtsreferenzen sind OK

- Variablen werden nicht als Funktionen genutzt
- Funktionen werden nicht als Variablen genutzt

=> Verweis auf VL Typprüfungen, Attributgrammatiken

Da Funktionen bereits vor dem Bekanntmachen der Definition aufgerufen werden dürfen, bietet sich ein **zweimaliger Durchlauf** (*pass*) an: Beim ersten Traversieren des AST werden alle Definitionen in der Symboltabelle gesammelt. Beim zweiten Durchlauf werden dann die Referenzen aufgelöst.

5.1.4.3. Das Mittel der Wahl: Tabellen für die Symbole (= Bezeichner)

Def.: *Symboltabellen* sind die zentrale Datenstruktur zur Identifizierung und Verwaltung von bezeichneten Elementen.

Die Organisation der Symboltabellen ist stark anwendungsabhängig. Je nach Sprachkonzept gibt es eine oder mehrere Symboltabellen, deren Einträge vom Lexer oder Parser angelegt werden. Die jeweiligen Inhalte jedes einzelnen Eintrags kommen aus den verschiedenen Phasen der Compilierung. Symboltabellen werden oft als Hashtables oder auch als Bäume implementiert, manchmal als verkettete Listen. In seltenen Fällen kommt man auch mit einem Stack aus.

Eine Symboltabelle enthält benutzerdefinierte Bezeichner (oder Verweise in eine Hashtable mit allen vorkommenden Namen), manchmal auch die Schlüsselwörter der Programmiersprache. Die einzelnen Felder eines Eintrags variieren stark, abhängig vom Typ des Bezeichners (= Bezeichnerklasse).

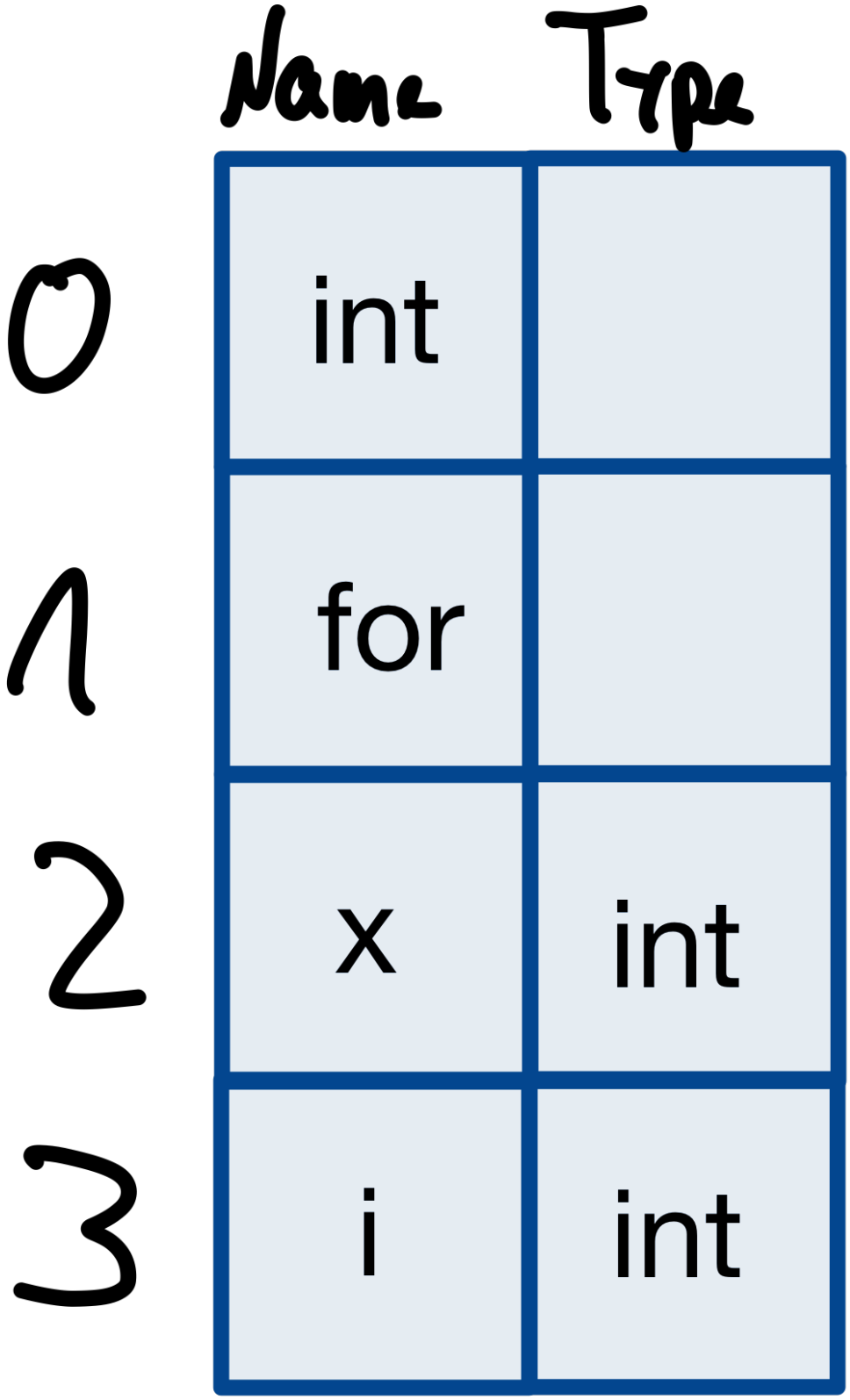
Manchmal gibt es für Datentypen eine Extra-Tabelle, ebenso eine für die Werte von Konstanten.

Manchmal werden die Namen selbst in eine (Hash-) Tabelle geschrieben. Die Symboltabelle enthält dann statt der Namen Verweise in diese (Hash-) Tabelle.

5.1.5. Einfache Verwaltung von Variablen primitiven Typs

```
int x = 0;
int i = 0;

for (i=0; i<10; i++) {
    x++;
}
```


Name
Type
0
int
1
for
2
x
int
3
i
int

Bsp.: Die zu übersetzende Sprache hat nur einen (den globalen) Scope und kennt nur Bezeichner für Variablen.

- **Eine** Symboltabelle für **alle** Bezeichner
- Jeder Bezeichner ist der Name einer Variablen
- Symboltabelle wird evtl. mit Einträgen aller Schlüsselwörter initialisiert – warum?
- Scanner erkennt Bezeichner und sucht ihn in der Symboltabelle
- Ist der Bezeichner nicht vorhanden, wird ein (bis auf den Namen leerer) Eintrag angelegt
- Scanner übergibt dem Parser das erkannte Token und einen Verweis auf den Symboltabelleneintrag

Die Symboltabelle könnte hier eine (Hash-) Tabelle oder eine einfache verkettete Liste sein.

5.1.6. Was kann jetzt weiter passieren?

```
1  int x = 0;
2  int i = 0;
3
4  for (i=0; i<10; i++) {
5      x++;
6  }
7
8  a = 42;
```

In vielen Sprachen muss überprüft werden, ob es ein definierendes Vorkommen des Bezeichners oder ein angewandtes Vorkommen ist.

5.1.6.1. Definitionen und Deklarationen von Bezeichnern

Def.: Die *Definition* eines (bisher nicht existenten) Bezeichners in einem Programm generiert einen neuen Bezeichner und legt für ihn seinem Typ entsprechend Speicherplatz an.

Def.: Unter der *Deklaration* eines (bereits existierenden) Bezeichners verstehen wir seine Bekanntmachung, damit er benutzt werden kann. Er ist oft in einem anderen Scope definiert und bekommt dort Speicherplatz zugeteilt.

Insbesondere werden auch Typen deklariert. Hier gibt es in der Regel gar keine Speicherplatzzuweisung.

Ein Bezeichner kann beliebig oft deklariert werden, während er in einem Programm nur einmal definiert werden kann. Oft wird bei der Deklarationen eines Elements sein Namensraum mit angegeben.

Vorsicht: Die Begriffe werden auch anders verwendet. Z.B. findet sich in der Java-Literatur der Begriff *Deklaration* anstelle von *Definition*.

Anmerkung: Deklarationen beziehen sich auf Definitionen, die woanders in einer Symboltabelle stehen, evtl. in einer anderen Datei, also in diesem Compilerlauf nicht zugänglich sind und erst von Linker aufgelöst werden können. Beim Auftreten einer Deklaration muss die dazugehörige Definition gesucht werden,und wenn vorhanden, im Symboltabelleneintrag für den deklarierten Bezeichner festgehalten werden. Hier ist evtl. ein zweiter Baumdurchlauf nötig, um alle offenen Deklarationen, die sich auf Definitionen in derselben Datei beziehen, aufzulösen.

Wird bei objektorientierten Sprachen ein Objekt definiert, dessen Klassendefinition in einer anderen Datei liegt, kann man die Definition des Objekts gleichzeitig als Deklaration der Klasse auffassen (Java).

5.1.7. Wo werden Verweise in Symboltabellen gebraucht?

=> Parse Tree und AST enthalten Verweise auf Symboltabelleneinträge

- Im Parse Tree enthält der Knoten für einen Bezeichner einen Verweis auf den Symboltabelleneintrag.
- Parser und semantische Analyse (AST) vervollständigen die Einträge.
- Attribute des AST können Feldern der Symboltabelle entsprechen, bzw. sich aus ihnen berechnen.
- Für Debugging-Zwecke können die Symboltabellen die ganze Compilierung und das Linken überleben.

5.1.8. Grenzen der semantischen Analyse

Welche semantischen Eigenschaften einer Sprache kann die semantische Analyse nicht überprüfen?

- Wer ist dann dafür verantwortlich?
- Wie äußert sich das im Fehlerfall?

Dinge, die erst durch eine Ausführung/Interpretation eines Programms berechnet werden können.

Beispielsweise können Werte von Ausdrücken oft erst zur Laufzeit bestimmt werden. Insbesondere kann die semantische Analyse in der Regel nicht feststellen, ob ein Null-Pointer übergeben wird und anschließend dereferenziert wird.

5.1.9. Wrap-Up

- Semantische Analyse:
  - Identifikation und Sammlung der Bezeichner
  - Zuordnung zur richtigen Ebene (Scopes)
  - Validieren der Nutzung von Symbolen
  - Typ-Inferenz
  - Typkonsistenz (Ausdrücke, Funktionsaufrufe, …)
- Symboltabellen: Verwaltung von Symbolen und Typen (Informationen über Bezeichner)
- Symboltabelleneinträge werden an verschiedenen Stellen des Compilers generiert und benutzt

Zum Nachlesen

- Mogensen (2017): Kapitel 3
- Parr (2014): Kapitel 6.4 und 8.4
- Parr (2010): Kapitel 6, 7 und 8

Lernziele

- k2: Ich kann die Aufgaben der semantischen Analyse an einem Beispiel erklären
- k2: Ich kann die Bedeutung von Symboltabellen erklären und die Aufgaben und Verbindung zu den Compiler-Phasen verdeutlichen

5.2. SymbTab1: Nested Scopes

TL;DR

In Symboltabellen werden Informationen über Bezeichner verwaltet. Wenn es in der zu übersetzenden Sprache *Nested Scopes* gibt, spiegelt sich dies in den Symboltabellen wider: Auch hier wird eine entsprechende hierarchische Organisation notwendig. In der Regel nutzt man Tabellen, die untereinander verlinkt sind.
Eine wichtige Aufgabe ist das Binden von Bezeichner gleichen Namens an ihren jeweiligen Scope => `bind()`. Zusätzlich müssen Symboltabellen auch das Abrufen von Bezeichnern aus dem aktuellen Scope oder den Elternscopes unterstützen => `resolve()`.

Videos

Vorlesung [YT], [HSBI]

5.2.1. Scopes und Name Spaces

Def.: Unter dem *Gültigkeitsbereich* (Sichtbarkeitsbereich, Scope) eines Bezeichners versteht man den Programmabschnitt, in dem der Bezeichner sichtbar und nutzbar ist. Das ist oft der kleinste umgebende Block, außer darin enthaltene Scopes, die ein eigenes Element dieses Namens benutzen.

Scopes sind fast immer hierarchisch angeordnet.

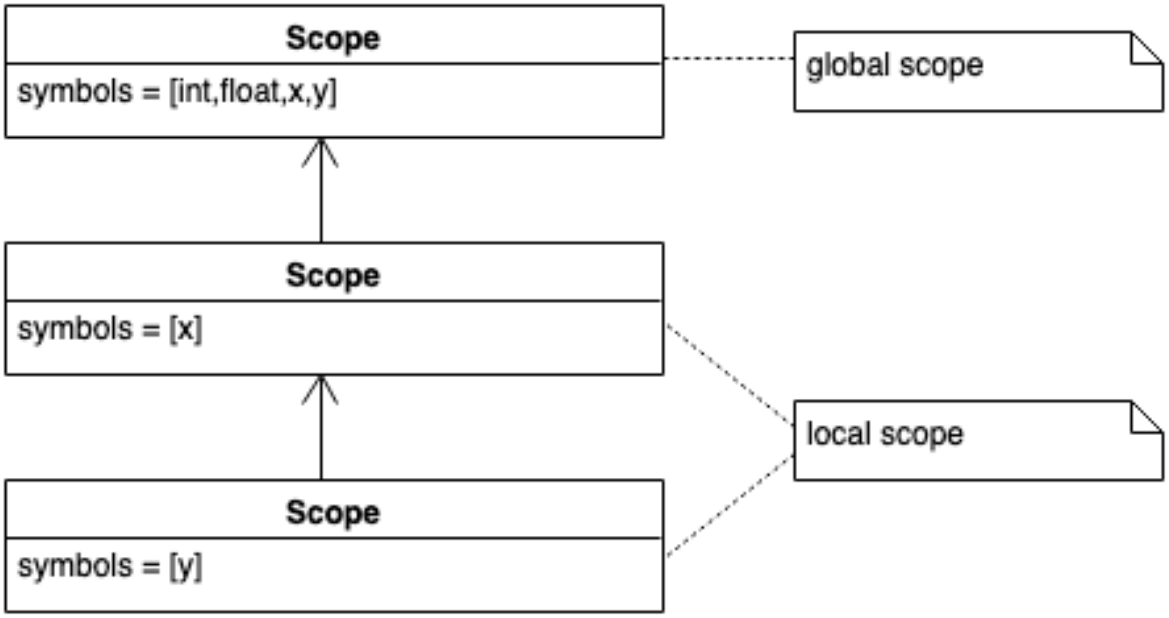
Def.: Unter einem *Namensraum* (name space) versteht man die Menge der zu einem Zeitpunkt sichtbaren Bezeichner.

Es gibt Sprachen, in denen man eigene Namensräume explizit definieren kann (z.B. C++).

Vorsicht: Diese Begriffe werden nicht immer gleich definiert und auch gerne verwechselt.

5.2.2. Symbole und (nested) Scopes

```
1  int x = 42;
2  float y;
3  {
4      int x;
5      x = 1;
6      y = 2;
7      { int y = x; }
8  }
```



Aufgaben:

- `bind()`: Symbole im Scope definieren
- `resolve()`: Symbole aus Scope oder Eltern-Scope abrufen

5.2.2.1. Hinzunahme von Scopes

Bsp.: Die zu übersetzende Sprache ist scope-basiert und kennt nur Bezeichner für Variablen

Scopes können ineinander verschachtelt sein. Die Spezifikation der zu übersetzenden Sprache legt fest, in welcher Reihenfolge Scopes zu durchsuchen sind, wenn auf einen Bezeichner Bezug genommen wird, der nicht im aktuellen Scope definiert ist.

Insgesamt bilden die Scopes oft eine Baumstruktur, wobei jeder Knoten einen Scope repräsentiert und seine Kinder die direkt in ihm enthaltenen Scopes sind. Dabei ist es in der Regel so, dass Scopes sich entweder vollständig überlappen oder gar nicht. Wenn ein Bezeichner nicht im aktuellen Scope vorhanden ist, muss er in der Regel in umschließenden Scopes gesucht werden. Hier kann ein Stack aller “offenen” Scopes benutzt werden.

5.2.2.2. Grundlegendes Vorgehen

Das Element, das einen neuen Scope definiert, steht selbst in dem aktuell behandelten Scope. Wenn dieses Element selbst ein Bezeichner ist, gehört dieser in den aktuellen Scope. Nur das, was nur innerhalb des oben genannten Elements oder Bezeichners definiert wird, gehört in den Scope des Elements oder Bezeichners.

5.2.3. Nested Scopes: Symbole und Scopes

5.2.3.1. Implementierung mit hierarchischen (verketteten) Tabellen

Pro Scope wird eine Symboltabelle angelegt, dabei enthält jede Symboltabelle zusätzlich einen Verweis auf ihre Vorgängersymboltabelle für den umgebenden Scope. Die globale Symboltabelle wird typischerweise mit allen Schlüsselwörtern initialisiert.

- Wenn ein neuer Scope betreten wird, wird eine neue Symboltabelle erzeugt.
- Scanner: Erkennt Bezeichner und sucht ihn in der Symboltabelle des aktuellen Scopes bzw. trägt ihn dort ein und übergibt dem Parser das erkannte Token und einen Verweis auf den Symboltabelleneintrag (**Erinnerung**: Der Scanner wird i.d.R. vom Parser aus aufgerufen, d.h. der Parser setzt den aktuellen Scope!)
- Parser:
  - Wird ein neues Element (ein Bezeichner) definiert, muss bestimmt werden, ob es einen eigenen Scope hat. Wenn ja, wird eine neue Symboltabelle für den Scope angelegt. Sie enthält alle Definitionen von Elementen, die in diesem Scope liegen. Der Bezeichner selbst wird in die aktuelle Symboltabelle eingetragen mit einem Verweis auf die neue Tabelle, die all die Bezeichner beinhaltet, die außerhalb dieses Scopes nicht sichtbar sein sollen. Die Tabellen werden untereinander verzeigert.
  - Wird ein Element deklariert oder benutzt, muss sein Eintrag in allen sichtbaren Scopes in der richtigen Reihenfolge entlang der Verzeigerung gesucht (und je nach Sprachdefinition auch gefunden) werden.
- Der Parse-Tree enthält im Knoten für den Bezeichner den Verweis in die Symboltabelle

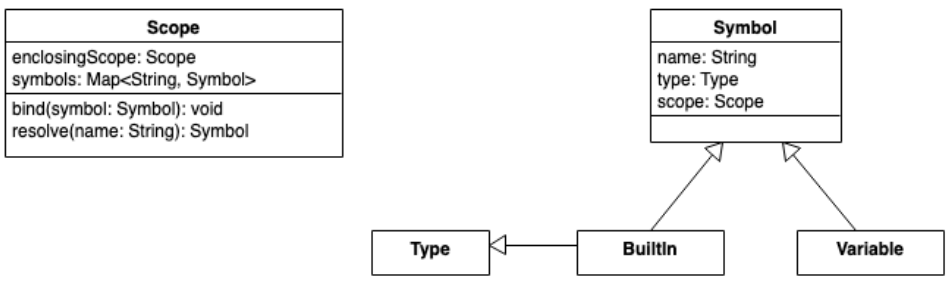
5.2.3.2. Klassenhierarchie für Scopes

Für die Scopes wird eine Klasse `Scope` definiert mit den Methoden `bind()` (zum Definieren von Symbolen im Scope) und `resolve()` (zum Abrufen von Symbolen aus dem Scope oder dem umgebenden Scope).

Für lokale Scopes wird eine Instanz dieser Klasse angelegt, die eine Referenz auf den einschließenden Scope im Attribut `enclosingScope` hält. Für den globalen Scope ist diese Referenz einfach leer (`None`).

5.2.3.3. Klassen und Interfaces für Symbole

Für die Symbole gibt es die Klasse `Symbol`, wo für jedes Symbol Name und Typ gespeichert wird. Variablensymbole leiten direkt von dieser Klasse ab. Für die eingebauten Typen wird ein “Marker-Interface” `Type` erstellt, um Variablen- und Typ-Symbole unterscheiden zu können.



Quelle: Nested Scopes: Eigene Modellierung nach einer Idee in (Parr 2010, p. 142)

5.2.3.4. Alternative Implementierung über einen Stack

- Der Parse Tree bzw. der AST enthalten an den Knoten, die jeweils einen ganzen Scope repräsentieren, einen Verweis auf die Symboltabelle dieses Scopes.
- Die Scopes werden in einem Stack verwaltet.
- Wird ein Scope betreten beim Baumdurchlauf, wird ein Verweis auf seine Symboltabelle auf den Stack gepackt.
- Die Suche von Bezeichnern in umliegenden Scopes erfordert ein Durchsuchen des Stacks von oben nach unten.
- Beim Verlassen eines Scopes beim Baumdurchlauf wird der Scope vom Stack entfernt.

5.2.4. Nested Scopes: Definieren und Auflösen von Namen

```
1  class Scope:
2      Scope enclosingScope    # None if global (outermost) scope
3      Symbol<String, Symbol> symbols
4
5      def resolve(name):
6          # do we know "name" here?
7          if symbols[name]: return symbols[name]
8          # if not here, check any enclosing scope
9          if enclosingScope: return enclosingScope.resolve(name)
10         else: return None     # not found
11
12     def bind(symbol):
13         symbols[symbol.name] = symbol
14         symbol.scope = self     # track the scope in each symbol
```

Quelle: Nested Scopes: Eigene Implementierung nach einer Idee in (Parr 2010, p. 169)

Anmerkung: In der Klasse `Symbol` kann man ein Feld `scope` vom Typ `Scope` implementieren. Damit "weiss" jedes Symbol, in welchem Scope es definiert ist und man muss sich auf der Suche nach dem Scope eines Symbols ggf. nicht erst durch die Baumstruktur hangeln. Aus technischer Sicht verhindert das Attribut das Aufräumen eines lokalen Scopes durch den Garbage Collector, wenn man den lokalen Scope wieder verlässt: Jeder Scope hat eine Referenz auf den umgebenden (Eltern-) Scope (Feld `enclosingScope`). Wenn man den aktuellen Scope "nach oben" verlässt, würde der eben verlassene lokale Scope bei nächster Gelegenheit aufgeräumt, wenn es keine weiteren Referenzen auf diesen gäbe. Da nun aber die Symbole, die in diesem Scope definiert wurden, auf diesen verweisen, passiert das nicht :)

5.2.5. Nested Scopes: Listener

Mit einem passenden Listener kann man damit die nötigen Scopes aufbauen:

- `enterStart`:

  - erzeuge neuen globalen Scope
  - definiere und pushe die eingebauten Typen
- `exitVarDecl`:
  - löse den Typ der Variablen im aktuellen Scope auf
  - definiere ein neues Variablensymbol im aktuellen Scope
- `exitVar`:
  - löse die Variable im aktuellen Scope auf
- `enterBlock`:
  - erzeuge neuen lokalen Scope, wobei der aktuelle Scope der Elternscope ist
  - ersetze den aktuellen Scope durch den lokalen Scope
- `exitBlock`:
  - ersetze den aktuellen Scope durch dessen Elternscope

```
start   :   stat+ ;

stat    : block | varDecl | expr ';' ;
block   : '{' stat* '}' ;

varDecl : type ID ('=' expr)? ';' ;
expr    : var '=' INT ;

var     : ID ;
type    : 'float' | 'int' ;
```

Relevanter Ausschnitt aus der Grammatik

```
int x = 42;

{ int y = 9; x = 7; }
```

```
class MyListener(BaseListener):
    Scope scope

    def enterStart(Parser.FileContext ctx):
        globals = Scope()
        globals.bind(BuiltIn("int"))
        globals.bind(BuiltIn("float"))
        scope = globals

    def enterBlock(Parser.BlockContext ctx):
        scope = Scope(scope)
    def exitBlock(Parser.BlockContext ctx):
        scope = scope.enclosingScope

    def exitVarDecl(Parser.VarDeclContext ctx):
        t = scope.resolve(ctx.type().getText())
        var = Variable(ctx.ID().getText(), t)
```

```
18          scope.bind(var)
19      def exitVar(Parser.VarContext ctx):
20          name = ctx.ID().getText()
21          var = scope.resolve(name)
22          if var == None: error("no such var: " + name)
```

Anmerkung: Um den Code auf die Folie zu bekommen, ist dies ein Mix aus Java und Python geworden. Sry ;)

In der Methode `exitVar()` wird das Variablensymbol beim Ablaufen des AST lediglich aufgelöst und ein Fehler geworfen, wenn das Variablensymbol (noch) nicht bekannt ist. Hier könnte man weiteres Type-Checking und/oder -Propagation ansetzen.

Später im Interpreter muss an dieser Stelle dann aber auch der **Wert** der Variablen abgerufen werden …

5.2.6. Löschen von Symboltabellen

Möglicherweise sind die Symboltabellen nach der Identifizierungsphase der Elemente überflüssig, weil die zusammengetragenen Informationen als Attribute im AST stehen. Die Knoten enthalten dann Verweise auf definierende Knoten von Elementen, nicht mehr auf Einträge in den Symboltabellen. In diesem Fall können die Symboltabellen nach der Identifizierung gelöscht werden, wenn sie nicht z.B. für einen symbolischen Debugger noch gebraucht werden.

5.2.7. Wrap-Up

- Symboltabellen: Verwaltung von Symbolen und Typen (Informationen über Bezeichner)

- Blöcke: Nested Scopes => hierarchische Organisation
- Binden von Bezeichner gleichen Namens an ihren jeweiligen Scope => `bind()`
- Abrufen von Bezeichnern aus dem aktuellen Scope oder den Elternscopes => `resolve()`

Zum Nachlesen

- Mogensen (2017): Kapitel 3
- Parr (2014): Kapitel 6.4 und 8.4
- Parr (2010): Kapitel 6, 7 und 8

Lernziele

- k3: Ich kann Symboltabellen für Nested Scopes implementieren unter Nutzung von passenden Strukturen/Klassen und einem Listener
- k3: Ich kann Symbole über die Scopes auflösen
- k3: Ich kann einfache statische Prüfungen anhand der Symboltabellen durchführen

5.3. SymbTab2: Funktionen

TL;DR

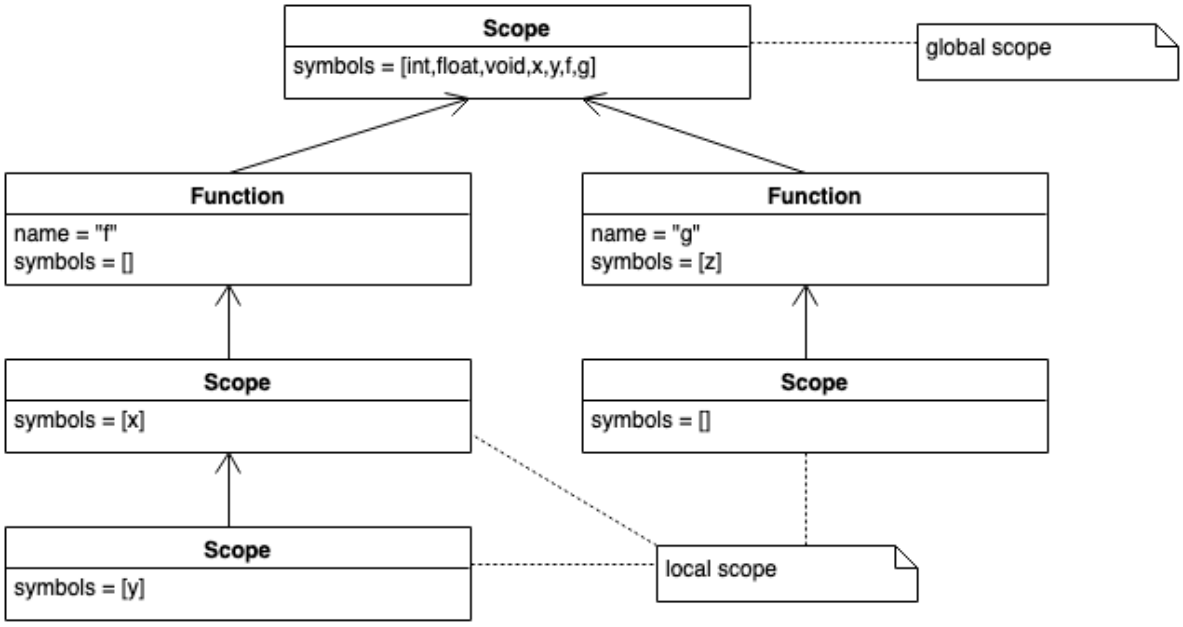
Eine Funktion sind selbst ein Symbol, welches in einem Scope gilt und entsprechend in der Symboltabelle eingetragen wird. Darüber hinaus bildet sie einen neuen verschachtelten Scope, in dem die Funktionsparameter und der Funktionskörper definiert werden müssen.
Entsprechend müssen die Strukturen für die Symboltabellen sowie das Eintragen und das Auflösen von Symbolen erweitert werden.

Videos

Vorlesung [YT], [HSBI]

5.3.1. Funktionen und Scopes

```
1  int x = 42;
2  int y;
3  void f() {
4      int x;
5      x = 1;
6      y = 2;
7      { int y = x; }
8  }
9  void g(int z){}
```



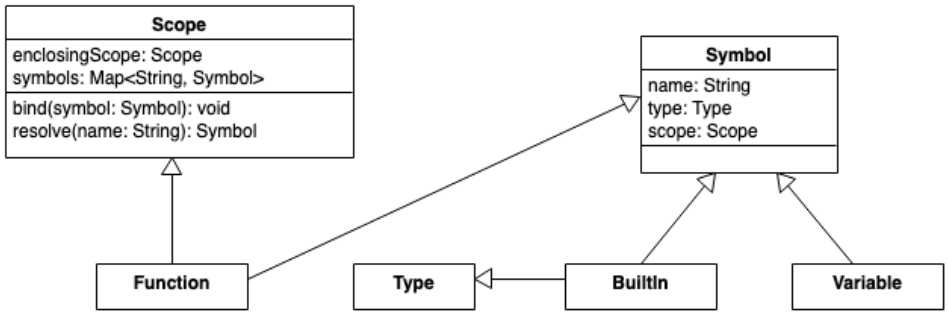
5.3.1.1. Behandlung von Funktionsdefinitionen

- Jeder Symboltabelleneintrag braucht ein Feld, das angibt, ob es sich um eine Variable, eine Funktion, … handelt. Alternativ eine eigene Klasse ableiten …
- Der Name der Funktion steht als Bezeichner in der Symboltabelle des Scopes, in dem die Funktion definiert wird.
- Der Symboltabelleneintrag für den Funktionsnamen enthält Verweise auf die Parameter.
- Der Symboltabelleneintrag für den Funktionsnamen enthält Angaben über den Rückgabetypen.
- Jede Funktion wird grundsätzlich wie ein neuer Scope behandelt.
- Die formalen Parameter werden als Einträge in der Symboltabelle für den Scope der Funktion angelegt and entsprechend als Parameter gekennzeichnet.

5.3.1.2. Behandlung von Funktionsaufrufen

- Der Name der Funktion steht als Bezeichner in der Symboltabelle des Scopes, in dem die Funktion aufgerufen wird und wird als Aufruf gekennzeichnet.
- Der Symboltabelleneintrag für den Funktionsnamen enthält Verweise auf die aktuellen Parameter.
- Die Definition der Funktion wird in den zugänglichen Scopes gesucht (wie oben) und ein Verweis darauf in der Symboltabelle gespeichert.

5.3.2. Erweiterung des Klassendiagramms für Funktions-Scopes



Quelle: Funktions-Scopes: Eigene Modellierung nach einer Idee in (Parr 2010, p. 147)

5.3.3. Funktionen sind Symbole *und* Scopes

```
1  class Function(Scope, Symbol):
2      def __init__(name, retType, enclScope):
3          Symbol.__init__(name, retType)      # we are "Symbol" ...
4          enclosingScope = enclScope          # ... and "Scope"
```

5.3.4. Funktionen: Listener

Den Listener zum Aufbau der Scopes könnte man entsprechend erweitern:

- `enterFuncDecl`:
  - löse den Typ der Funktion im aktuellen Scope auf
  - lege neues Funktionssymbol an, wobei der aktuelle Scope der Elternscope ist
  - definiere das Funktionssymbol im aktuellen Scope
  - ersetze den aktuellen Scope durch das Funktionssymbol
- `exitFuncDecl`:
  - ersetze den aktuellen Scope durch dessen Elternscope
- `exitParam`: analog zu `exitVarDecl`
  - löse den Typ der Variablen im aktuellen Scope auf
  - definiere ein neues Variablensymbol im aktuellen Scope
- `exitCall`: analog zu `exitVar`
  - löse das Funktionssymbol (und die Argumente) im aktuellen Scope auf

```
funcDecl : type ID '(' params? ')' block ;
params   : param (',' param)* ;
param    : type ID ;

call     : ID '(' exprList? ')' ;
exprList : expr (',' expr)* ;
```

Relevanter Ausschnitt aus der Grammatik

```
int f(int x) {
    int y = 9;
}

int x = f(x);
```

```
def enterFuncDecl(Parser.FuncDeclContext ctx):
    name = ctx.ID().getText()
    type = scope.resolve(ctx.type().getText())
    func = Function(name, type, scope)
    scope.bind(func)
    # change current scope to function scope
    scope = func

def exitFuncDecl(Parser.FuncDeclContext ctx):
    scope = scope.enclosingScope
def exitParam(Parser.ParamContext ctx):
    t = scope.resolve(ctx.type().getText())
    var = Variable(ctx.ID().getText(), t)
    scope.bind(var)

def exitCall(Parser.CallContext ctx):
```

```
17      name = ctx.ID().getText()
18      func = scope.resolve(name)
19      if func == None:
20          error("no such function: " + name)
21      if func.type == Variable:
22          error(name + " is not a function")
```

Anmerkung: Um den Code auf die Folie zu bekommen, ist dies wieder ein Mix aus Java und Python geworden. Sry ;)

Im Vergleich zu den einfachen *nested scopes* kommt hier nur ein weiterer Scope für den Funktionskopf dazu. Dieser spielt eine Doppelrolle: Er ist sowohl ein Symbol (welches im Elternscope bekannt ist) als auch ein eigener (lokaler) Scope für die Funktionsparameter.

Um später im Interpreter eine Funktion tatsächlich auswerten zu können, muss im Scope der Funktion zusätzlich der AST-Knoten der Funktionsdefinition gespeichert werden (weiteres Feld/Attribut in `Function`)!

5.3.5. Wrap-Up

- Symboltabellen: Verwaltung von Symbolen und Typen (Informationen über Bezeichner)
- Funktionen: Nested Scopes => hierarchische Organisation
- Umgang mit dem Funktionsnamen, den Parametern und dem Funktionskörper

Zum Nachlesen

- Mogensen (2017): Kapitel 3
- Parr (2014): Kapitel 6.4 und 8.4
- Parr (2010): Kapitel 6, 7 und 8

Lernziele

- k3: Ich kann Symboltabellen für Nested Scopes implementieren unter Nutzung von passenden Strukturen/Klassen und einem Listener
- k3: Ich kann Parameter und lokale Variablen von Funktionen auflösen

Challenges

Diskutieren Sie folgende Fragen:

- Warum werden überhaupt Symboltabellen eingesetzt?
- Warum muss man zwischen Deklaration und Definition unterscheiden?
- Erklären Sie die Verbindung einer Symboltabelle zu den einzelnen Phasen einer Compiler-Pipeline.
- Wo liegen die Grenzen der semantischen Analyse?
- Warum kann man im Allgemeinen nicht die Symboltabellen nutzen, um die Werte von Symbolen mit zu speichern?
- Wieso sind Funktionen Scope und Symbol gleichzeitig?
- Skizzieren Sie für eine Funktionsdeklaration mit Parametern die resultierende Symboltabelle.

- Erklären Sie, wie man beim Funktionsaufruf vorgehen würde. Werden dabei Einträge in der Symboltabelle erzeugt?

5.4. SymbTab3: Strukturen und Klassen

TL;DR

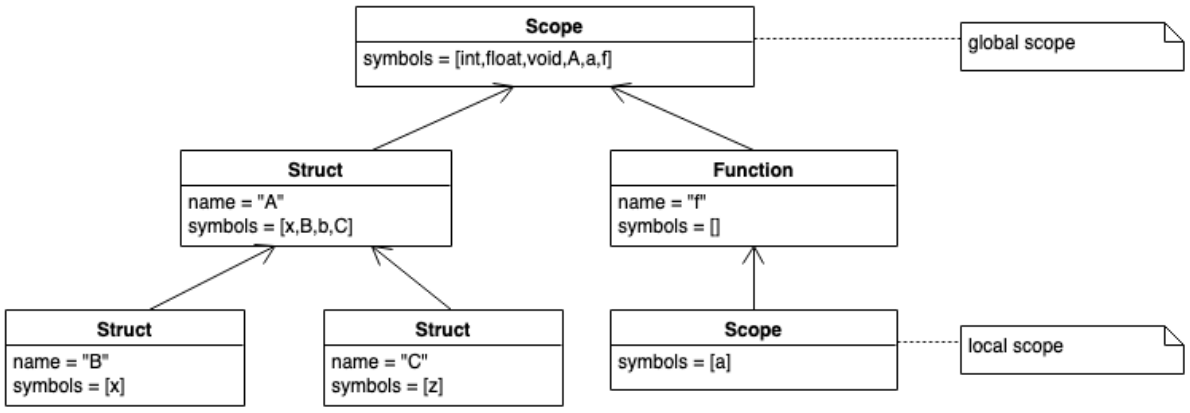
Strukturen und Klassen bilden jeweils einen eigenen verschachtelten Scope, worin die Attribute und Methoden definiert werden.
Bei der Namensauflösung muss man dies beachten und darf beim Zugriff auf Attribute und Methoden nicht einfach in den übergeordneten Scope schauen. Zusätzlich müssen hier Vererbungshierarchien in der Struktur der Symboltabelle berücksichtigt werden.

Videos

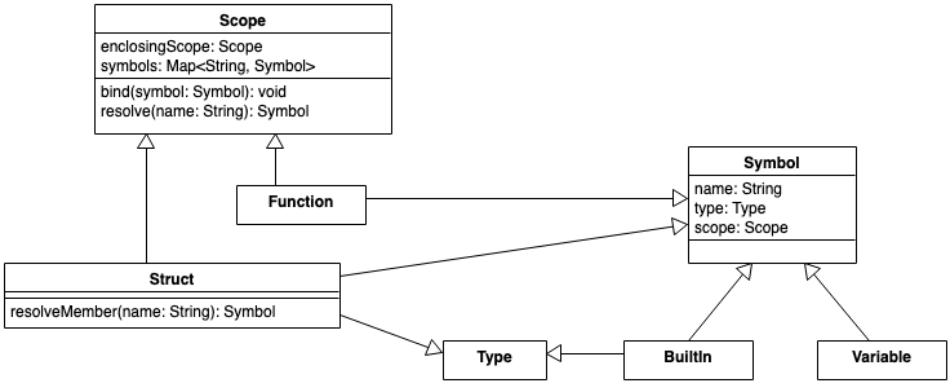
Vorlesung [YT], [HSBI]

5.4.1. Strukturen

```
1  struct A {
2      int x;
3      struct B {int x;};
4      B b;
5      struct C {int z;};
6  };
7  A a;
8  void f() {
9      A a;
10     a.b.x = 42;
11 }
```



5.4.2. Strukturen: Erweiterung der Symbole und Scopes



Quelle: Structs: Eigene Modellierung nach einer Idee in (Parr 2010, p. 162)

Strukturen stellen wie Funktionen sowohl einen Scope als auch ein Symbol dar.

Zusätzlich stellt eine Struktur (-definition) aber auch einen neuen Typ dar, weshalb `Struct` auch noch das Interface `Type` "implementiert".

5.4.3. Strukturen: Auflösen von Namen

```
class Struct(Scope, Symbol, Type):
    def resolveMember(name):
        return symbols[name]
```

=> Auflösen von "`a.b`"(im Listener in `exitMember()`):

- `a` im "normalen" Modus mit `resolve()` über den aktuellen Scope
- Typ von `a` ist `Struct` mit Verweis auf den eigenen Scope
- `b` nur innerhalb des `Struct`-Scopes mit `resolveMember()`

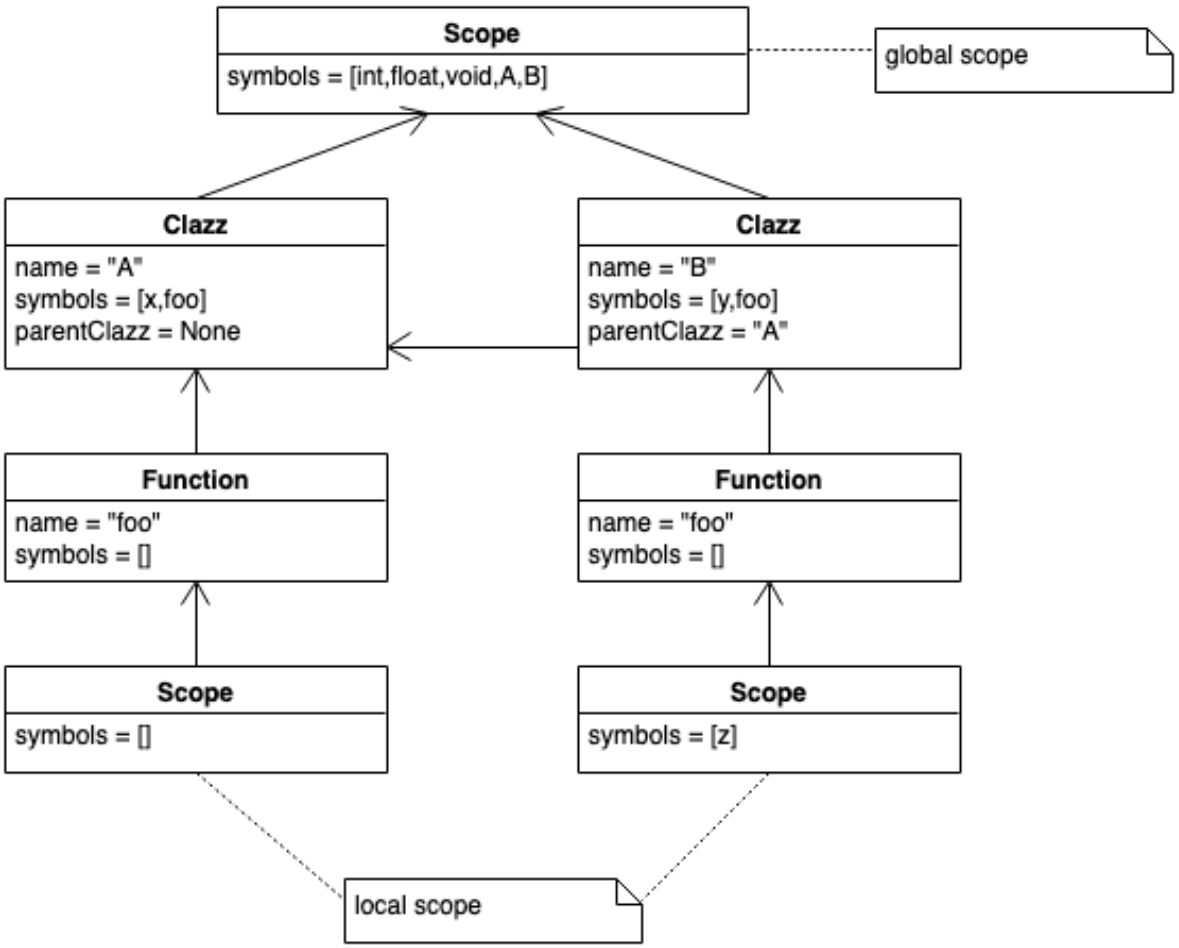
In der Grammatik würde es eine Regel `member` geben, die auf eine Struktur der Art `ID.ID` anspricht (d.h. eigentlich den Teil `.ID`), und entsprechend zu Methoden `enterMember()` und `exitMember()` im Listener führt.

Das Symbol für `a` hat als `type`-Attribut eine Referenz auf die `Struct`, die ja einen eigenen Scope hat (`symbols` -Map). Darin muss dann `b` aufgelöst werden.

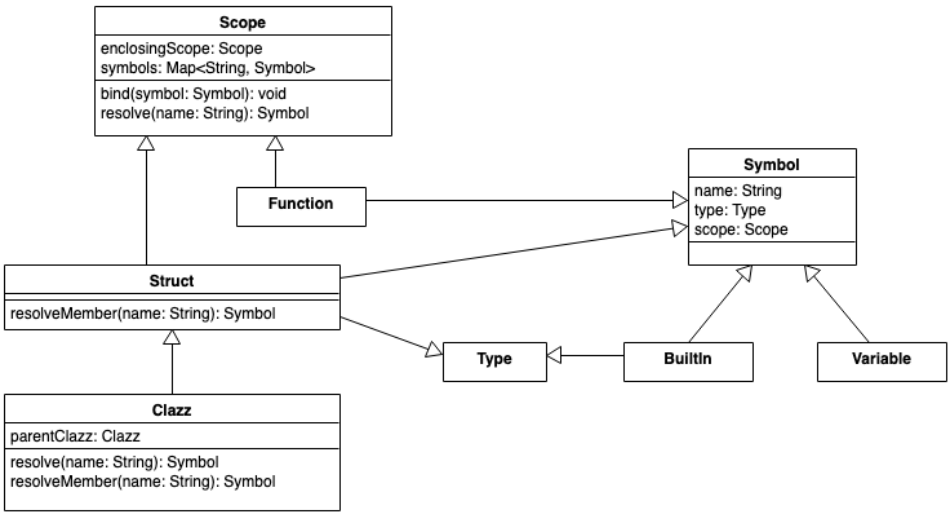
5.4.4. Klassen

```
class A {
public:
    int x;
    void foo() { ; }
```

```
};
class B : public A {
public
    int y;
    void foo() {
        int z = x+y;
    }
};
```



5.4.5. Klassen: Erweiterung der Symbole und Scopes



Quelle: Klassen: Eigene Modellierung nach einer Idee in (Parr 2010, p. 167)

Bei Klassen kommt in den Tabellen ein weiterer Pointer `parentClazz` auf die Elternklasse hinzu (in der Superklasse ist der Wert `None`).

5.4.6. Klassen: Auflösen von Namen

```
1  class Clazz(Struct):
2      Clazz parentClazz   # None if base class
3
4      def resolve(name):
5          # do we know "name" here?
6          if symbols[name]: return symbols[name]
7          # NEW: if not here, check any parent class ...
8          if parentClazz and parentClazz.resolve(name): return parentClazz.
               resolve(name)
9          else:
10             # ... or enclosing scope if base class
11             if enclosingScope: return enclosingScope.resolve(name)
12             else: return None     # not found
13
14     def resolveMember(name):
15         if symbols[name]: return symbols[name]
16         # NEW: check parent class
17         if parentClazz: return parentClazz.resolveMember(name)
18         else: return None
```

Quelle: Klassen: Eigene Implementierung nach einer Idee in (Parr 2010, p. 172)

Hinweis: Die obige Implementierungsskizze soll vor allem das Prinzip demonstrieren - sie ist aus Gründen der Lesbarkeit nicht besonders effizient: beispielsweise wird `parentClazz.resolve(name)` mehrfach evaluiert …

Beim Auflösen von Attributen oder Methoden muss zunächst in der Klasse selbst gesucht werden, anschließend in der Elternklasse.

Beispiel (mit den obigen Klassen `A` und `B`):

```
B foo;
foo.x = 42;
```

Hier wird analog zu den Structs zuerst `foo` mit `resolve()` im lokalen Scope aufgelöst. Der Typ des Symbols `foo` ist ein `Clazz`, was zugleich ein Scope ist. In diesem Scope wird nun mit `resolveMember()` nach dem Symbol `x` gesucht. Falls es hier nicht gefunden werden kann, wird in der Elternklasse (sofern vorhanden) weiter mit`resolveMember()` gesucht.

Die normale Namensauflösung wird ebenfalls erweitert um die Auflösung in der Elternklasse.

Beispiel:

```
int wuppie;
class A {
public:
    int x;
    void foo() { ; }
};
class B : public A {
public
    int y;
    void foo() {
        int z = x+y+wuppie;
    }
};
```

Hier würde `wuppie` als Symbol im globalen Scope definiert werden. Beim Verarbeiten von `int z = x+y +wuppie;` würde mit `resolve()` nach `wuppie` gesucht: Zuerst im lokalen Scope unterhalb der Funktion, dann im Funktions-Scope, dann im Klassen-Scope von `B`. Hier sucht `resolve()` auch zunächst lokal, geht dann aber die Vererbungshierarchie entlang (sofern wie hier vorhanden). Erst in der Superklasse (wenn der `parentClazz`-Zeiger `None` ist), löst `resolve()` wieder normal auf und sucht um umgebenden Scope. Auf diese Weise kann man wie gezeigt in Klassen (Methoden) auf globale Variablen verweisen …

Anmerkung: Durch dieses Vorgehen wird im Prinzip in Methoden aus dem Zugriff auf ein Feld `x` implizit ein `this.x` aufgelöst, wobei `this` die Klasse auflöst und `x` als Attribut darin.

5.4.7. Wrap-Up

- Symboltabellen: Verwaltung von Symbolen und Typen (Informationen über Bezeichner)

- Strukturen und Klassen bilden eigenen Scope
- Strukturen/Klassen lösen etwas anders auf: Zugriff auf Attribute und Methoden

Zum Nachlesen

- Mogensen (2017): Kapitel 3
- Parr (2014): Kapitel 6.4 und 8.4
- Parr (2010): Kapitel 6, 7 und 8

Lernziele

- k3: Ich kann Symboltabellen für Nested Scopes implementieren unter Nutzung von passenden Strukturen/Klassen und einem Listener
- k3: Ich kann Attribute von Klassen und Strukturen auflösen

Challenges

Symboltabellen praktisch

Betrachten Sie folgenden Java-Code:

1. Umkreisen Sie alle Symbole.
2. Zeichen Sie Pfeile von Symbol-Referenzen zur jeweiligen Definition (falls vorhanden).
3. Identifizieren Sie alle benannten Scopes.
4. Identifizieren Sie alle anonymen Scopes.
5. Geben Sie die resultierende Symboltabelle an (Strukturen wie in VL besprochen).

```
1  package a.b;
2
3  import u.Y;
4
5  class X extends Y {
6      int f(int x) {
7          int x,y;
8          { int x; x - y + 1; }
9          x = y + 1;
10     }
11 }
12
13 class Z {
14     class W extends X {
15         int x;
16         void foo() { f(34); }
17     }
18     int x,z;
19     int f(int x) {
20         int y;
21         y = x;
22         z = x;
23     }
24 }
```

5.5. Typen, Type Checking und Attributierte Grammatiken

Weitere Unterlagen

- Annotierte Folien: Typen, Type Checking und attributierte Grammatiken

5.5.1. Motivation

5.5.1.1. Ist das alles erlaubt?

Operation erlaubt?

Zuweisung erlaubt?

Welcher Ausdruck hat welchen Typ?

(Welcher Code muss dafür erzeugt werden?)

- a = b
- a = f(b)
- a = b + c
- a = b + o.nummer
- if (f(a) == f(b))

5.5.1.2. Taschenrechner: Parsen von Ausdrücken wie 3*5+4

```
expr : expr '+' term
     | term
     ;
term : term '*' DIGIT
     | DIGIT
     ;

DIGIT : [0-9] ;
```

=> Wie den Ausdruck **ausrechnen**?

Anmerkung: Heute geht es um die einfachste Form der semantischen Analyse: Anreichern einer Grammatik um Attribute und Aktionen, die während des Parsens oder bei der Traversierung des Parse-Trees ausgewertet werden.

5.5.2. Semantische Analyse

5.5.2.1. Das haben wir bis jetzt

Wir haben den AST vorliegen.

Idealerweise enthält er bei jedem Bezeichner einen Verweis in sogenannte Symboltabellen (siehe spätere Veranstaltung).

5.5.2.2. Was kann beim Parsen schon überprüft / bestimmt werden?

Hier entsteht ein Tafelbild.

5.5.2.3. Was fehlt jetzt noch?

Kontextsensitive Analysen

Hier entsteht ein Tafelbild.

5.5.3. Analyse von Datentypen

5.5.3.1. Typisierung

- stark oder statisch typisierte Sprachen: Alle oder fast alle Typüberprüfungen finden in der semantischen Analyse statt (C, C++, Java)
- schwach oder dynamisch typisierte Sprachen: Alle oder fast alle Typüberprüfungen finden zur Laufzeit statt (Python, Lisp, Perl)
- untypisierte Sprachen: keinerlei Typüberprüfungen (Maschinensprache)

5.5.3.2. Ausdrücke

Jetzt muss für jeden Ausdruck im weitesten Sinne sein Typ bestimmt werden.

Ausdrücke können hier sein:

- rechte Seiten von Zuweisungen
- linke Seiten von Zuweisungen
- Funktions- und Methodenaufrufe
- jeder einzelne aktuelle Parameter in Funktions- und Methodenaufrufen
- Bedingungen in Kontrollstrukturen

5.5.3.3. Typinferenz

Def.: *Typinferenz* ist die Bestimmung des Datentyps jedes Bezeichners und jedes Ausdrucks im Code.

- Die Typen von Unterausdrücken bestimmen den Typ eines Ausdrucks
- Kalkül mit sog. Inferenzregeln der Form

$$\frac{f : s \to t \quad x : s}{f(x) : t}$$

(Wenn f den Typ $s \to t$ hat und x den Typ s, dann hat der Ausdruck f(x) den Typ t.)

- z. B. zur Auflösung von Überladung und Polymorphie zur Laufzeit

5.5.3.4. Statische Typprüfungen

Bsp.: Der + - Operator:

Typ 1. Operand	Typ 2. Operand	Ergebnistyp
int	int	int
float	float	float
int	float	float
float	int	float
string	string	string

5.5.3.5. Typkonvertierungen

- Der Compiler kann implizite Typkonvertierungen vornehmen, um einen Ausdruck zu verifizieren (siehe Sprachdefiniton)
- Typerweiterungen, z.B. von *int* nach *float* oder
- Bestimmung des kleinsten umschließenden Typ vorliegender Typen
- *Type Casts*: explizite Typkonvertiereungen

5.5.3.6. Nicht grundsätzlich statisch mögliche Typprüfungen

Bsp.: Der ^-Operator (a^b):

Typ 1. Operand	Typ 2. Operand	Ergebnistyp
int	int ≥ 0	int
int	int < 0	float
int	float	float
...	...	...

5.5.4. Attributierte Grammatiken

5.5.4.1. Was man damit macht

Die Syntaxanalyse kann keine kontextsensitiven Analysen durchführen

- Kontextsensitive Grammatiken benutzen: Laufzeitprobleme, das Parsen von cs-Grammatiken ist *PSPACE-complete*
- Parsergenerator *Bison*: generiert LALR(1)-Parser, aber auch sog. *Generalized LR (GLR) Parser*, die bei nicht-lösbaren Konflikten in der Grammatik (Reduce/Reduce oder Shift/Reduce) parallel den Input mit jede der Möglichkeiten weiterparsen
- Anderer Ansatz: Berücksichtigung kontextsensitiver Abhängigkeiten mit Hilfe attributierter Grammatiken, zur Typanalyse, auch zur Codegenerierung
- Weitergabe von Informationen im Baum

5.5.5. Syntax-gesteuerte Übersetzung: Attribute und Aktionen

5.5.5.1. Berechnen der Ausdrücke

```
1 expr : expr '+' term ;
```

```
1 translate expr ;
2 translate term ;
3 handle + ;
```

5.5.5.2. Attributierte Grammatiken (SDD)

auch "*syntax-directed definition*"

Anreichern einer CFG:

- Zuordnung einer Menge von Attributen zu den Symbolen (Terminal- und Nicht-Terminal-Symbole)
- Zuordnung einer Menge von *semantischen Regeln* (Evaluationsregeln) zu den Produktionen

5.5.5.3. Definition: Attributierte Grammatik

Eine *attributierte Grammatik AG = (G,A,R)* besteht aus folgenden Komponenten:

- Mengen A(X) der Attribute eines Nonterminals X
- *G = (N, T, P, S)* ist eine cf-Grammatik
- A = $\bigcup_{X \in (T \cup N)} A(X)$ mit $A(X) \cap A(Y) \neq \emptyset \Rightarrow X = Y$
- R = $\bigcup_{p \in P} R(p)$ mit $R(p) = \{X_i.a = f(\ldots) | p : X_0 \rightarrow X_1 \ldots X_n \in P, X_i.a \in A(X_i), 0 \leq i \leq n\}$

5.5.5.4. Abgeleitete und ererbte Attribute

Die in einer Produktion p definierten Attribute sind

AF(p) = $\{X_i.a \mid p : X_0 \to X_1 \dots X_n \in P, 0 \leq i \leq n, X_i.a = f(\dots) \in R(p)\}$

Disjunkte Teilmengen der Attribute: abgeleitete (synthesized) Attributen *AS(X)* und ererbte (inherited) Attributen *AI(X)*:

- *AS(X)* = $\{X.a \mid \exists p : X \to X_1 \dots X_n \in P, X.a \in AF(p)\}$
- *AI(X)* = $\{X.a \mid \exists q : Y \to uXv \in P, X.a \in AF(q)\}$

Abgeleitete Attribute geben Informationen von unten nach oben weiter, geerbte von oben nach unten.

Abhängigkeitsgraphen stellen die Abhängigkeiten der Attribute dar.

5.5.5.5. Beispiel: Attributgrammatiken

Produktion	Semantische Regel
`e : e1 '+'t ;`	`e.val = e1.val + t.val`
`e : t ;`	`e.val = t.val`
`t : t1 '*'D ;`	`t.val = t1.val * D.lexval`
`t : D ;`	`t.val = D.lexval`

Produktion	Semantische Regel
`t : D t';`	`t'.inh = D.lexval`
	`t.syn = t'.syn`
`t': '*'D t'1 ;`	`t'1.inh = t'.inh * D.lexval`
	`t'.syn = t'1.syn`
`t' :` ϵ `;`	`t'.syn = t'.inh`

Wenn ein Nichtterminal mehr als einmal in einer Produktion vorkommt, werden die Vorkommen nummeriert. (t, t1; t', t'1)

5.5.6. S-Attributgrammatiken und L-Attributgrammatiken

5.5.6.1. S-Attributgrammatiken

S-Attributgrammatiken: Grammatiken mit nur abgeleiteten Attributen, lassen sich während des Parsens mit LR-Parsern beim Reduzieren berechnen (Tiefensuche mit Postorder-Evaluation):

```
def visit(N):
    for each child C of N (from left to right):
        visit(C)
    eval(N)     # evaluate attributes of N
```

5.5.6.2. L-Attributgrammatiken

- Grammatiken, deren geerbte Atribute nur von einem Elternknoten oder einem linken Geschwisterknoten abhängig sind
- können während des Parsens mit LL-Parsern berechnet werden
- alle Kanten im Abhängigkeitsgraphen gehen nur von links nach rechts
- ein Links-Nach-Rechts-Durchlauf ist ausreichend
- S-attributierte SDD sind eine Teilmenge von L-attributierten SDD

5.5.6.3. Beispiel: S-Attributgrammatik

Produktion	Semantische Regel
`e : e1 '+'t ;`	`e.val = e1.val + t.val`
`e : t ;`	`e.val = t.val`
`t : t1 '*'D ;`	`t.val = t1.val * D.lexval`
`t : D ;`	`t.val = D.lexval`

5.5.6.4. Beispiel: Annotierter Syntaxbaum für 5*8+2

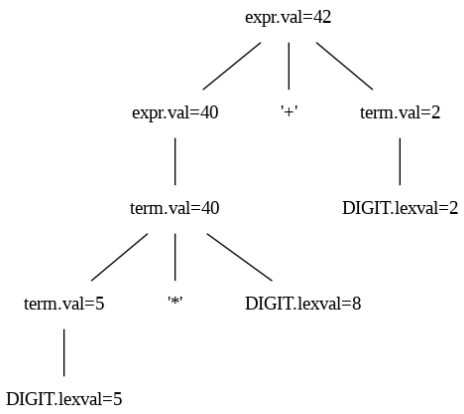


Abbildung 5.1.: Annotierter Parse-Tree

5.5.6.5. Erzeugung des AST aus dem Parse-Tree für 5*8+2

Produktion	Semantische Regel
`e : e1 '+'t ;`	`e.node = new Node('+', e1.node, t.node)`
`e : t ;`	`e.node = t.node`
`t : t1 '*'D ;`	`t.node = new Node('*', t1.node, new Leaf(D, D.lexval));`
`t : D ;`	`t.node = new Leaf(D, D.lexval);`

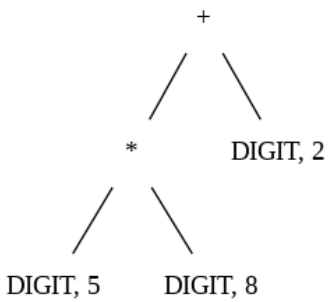


Abbildung 5.2.: AST

5.5.6.6. Beispiel: L-Attributgrammatik, berechnete u. geerbte Attribute, ohne Links-Rekursion

Teil der vorigen SDD zum Parsen und Berechnen von Ausdrücken wie 5*8+2, hier umformuliert ohne Links-Rekursion und mit berechneten und geerbten Attributen:

Produktion	Semantische Regel
`t : D t';`	`t'.inh = D.lexval`
	`t.syn = t'.syn`
`t': '*'D t'1 ;`	`t'1.inh = t'.inh * D.lexval`
	`t'.syn = t'1.syn`
`t' :` ϵ `;`	`t'.syn = t'.inh`

5*8 =>

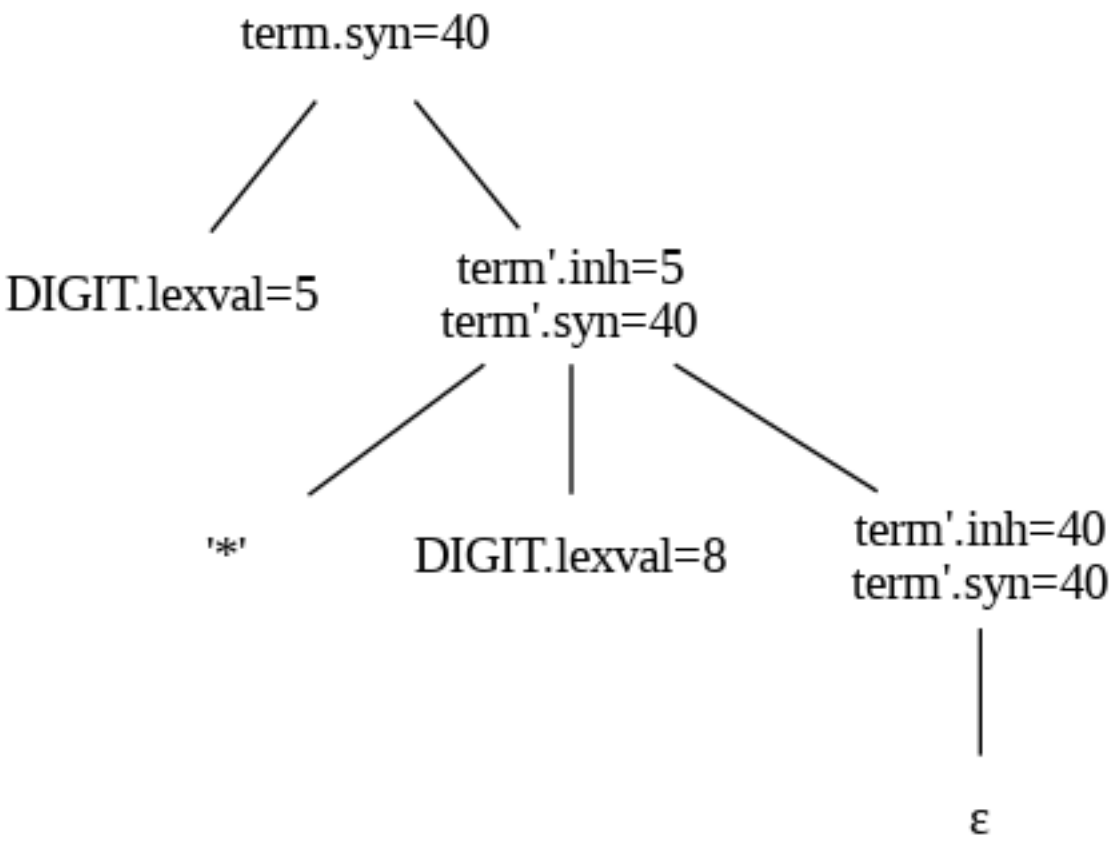


Abbildung 5.3.: Annotierter Parse-Tree mit berechneten und geerbten Attributen (nur Multiplikation)

Vorgriff: Dies ist ein Beispiel für eine “L-attributierte SDD”.

5.5.6.7. Beispiel: Typinferenz für 3+7+9 oder "hello"+"world"

Produktion	Semantische Regel
`e : e1 '+'t ;`	`e.type = f(e1.type, t.type)`
`e : t ;`	`e.type = t.type`
`t : NUM ;`	`t.type = "int"`
`t : NAME ;`	`t.type = "string"`

5.5.7. Syntax-gesteuerte Übersetzung (SDT)

5.5.7.1. Erweiterung attributierter Grammatiken

Syntax-directed translation scheme:

Zu den Attributen kommen **Semantische Aktionen**: Code-Fragmente als zusätzliche Knoten im Parse Tree an beliebigen Stellen in einer Produktion, die, wenn möglich, während des Parsens, ansonsten in weiteren Baumdurchläufen ausgeführt werden.

```
1  e : e1  {print e1.val;}
2      '+' {print "+";}
3      t   {e.val = e1.val + t.val; print(e.val);}
4    ;
```

5.5.7.2. S-attributierte SDD, LR-Grammatik: Bottom-Up-Parsierbar

Die Aktionen werden am Ende jeder Produktion eingefügt (“postfix SDT”).

Produktion	Semantische Regel
`e : e1 '+'t ;`	`e.val = e1.val + t.val`
`e : t ;`	`e.val = t.val`
`t : t1 '*'D ;`	`t.val = t1.val * D.lexval`
`t : D ;`	`t.val = D.lexval`

```
1  e : e1 '+' t  {e.val = e1.val + t.val; print(e.val);} ;
2  e : t         {e.val = t.val;} ;
3  t : t1 '*' D  {t.val = t1.val * D.lexval;} ;
4  t : D         {t.val = D.lexval;} ;
```

5.5.7.3. L-attributierte SDD, LL-Grammatik: top-down-parsebar (1/2)

Produktion	Semantische Regel
`t : D t';`	`t'.inh = D.lexval`
	`t.syn = t'.syn`
`t': '*'D t'1 ;`	`t'1.inh = t'.inh * D.lexval`
	`t'.syn = t'1.syn`
`t' :` ϵ `;`	`t'.syn = t'.inh`

```
t  : D {t'.inh = D.lexval;} t' {t.syn = t'.syn;} ;
t' : '*' D {t'1.inh = t'.inh * D.lexval;} t'1 {t'.syn = t'1.syn;} ;
t' : e {t'.syn = t'.inh;} ;
```

5.5.7.4. L-attributierte SDD, LL-Grammatik: Top-Down-Parsierbar (2/2)

- LL-Grammatik: Jede L-attributierte SDD direkt während des Top-Down-Parsens implementierbar/berechenbar
- SDT dazu:
  - Aktionen, die ein berechnetes Attribut des Kopfes einer Produktion berechnen, an das Ende der Produktion anfügen
  - Aktionen, die geerbte Attribute für ein Nicht-Terminalsymbol A berechnen, direkt vor dem Auftreten von A im Körper der Produktion eingefügen

5.5.8. Implementierung im rekursiven Abstieg

5.5.8.1. Implementierung im rekursiven Abstieg

- Geerbte Attribute sind Parameter für die Funktionen für die Nicht-Terminalsymbole
- berechnete Attribute sind Rückgabewerte dieser Funktionen.

```
T t'(T inh) {
    match('*');
    T t1inh = inh * match(D);
    return t'(t1inh);
}
```

5.5.9. Wrap-Up

5.5.9.1. Wrap-Up

- Die Typinferenz benötigt Informationen aus der Symboltabelle
- Einfache semantische Analyse: Attribute und semantische Regeln (SDD)
- Umsetzung mit SDT: Attribute und eingebettete Aktionen
- Reihenfolge der Auswertung u.U. schwierig

  Bestimmte SDT-Klassen können direkt beim Parsing abgearbeitet werden:
  - S-attributierte SDD, LR-Grammatik: bottom-up-parsebar
  - L-attributierte SDD, LL-Grammatik: top-down-parsebar

Ansonsten werden die Attribute und eingebetteten Aktionen in den Parse-Tree, bzw. AST, integriert und bei einer (späteren) Traversierung abgearbeitet.

Zum Nachlesen

- Aho u. a. (2023): Kapitel 2.3 und 5
- Levine (2009)

Lernziele

- k2: Ich kann das Konzept der attributierten Grammatiken erklären und in einem Beispiel die Anreicherung mit Attributen und semantischen Regeln demonstrieren
- k2: Ich kann den Unterschied zwischen geerbten und berechneten Attributen erklären
- k2: Ich kann die Umsetzung von SDD mit Hilfe von SDT erklären
- k3: Ich kann eine einfache semantische Analyse mit Hilfe von attributierten Grammatiken durchführen

6. Interpreter

Ein Interpreter erzeugt keinen Code, sondern führt Source-Code (interaktiv) aus. Die einfachste Möglichkeit ist der Einsatz von attributierten Grammatiken, wo der Code bereits beim Parsen ausgeführt wird ("syntaxgesteuerte Interpretation"). Mehr Möglichkeiten hat man dagegen bei der Traversierung des AST, beispielsweise mit dem Visitor-Pattern. Auch die Abarbeitung von Bytecode in einer Virtuellen Maschine (VM) zählt zur Interpretation.

(Register- und Stack-basierte Interpreter betrachten wir im Rahmen der Veranstaltung aktuell nicht.)

6.1. AST-basierte Interpreter: Basics

TL;DR

Ein AST-basierter Interpreter besteht oft aus einem "Visitor-Dispatcher": Man traversiert mit einer `eval()`-Funktion den AST und ruft je nach Knotentyp die passende Funktion auf. Dabei werden bei Ausdrücken (*Expressions*) Werte berechnet und zurückgegeben, d.h. hier hat man einen Rückgabewert und ein entsprechendes `return` im `switch`/`case`, während man bei Anweisungen (*Statements*) keinen Rückgabewert hat.
Der Wert von Literalen ergibt sich direkt durch die Übersetzung des jeweiligen Werts in den passenden Typ der Implementierungssprache. Bei einfachen Ausdrücken kann man auf das in Syntaxgesteuerte Interpreter demonstrierte Vorgehen zurückgreifen: Man interpretiert zunächst die Teilausdrücke durch den Aufruf von `eval()` für die jeweiligen AST-Kindknoten und berechnet daraus das gewünschte Ergebnis.
Für Blöcke und Variablen muss man analog zum Aufbau von Symboltabellen wieder Scopes berücksichtigen, d.h. man benötigt Strukturen ähnlich zu den Symboltabellen (hier "Umgebung" (*Environment*) genannt). Es gibt eine globale Umgebung, und mit dem Betreten eines neuen Blocks wird eine neue Umgebung aufgemacht, deren Eltern-Umgebung die bisherige Umgebung ist.
Zu jedem Namen kann man in einer Umgebung einen Wert definieren bzw. abrufen. Dabei muss man je nach Semantik der zu interpretierenden Sprache unterscheiden zwischen der "Definition" und der "Zuweisung" einer Variablen: Die Definition erfolgt i.d.R. in der aktuellen Umgebung, bei der Zuweisung sucht man ausgehend von der aktuellen Umgebung bis hoch zur globalen Umgebung nach dem ersten Vorkommen der Variablen und setzt den Wert in der gefundenen Umgebung. Bei Sprachen, die Variablen beim ersten Zugriff definieren, muss man dieses Verhalten entsprechend anpassen.

Videos

Vorlesung [YT], [HSBI]

6.1.1. Aufgaben im Interpreter

Im Allgemeinen reichen einfache syntaxgesteuerte Interpreter nicht aus. Normalerweise simuliert ein Interpreter die Ausführung eines Programms durch den Computer. D.h. der Interpreter muss über die entsprechenden Eigenschaften verfügen: Prozessor, Code-Speicher, Datenspeicher, Stack …

```
int x = 42;
int f(int x) {
    int y = 9;
    return y+x;
}

x = f(x);
```

- Aufbauen des AST … => Lexer+Parser
- Auflösen von Symbolen/Namen … => Symboltabellen, Resolving
- Type-Checking und -Inference … => Semantische Analyse (auf Symboltabellen)

- Speichern von Daten: Name+Wert vs. Adresse+Wert (Erinnerung: Data-Segment und Stack im virtuellen Speicher)
- Ausführen von Anweisungen Text-Segment im virtuellen Speicher; hier über den AST
- Aufruf von Funktionen und Methoden Kontextwechsel nötig: Was ist von wo aus sichtbar?

6.1.2. AST-basierte Interpreter: Visitor-Dispatcher

```
def eval(self, AST t):
    if   t.type == Parser.BLOCK  : block(t)
    elif t.type == Parser.ASSIGN : assign(t)
    elif t.type == Parser.RETURN : ret(t)
    elif t.type == Parser.IF     : ifstat(t)
    elif t.type == Parser.CALL   : return call(t)
    elif t.type == Parser.ADD    : return add(t)
    elif t.type == Parser.MUL    : return mul(t)
    elif t.type == Parser.INT    : return Integer.parseInt(t.getText())
    elif t.type == Parser.ID     : return load(t)
    else : ...  # catch unhandled node types
    return None;
```

Nach dem Aufbau des AST durch Scanner und Parser und der semantischen Analyse anhand der Symboltabellen müssen die Ausdrücke (*expressions*) und Anweisungen (*statements*) durch den Interpreter ausgewertet werden. Eine Möglichkeit dazu ist das Traversieren des AST mit dem Visitor-Pattern. Basierend auf dem Typ des aktuell betrachteten AST-Knotens wird entschieden, wie damit umgegangen werden soll. Dies erinnert an den Aufbau der Symboltabellen …

Tip

Exkurs Expressions (Ausdrücke) vs. Statements (Anweisungen)
In Programmiersprachen unterscheiden wir häufig **Expressions** (*Ausdrücke*) und **Statements** (*Anweisungen*).
Expressions sind dabei syntaktische Konstrukte einer Programmiersprache, die (in einem gegebenen Kontext) zu einem Wert **evaluiert** werden können. Typische Expressions sind beispielsweise Ausdrücke wie `2*3` oder `foo(42);`… In manchen Sprachen sind beispielsweise auch Zuweisungen Expressions: `v = 42 + 7;` würde in C der Variablen `v` den Wert 49 zuweisen, dies ist gleichzeitig auch der Wert des gesamten Ausdrucks. Man könnte in C also Dinge formulieren wie `if (v = 42 + 7)...` (wobei das Interpretieren eines Integers in einem bool'schen Kontext nochmal ein anderes Problem ist).
Statements sind syntaktische Konstrukte in Programmiersprachen, die **ausgeführt** werden können und dabei in der Regel einen Zustand im Programm verändern, also einen Seiteneffekt haben. Die Ausführung eines Statements hat normalerweise keinen Wert an sich. Typische Beispiele sind Zuweisungen `v = 7`, Kontrollfluss `if (...)then {...} else {...}`, Schleifen `for x in foo: ...`, `switch`/`case`-Statements. (Es gibt aber auch Programmiersprachen, wo ein `if`/`then`/`else`-Konstrukt eine Expression ist, also bei der Ausführung einen Wert ergibt.) In den meisten Programmiersprachen können Expressions Teile von Statements bilden: In `v = 42 + 7` ist die gesamte Zuweisung eine Anweisung (Seiteneffekt: die Variable `v` hat danach einen anderen Zustand), und der Teil `42 + 7` ist ein Ausdruck, der ausgewertet werden kann und üblicherweise den Wert 49 ergibt (außer man beauftragt ein LLM mit der Auswertung). In C-ähnlichen Sprachen kann durch Hinzufügen eines Semikolons aus dem Ausdruck `42 +7` eine Anweisung gemacht werden…
Vergleiche auch Nystrom (2021), Kapitel 6 "Parsing Expressions", Kapitel 7 "Evaluating Expressions" und Kapitel 8 "Statements and State", aber auch Wikipedia: Expression und Wikipedia: Statement.

Die `eval()`-Methode bildet das Kernstück des (AST-traversierenden) Interpreters. Hier wird passend zum aktuellen AST-Knoten die passende Methode des Interpreters aufgerufen.

Hinweis: Im obigen Beispiel wird nicht zwischen der Auswertung von Ausdrücken und Anweisungen unterschieden, es wird die selbe Methode `eval()` genutzt. Allerdings liefern Ausdrücke einen Wert zurück (erkennbar am `return` im jeweiligen `switch`/`case`-Zweig), während Anweisungen keinen Wert liefern.

In den folgenden Beispielen wird davon ausgegangen, dass ein komplettes Programm eingelesen, geparst, vorverarbeitet und dann interpretiert wird.

Für einen interaktiven Interpreter würde man in einer Schleife die Eingaben lesen, parsen und vorverarbeiten und dann interpretieren. Dabei würde jeweils der AST und die Symboltabelle *ergänzt*, damit die neuen Eingaben auf frühere verarbeitete Eingaben zurückgreifen können. Durch die Form der Schleife "Einlesen – Verarbeiten – Auswerten" hat sich auch der Name "*Read-Eval-Loop*" bzw. "*Read-Eval-Print-Loop*" (**REPL**) eingebürgert.

6.1.3. Auswertung von Literalen und Ausdrücken

- Typen mappen: Zielsprache => Implementierungssprache

  Die in der Zielsprache verwendeten (primitiven) Typen müssen auf passende Typen der Sprache, in der der Interpreter selbst implementiert ist, abgebildet werden.

Beispielsweise könnte man den Typ `nil` der Zielsprache auf den Typ **`null`** des in Java implementierten Interpreters abbilden, oder den Typ `number` der Zielsprache auf den Typ `Double` in Java mappen.

- Literale auswerten:

```
INT: [0-9]+ ;
```

```
elif t.type == Parser.INT : return Integer.parseInt(t.getText())
```

Das ist der einfachste Teil … Die primitiven Typen der Zielsprache, für die es meist ein eigenes Token gibt, müssen als Datentyp der Interpreter-Programmiersprache ausgewertet werden.

- Ausdrücke auswerten:

```
add: e1=expr "+" e2=expr ;
```

```
def add(self, AST t):
    lhs = eval(t.e1())
    rhs = eval(t.e2())
    return (double)lhs + (double)rhs  # Semantik!
```

Die meisten möglichen Fehlerzustände sind bereits durch den Parser und bei der semantischen Analyse abgefangen worden. Falls zur Laufzeit die Auswertung der beiden Summanden keine Zahl ergibt, würde eine Java-Exception geworfen, die man an geeigneter Stelle fangen und behandeln muss. Der Interpreter soll sich ja nicht mit einem Stack-Trace verabschieden, sondern soll eine Fehlermeldung präsentieren und danach normal weiter machen …

6.1.4. Kontrollstrukturen

```
ifstat: 'if' expr 'then' s1=stat ('else' s2=stat)? ;
```

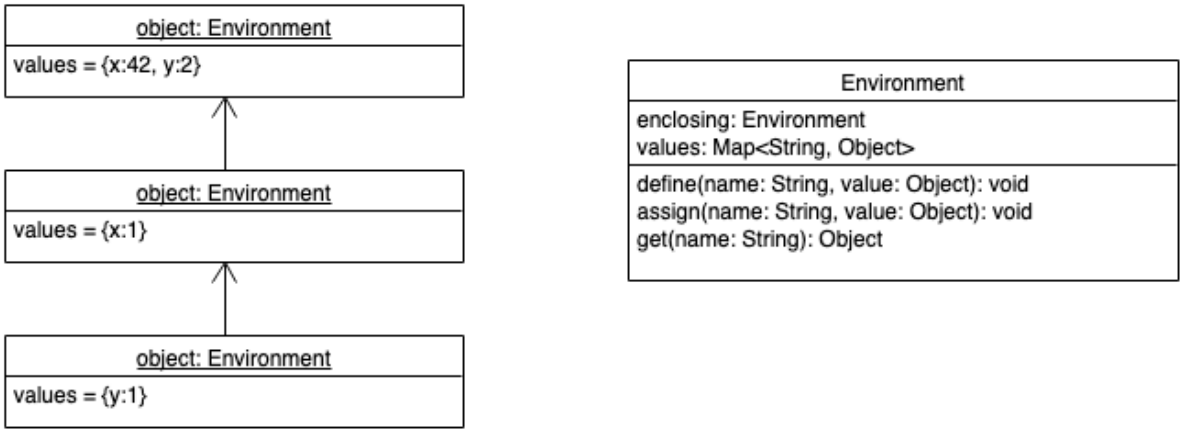
```
def ifstat(self, AST t):
    if eval(t.expr()): eval(t.s1())
    else:
        if t.s2(): eval(t.s2())
```

Analog können die anderen bekannten Kontrollstrukturen umgesetzt werden, etwa **`switch`**/**`case`**, **`while`** oder **`for`**.

Dabei können erste Optimierungen vorgenommen werden: Beispielsweise könnten **`for`**-Schleifen im Interpreter in **`while`**-Schleifen transformiert werden, wodurch im Interpreter nur ein Schleifenkonstrukt implementiert werden müsste.

6.1.5. Zustände: Auswerten von Anweisungen

```
int x = 42;
float y;
{
    int x;
    x = 1;
    y = 2;
    { int y = x; }
}
```



Das erinnert nicht nur zufällig an den Aufbau der Symboltabellen :-)

Und so lange es nur um Variablen ginge, könnte man die Symboltabellen für das Speichern der Werte nutzen. Allerdings müssen wir noch Funktionen und Strukturen bzw. Klassen realisieren, und spätestens dann kann man die Symboltabelle nicht mehr zum Speichern von Werten einsetzen. Also lohnt es sich, direkt neue Strukturen für das Halten von Variablen und Werten aufzubauen.

6.1.6. Detail: Felder im Interpreter

Eine mögliche Implementierung für einen Interpreter basierend auf einem ANTLR-Visitor ist nachfolgend gezeigt.

Hinweis: Bei der Ableitung des `BaseVisitor<T>` muss der Typ `T` festgelegt werden. Dieser fungiert als Rückgabetyp für die Visitor-Methoden. Entsprechend können alle Methoden nur einen gemeinsamen (Ober-) Typ zurückliefern, weshalb man sich an der Stelle oft mit `Object` behilft und dann manuell den konkreten Typ abfragen und korrekt casten muss.

```
class Interpreter(BaseVisitor<Object>):
    __init__(self, AST t):
        BaseVisitor<Object>.__init__(self)
        self.root = t
        self.env = Environment()
```

Quelle: AST-Interpreter: Eigener Code basierend auf einer Idee nach Interpreter.java by Bob Nystrom on Github.com (MIT)

6.1.7. Ausführen einer Variablendeklaration

```
varDecl: "var" ID ("=" expr)? ";" ;
```

```
def varDecl(self, AST t):
    # deklarierte Variable (String)
    name = t.ID().getText()

    value = None;  # TODO: Typ der Variablen beachten (Defaultwert)
    if t.expr(): value = eval(t.expr())

    self.env.define(name, value)

    return None
```

Wenn wir bei der Traversierung des AST mit `eval()` bei einer Variablendeklaration vorbeikommen, also etwa `int x;` oder `int x = wuppie + fluppie;`, dann wird im **aktuellen** Environment der String "x" sowie der Wert (im zweiten Fall) eingetragen.

6.1.8. Ausführen einer Zuweisung

```
assign: ID "=" expr;
```

```
def assign(self, AST t):
    lhs = t.ID().getText()
    value = eval(t.expr())

    self.env.assign(lhs, value)  # Semantik!
}

class Environment:
    def assign(self, String n, Object v):
        if self.values[n]: self.values[n] = v
        elif self.enclosing: self.enclosing.assign(n, v)
        else: raise RuntimeError(n, "undefined variable")
```

Quelle: Evaluieren einer Zuweisung: Eigener Code basierend auf einer Idee nach Environment.java by Bob Nystrom on Github.com (MIT)

Wenn wir bei der Traversierung des AST mit `eval()` bei einer Zuweisung vorbeikommen, also etwa `x = 7;` oder `x = wuppie + fluppie;`, dann wird zunächst im aktuellen Environment die rechte Seite der Zuweisung ausgewertet (Aufruf von `eval()`). Anschließend wird der Wert für die Variable im Environment eingetragen: Entweder sie wurde im aktuellen Environment früher bereits definiert, dann wird der neue Wert hier eingetragen. Ansonsten wird entlang der Verschachtelungshierarchie gesucht und entsprechend eingetragen. Falls die Variable nicht gefunden werden kann, wird eine Exception ausgelöst.

An dieser Stelle kann man über die Methode `assign` in der Klasse `Environment` dafür sorgen, dass nur bereits deklarierte Variablen zugewiesen werden dürfen. Wenn man stattdessen wie etwa in Python das implizite Erzeugen neuer Variablen erlaubten möchte, würde man statt `Environment#assign` einfach `Environment#define` nutzen …

Anmerkung: Der gezeigte Code funktioniert nur für normale Variablen, nicht für Zugriffe auf Attribute einer Struct oder Klasse!

6.1.9. Blöcke: Umgang mit verschachtelten Environments

```
block:  '{' stat* '}' ;
```

```
def block(self, AST t):
    prev = self.env

    try:
        self.env = Environment(self.env)
        for s in t.stat(): eval(s)
    finally: self.env = prev

    return None;
```

Quelle: Nested Environments: Eigener Code basierend auf einer Idee nach Interpreter.java by Bob Nystrom on Github.com (MIT)

Beim Interpretieren von Blöcken muss man einfach nur eine weitere Verschachtelungsebene für die Environments anlegen und darin dann die Anweisungen eines Blockes auswerten …

Wichtig: Egal, was beim Auswerten der Anweisungen in einem Block passiert: Es muss am Ende die ursprüngliche Umgebung wieder hergestellt werden (`finally`-Block).

6.1.10. Wrap-Up

- Interpreter simulieren die Programmausführung
  - Namen und Symbole auflösen
  - Speicherbereiche simulieren
  - Code ausführen: Read-Eval-Loop

- Traversierung des AST: `eval(AST t)` als Visitor-Dispatcher
- Scopes mit `Environment` (analog zu Symboltabellen)
- Interpretation von Blöcken und Variablen (Deklaration, Zuweisung)

Zum Nachlesen

- Nystrom (2021): Kapitel: A Tree-Walk Interpreter, insb. 8. Statements and State
- Grune u. a. (2012): Kapitel 6
- Mogensen (2017): Kapitel 4

Lernziele

- k3: Ich kann die Traversierung von Parse-Trees implementieren und dabei mit Hilfe des Visitor-Patterns Aktionen ausführen

- k3: Ich kann Environment-Strukturen analog zu den Symboltabellen aufbauen, um Namen und Werte dynamisch zu speichern
- k3: Ich kann eine Read-Eval-Schleife implementieren und dabei durch Traversierung des AST die dort abgelegten Anweisungen und Ausdrücke und Kontrollstrukturen ausführen

6.2. AST-basierte Interpreter: Funktionen und Klassen

TL;DR

Üblicherweise können Funktionen auf die Umgebung zurückgreifen, in der die Definition der Funktion erfolgt ist ("**Closure**"). Deshalb wird beim Interpretieren einer Funktionsdefinition der jeweilige AST-Knoten (mit dem Block des Funktionskörpers) und die aktuelle Umgebung in einer Struktur zusammengefasst. Zusätzlich muss in der aktuellen Umgebung der Name der Funktion zusammen mit der eben erzeugten Struktur ("Funktionsobjekt") als Wert definiert werden.
Beim Funktionsaufruf löst man den Funktionsnamen in der aktuellen Umgebung auf und erhält das Funktionsobjekt mit dem AST der Funktion und der Closure. Die Funktionsparameter werden ebenfalls in der aktuellen Umgebung aufgelöst (Aufruf von `eval()` für die AST-Kindknoten des Funktionsaufrufs). Zur Interpretation der Funktion legt man sich eine neue Umgebung an, deren Eltern-Umgebung die Closure der Funktion ist, definiert die Funktionsparameter (Name und eben ermittelter Wert) in dieser neuen Umgebung und interpretiert dann den AST-Kindknoten des Funktionsblocks in dieser neuen Umgebung.
Für den Rückgabewert muss man ein wenig tricksen: Ein Block hat normalerweise keinen Wert. Eine Möglichkeit wäre, bei der Interpretation eines **`return`**-Statements eine Exception mit dem Wert des Ausdruck hinter dem "**`return`**" zu werfen und im `eval()` des Funktionsblock zu fangen.
Für Klassen kann man analog verfahren. Methoden sind zunächst einfach Funktionen, die in einem Klassenobjekt gesammelt werden. Das Erzeugen einer Instanz einer Klasse ist die Interpretation eines "Aufrufs" der Klasse (analog zum Aufruf einer Funktion): Dabei wird ein spezielles Instanzobjekt erzeugt, welches auf die Klasse verweist und welches die Werte der Attribute hält. Beim Aufruf von Methoden auf einem Instanzobjekt wird der Name der Funktion über das Klassenobjekt aufgelöst, eine neue Umgebung erzeugt mit der Closure der Funktion als Eltern-Umgebung und das Instanzobjekt wird in dieser Umgebung definiert als "**`this`**" oder "`self`". Anschließend wird ein neues Funktionsobjekt mit der eben erzeugten Umgebung und dem Funktions-AST erzeugt und zurückgeliefert. Dieses neue Funktionsobjekt wird dann wie eine normale Funktion aufgerufen (interpretiert, s.o.). Der Zugriff in der Methode auf die Attribute der Klasse erfolgt dann über **`this`** bzw. `self`, welche in der Closure der Funktion nun definiert sind und auf das Instanzobjekt mit den Attributen verweisen.

Videos

Vorlesung [YT], [HSBI]

6.2.1. Funktionen

```
int foo(int a, int b, int c) {
    print a + b + c;
}

foo(1, 2, 3);
```

```
def makeCounter():
    var i = 0
    def count():
        i = i + 1
        print i
    return count;

counter = makeCounter()
counter()   # "1"
counter()   # "2"
```

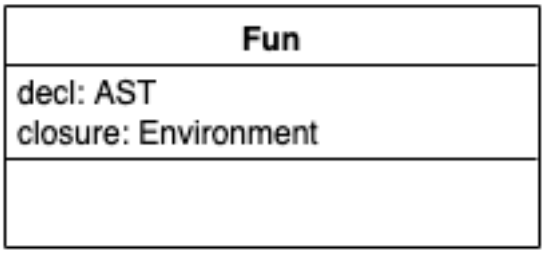
Die Funktionsdeklaration muss im aktuellen Kontext abgelegt werden, dazu wird der AST-Teilbaum der Deklaration benötigt.

Beim Aufruf muss man das Funktionssymbol im aktuellen Kontext suchen, die Argumente auswerten, einen neuen lokalen Kontext anlegen und darin die Parameter definieren (mit den eben ausgewerteten Werten) und anschließend den AST-Teilbaum des Funktionskörpers im Interpreter mit `eval()` auswerten …

6.2.2. Ausführen einer Funktionsdeklaration

```
funcDecl : type ID '(' params? ')' block ;
funcCall : ID '(' exprList? ')' ;
```

```
def funcDecl(self, AST t):
    fn = Fun(t, self.env)
    self.env.define(t.ID().getText(), fn)
```



Quelle: Funktionsdeklaration: Eigener Code basierend auf einer Idee nach LoxFunction.java by Bob Nystrom on Github.com (MIT)

Man definiert im aktuellen Environment den Funktionsnamen und hält dazu den aktuellen Kontext (aktuelles Environment) sowie den AST-Knoten mit der eigentlichen Funktionsdefinition fest.

Für *Closures* ist der aktuelle Kontext wichtig, sobald man die Funktion ausführen muss. In (Parr 2010, 236) wird beispielsweise einfach nur ein neuer Memory-Space (entspricht ungefähr hier einem neuen lokalen Environment) angelegt, in dem die im Funktionskörper definierten Symbole angelegt werden. Die Suche nach Symbolen erfolgt dort nur im Memory-Space (Environment) der Funktion bzw. im globalen Scope (Environment).

6.2.3. Ausführen eines Funktionsaufrufs

```
funcDecl : type ID '(' params? ')' block ;
funcCall : ID '(' exprList? ')' ;
```

```
def funcCall(self, AST t):
    fn = (Fun)eval(t.ID())
    args = [eval(a)  for a in t.exprList()]

    prev = self.env;  self.env = Environment(fn.closure)
    for i in range(args.size()):
        self.env.define(fn.decl.params()[i].getText(), args[i])

    eval(fn.decl.block())
    self.env = prev
```

Quelle: Funktionsaufruf: Eigener Code basierend auf einer Idee nach LoxFunction.java by Bob Nystrom on Github.com (MIT)

Zunächst wird die `ID` im aktuellen Kontext ausgewertet. In der obigen Grammatik ist dies tatsächlich nur ein Funktionsname, aber man könnte über diesen Mechanismus auch Ausdrücke erlauben und damit Funktionspointer bzw. Funktionsreferenzen realisieren … Im Ergebnis hat man das Funktionsobjekt mit dem zugehörigen AST-Knoten und dem Kontext zur Deklarationszeit.

Die Argumente der Funktion werden nacheinander ebenfalls im aktuellen Kontext ausgewertet.

Um den Funktionsblock auszuwerten, legt man einen neuen temporären Kontext über dem Closure-Kontext der Funktion an und definiert darin die Parameter der Funktion samt den aktuellen Werten. Dann lässt man den Interpreter über den Visitor-Dispatch den Funktionskörper evaluieren und schaltet wieder auf den Kontext vor der Funktionsauswertung zurück.

6.2.4. Funktionsaufruf: Rückgabewerte

```
def funcCall(self, AST t):
    ...

    eval(fn.decl.block())

    ...
    return None  # (Wirkung)
```

```
class ReturnEx(RuntimeException):
    __init__(self, v): self.value = v

def return(self, AST t):
    raise ReturnEx(eval(t.expr()))

def funcCall(self, AST t):
    ...
    erg = None
    try: eval(fn.decl.block())
    except ReturnEx as r: erg = r.value
    ...
    return erg;
```

Quelle: Rückgabewerte: Eigener Code basierend auf einer Idee nach Return.java und LoxFunction.java by Bob Nystrom on Github.com (MIT)

Rückgabewerte für den Funktionsaufruf werden innerhalb von `block` berechnet, wo eine Reihe von Anweisungen interpretiert werden, weshalb `block` ursprünglich keinen Rückgabewert hat. Im Prinzip könnte man `block` etwas zurück geben lassen, was durch die möglicherweise tiefe Rekursion relativ umständlich werden kann.

An dieser Stelle kann man den Exceptions-Mechanismus **missbrauchen** und bei der Auswertung eines **`return`** mit dem Ergebniswert direkt zum Funktionsaufruf zurück springen. In Methoden, wo man einen neuen lokalen Kontext anlegt und die globale `env`-Variable temporär damit ersetzt, muss man dann ebenfalls mit **`try`**/**`catch`** arbeiten und im **`finally`**-Block die Umgebung zurücksetzen und die Exception erneut werfen.

6.2.5. Native Funktionen

```
class Callable:
    def call(self, Interpreter i, List<Object> a): pass
class Fun(Callable): ...
class NativePrint(Fun):
    def call(self, Interpreter i, List<Object> a):
        for o in a: print a  # nur zur Demo, hier sinnvoller Code :-)

# Im Interpreter (Initialisierung):
self.env.define("print", NativePrint())

def funcCall(self, AST t):
    ...
#    prev = self.env;  self.env = Environment(fn.closure)
#    for i in range(args.size()): ...
#    eval(fn.decl.block()); self.env = prev
    fn.call(self, args)
```

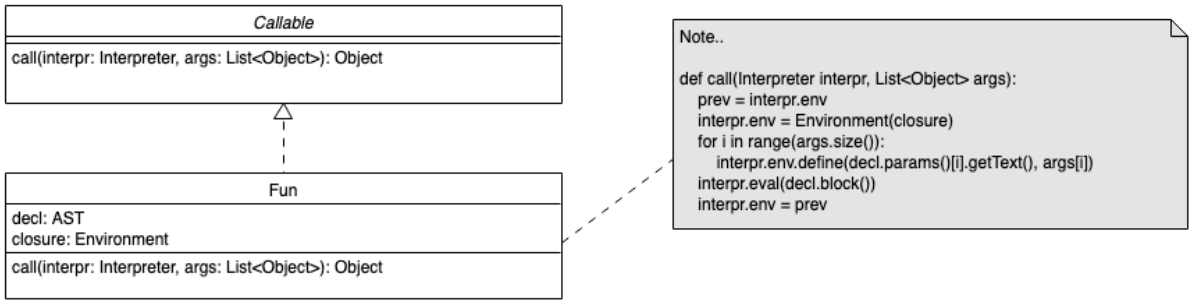
```
17        ...
```

Quelle: Native Funktionen: Eigener Code basierend auf einer Idee nach LoxCallable.java und LoxFunction.java by Bob Nystrom on Github.com (MIT)

Normalerweise wird beim Interpretieren eines Funktionsaufrufs der Funktionskörper (repräsentiert durch den entsprechenden AST-Teilbaum) durch einen rekursiven Aufruf von `eval` ausgewertet.

Für native Funktionen, die im Interpreter eingebettet sind, klappt das nicht mehr, da hier kein AST vorliegt.

Man erstellt ein neues Interface `Callable` mit der Hauptmethode `call()` und leitet die frühere Klasse `Fun` davon ab: `class Fun(Callable)`. Die Methode `funcCall()` des Interpreters ruft nun statt der `eval()`-Methode die `call()`-Methode des Funktionsobjekts auf und übergibt den Interpreter (== Zustand) und die Argumente. Die `call()`-Methode der Klasse `Fun` muss nun ihrerseits im Normalfall den im Funktionsobjekt referenzierten AST-Teilbaum des Funktionskörpers mit dem Aufruf von `eval()` interpretieren …



Für die nativen Funktionen leitet man einfach eine (anonyme) Klasse ab und speichert sie unter dem gewünschten Namen im globalen Kontext des Interpreters. Die `call()`-Methode wird dann entsprechend der gewünschten Funktion implementiert, d.h. hier erfolgt kein weiteres Auswerten des AST.

6.2.6. Klassen und Instanzen I

```
classDef : "class" ID "{" funcDecl* "}" ;
```

```
def classDef(self, AST t):
    methods = HashMap<String, Fun>()
    for m in t.funcDecl():
        fn = Fun(m, self.env)
        methods[m.ID().getText()] = fn

    clazz = Clazz(methods)
    self.env.define(t.ID().getText(), clazz)
```

Quelle: Klassen: Eigener Code basierend auf einer Idee nach Interpreter.java by Bob Nystrom on Github.com (MIT)

Anmerkung: In dieser Darstellung wird der Einfachheit halber nur auf Methoden eingegangen. Für Attribute müssten ähnliche Konstrukte implementiert werden.

6.2.7. Klassen und Instanzen II

```
class Clazz(Callable):
    __init__(self, Map<String, Fun> methods):
        self.methods = methods

    def call(self, Interpreter i, List<Object> a):
        return Instance(self)

    def findMethod(self, String name):
        return self.methods[name]

class Instance:
    __init__(self, Clazz clazz):
        self.clazz = clazz

    def get(self, String name):
        method = self.clazz.findMethod(name)
        if method != None: return method.bind(self)
        raise RuntimeError(name, "undefined method")
```

Quelle: Instanzen: Eigener Code basierend auf einer Idee nach LoxClass.java und LoxInstance.java by Bob Nystrom on Github.com (MIT)

Instanzen einer Klasse werden durch den funktionsartigen “Aufruf” der Klassen angelegt (parameterloser Konstruktor). Eine Instanz hält die Attribute (hier nicht gezeigt) und eine Referenz auf die Klasse, um später an die Methoden heranzukommen.

6.2.8. Zugriff auf Methoden (und Attribute)

```
getExpr : obj "." ID ;
```

```
def getExpr(self, AST t):
    obj = eval(t.obj())

    if isinstance(obj, Instance):
        return ((Instance)obj).get(t.ID().getText())

    raise RuntimeError(t.obj().getText(), "no object")
```

Beim Zugriff auf Attribute muss das Objekt im aktuellen Kontext evaluiert werden. Falls es eine Instanz von `Instance` ist, wird auf das Feld per interner Hash-Map zugriffen; sonst Exception.

6.2.9. Methoden und *this* oder *self*

```
class Fun(Callable):
    def bind(self, Instance i):
        e = Environment(self.closure)
        e.define("this", i)
        e.define("self", i)
        return Fun(self.decl, e)
```

Quelle: Methodenaufruf: Eigener Code basierend auf einer Idee nach LoxFunction.java by Bob Nystrom on Github.com (MIT)

Nach dem Interpretieren von Klassendefinitionen sind die Methoden in der Klasse selbst gespeichert, wobei der jeweilige `closure` auf den Klassenkontext zeigt.

Beim Auflösen eines Methodenaufrufs wird die gefundene Methode an die Instanz gebunden, d.h. es wird eine neue Funktion angelegt, deren `closure` auf den Kontext der Instanz zeigt. Zusätzlich wird in diesem Kontext noch die Variable "**`this`**" definiert, damit man damit auf die Instanz zugreifen kann.

In Python wird das in der Methodensignatur sichtbar: Der erste Parameter ist eine Referenz auf die Instanz, auf der diese Methode ausgeführt werden soll …

6.2.10. Wrap-Up

- Interpreter simulieren die Programmausführung
  - Namen und Symbole auflösen
  - Speicherbereiche simulieren
  - Code ausführen: Read-Eval-Loop
- Traversierung des AST: `eval(AST t)` als Visitor-Dispatcher
- Scopes mit `Environment` (analog zu Symboltabellen)
- Interpretation von Funktionen (Deklaration/Aufruf, native Funktionen)
- Interpretation von Klassen und Instanzen

Zum Nachlesen

- Nystrom (2021): Kapitel: A Tree-Walk Interpreter, insb. 10. Functions u. 12. Classes
- Grune u. a. (2012): Kapitel 6
- Mogensen (2017): Kapitel 4

Lernziele

- k3: Ich kann die Traversierung von Parse-Trees implementieren und dabei mit Hilfe des Visitor-Patterns Aktionen ausführen
- k3: Ich kann Environment-Strukturen analog zu den Symboltabellen aufbauen, um Namen und Werte dynamisch zu speichern
- k3: Ich kann eine Read-Eval-Schleife implementieren und dabei durch Traversierung des AST die dort abgelegten Klassendefinitionen und Methodenaufrufe ausführen

Challenges

- Wie interpretiert man Code?
- Warum kann man die Werte nicht einfach in Symboltabellen ablegen?
- Wie geht man mit Funktionen um? Warum? Kann man diese mehrfach aufrufen?
- Wieso erzeugt man eine neue Environment mit der Closure in der Funktion?
- Wie gehen native Funktionen?

Betrachten Sie folgenden Code-Ausschnitt:

```
int x = 42;
int f(int x) {
    int y = 9;
    return y+x;
}

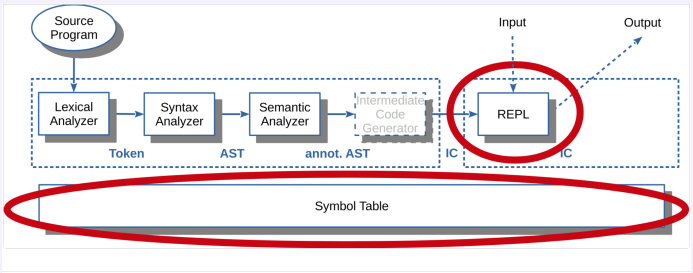
x = f(x);
```

1. Geben Sie den AST an.
2. Stellen Sie die Strukturen der Symboltabelle dar.
3. Stellen Sie die Strukturen im Interpreter dar.

6.3. Syntaxgesteuerte Interpreter

TL;DR

Zur Einordnung noch einmal die bisher betrachteten Phasen und die jeweiligen Ergebnisse:



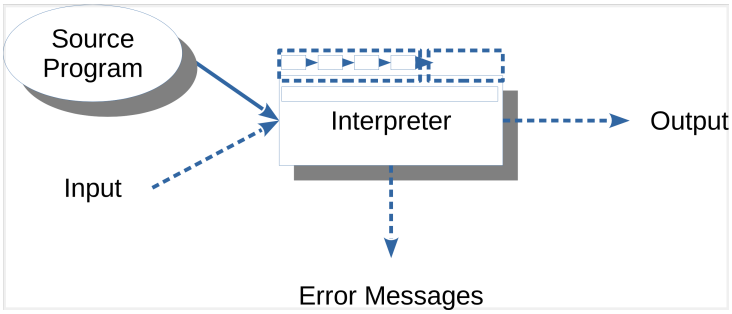
	Phase	Ergebnis
0	Lexer/Parser	AST
1	Semantische Analyse, Def-Phase	Symboltabelle (Definitionen), Verknüpfung Scopes mit AST-Knoten
2	Semantische Analyse, Ref-Phase	Prüfung auf nicht definierte Referenzen
3	Interpreter	Abarbeitung, Nutzung von AST und Symboltabelle

Das Erzeugen der Symboltabelle wird häufig in zwei Phasen aufgeteilt: Zunächst werden die Definitionen abgearbeitet und in der zweiten Phase wird noch einmal über den AST iteriert und die Referenzen werden geprüft. Dies hat den Vorteil, dass man mit Vorwärtsreferenzen arbeiten kann …
Für die semantische Analyse kann man gut mit Listenern arbeiten, für den Interpreter werden oft Visitors eingesetzt.
Die einfachste Form von Interpretern sind die "syntaxgesteuerten Interpreter". Durch den Einsatz von attributierten Grammatiken und eingebetteten Aktionen kann in einfachen Fällen der Programmcode bereits beim Parsen interpretiert werden, d.h. nach dem Parsen steht das Ergebnis fest.
Normalerweise traversiert man in Interpretern aber den AST, etwa mit dem Listener- oder Visitor-Pattern. Die in dieser Sitzung gezeigten einfachen Beispiele der syntaxgesteuerten Interpreter werden erweitert auf die jeweilige Traversierung mit dem Listener- bzw. Visitor-Pattern. Für nicht so einfache Fälle braucht man aber zusätzlich noch Speicherstrukturen, die wir in AST-basierte Interpreter: Basics und AST-basierte Interpreter: Funktionen und Klassen betrachten.

Videos

Vorlesung [YT], [HSBI]

6.3.1. Überblick Interpreter



Beim Interpreter durchläuft der Sourcecode nur das Frontend, also die Analyse. Es wird kein Code erzeugt, stattdessen führt der Interpreter die Anweisungen im AST bzw. IC aus. Dazu muss der Interpreter mit den Eingabedaten beschickt werden.

Es gibt verschiedene Varianten, beispielsweise:

- Syntaxgesteuerte Interpreter
  - Einfachste Variante, wird direkt im Parser mit abgearbeitet
  - Keine Symboltabellen, d.h. auch keine Typprüfung oder Vorwärtsdeklarationen o.ä. (d.h. erlaubt nur vergleichsweise einfache Sprachen)
  - Beispiel: siehe nächste Folie
- AST-basierte Interpreter

  - Nutzt den AST und Symboltabellen
  - Beispiel: siehe weiter unten

- Stack-basierte Interpreter
  - Simuliert eine *Stack Machine*, d.h. hält alle (temporären) Werte auf einem Stack
  - Arbeitet typischerweise auf bereits stark vereinfachtem Zwischencode (IR), etwa Bytecode

- Register-basierte Interpreter
  - Simuliert eine *Register Machine*, d.h. hält alle (temporären) Werte in virtuellen Prozessor-Registern
  - Arbeitet typischerweise auf bereits stark vereinfachtem Zwischencode (IR), etwa Bytecode

Weiterhin kann man Interpreter danach unterscheiden, ob sie interaktiv sind oder nicht. Python kann beispielsweise direkt komplette Dateien verarbeiten oder interaktiv Eingaben abarbeiten. Letztlich kommen dabei aber die oben dargestellten Varianten zum Einsatz.

6.3.2. Syntaxgesteuerte Interpreter: Attributierte Grammatiken

```
1  s     : expr                    {System.err.println($expr.v);} ;
2
3  expr returns [int v]
4        : e1=expr '*' e2=expr     {$v = $e1.v * $e2.v;}
5        | e1=expr '+' e2=expr     {$v = $e1.v + $e2.v;}
6        | DIGIT                   {$v = $DIGIT.int;}
7        ;
8
9  DIGIT : [0-9] ;
```

Die einfachste Form des Interpreters wird direkt beim Parsen ausgeführt und kommt ohne AST aus. Der Nachteil ist, dass der AST dabei nicht vorverarbeitet werden kann, insbesondere entfallen semantische Prüfungen weitgehend.

Über `returns [int v]` fügt man der Regel `expr` ein Attribut `v` (Integer) hinzu, welches man im jeweiligen Kontext abfragen bzw. setzen kann (agiert als Rückgabewert der generierten Methode). Auf diesen Wert kann in den Aktionen mit `$v` zugegriffen werden.

Da in den Alternativen der Regel `expr` jeweils zwei "Aufrufe" dieser Regel auftauchen, muss man per "`e1=expr`" bzw. "`e2=expr`" eindeutige Namen für die "Aufrufe" vergeben, hier `e1` und `e2`.

6.3.3. Eingebettete Aktionen in ANTLR I

Erinnerung: ANTLR generiert einen LL-Parser, d.h. es wird zu jeder Regel eine entsprechende Methode generiert.

Analog zum Rückgabewert der Regel (Methode) `expr()` kann auf die Eigenschaften der Token und Sub-Regeln zugegriffen werden: `$name.eigenschaft`. Dabei gibt es bei Token Eigenschaften wie `text` (gematchter Text bei Token), `type` (Typ eines Tokens), **`int`** (Integerwert eines Tokens, entspricht `Integer.valueOf($Token.text)`). Parser-Regeln haben u.a. ein `text`-Attribut und ein spezielles Kontext-Objekt (`ctx`).

Die allgemeine Form lautet:

```
rulename[args] returns [retvals] locals [localvars] : ... ;
```

Dabei werden die in "`[...]`" genannten Parameter mit Komma getrennt (Achtung: Abhängig von Zielsprache!).

Beispiel:

```
add[int x] returns [int r] : '+=' INT {$r = $x + $INT.int;} ;
```

6.3.4. Eingebettete Aktionen in ANTLR II

```
@members {
    int count = 0;
}

expr returns [int v]
      @after {System.out.println(count);}
      : e1=expr '*' e2=expr     {$v = $e1.v * $e2.v; count++;}
      | e1=expr '+' e2=expr     {$v = $e1.v + $e2.v; count++;}
      | DIGIT                   {$v = $DIGIT.int;}
      ;

DIGIT : [0-9] ;
```

Mit `@members { ... }` können im generierten Parser weitere Attribute angelegt werden, die in den Regeln normal genutzt werden können.

Die mit `@after` markierte Aktion wird am Ende der Regel `list` ausgeführt. Analog existiert `@init`.

6.3.5. ANTLR: Traversierung des AST und Auslesen von Kontext-Objekten

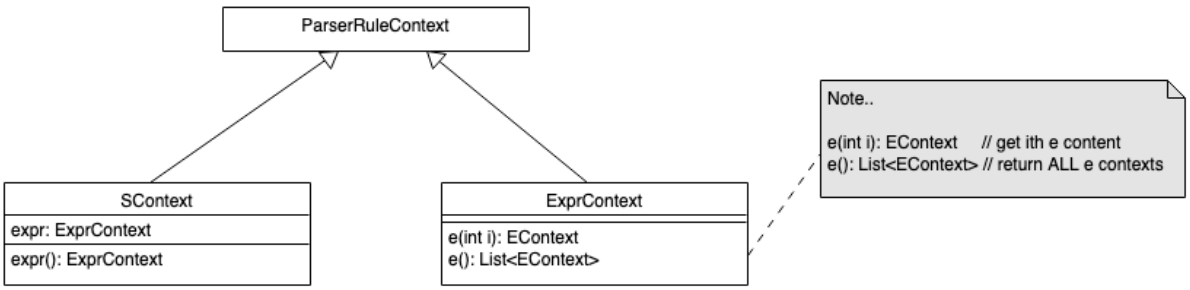
Mit dem obigen Beispiel, welches dem Einsatz einer L-attributierten SDD in ANTLR entspricht, können einfache Aufgaben bereits beim Parsen erledigt werden.

Für den etwas komplexeren Einsatz von attributierten Grammatiken kann man die von ANTLR erzeugten Kontext-Objekte für die einzelnen AST-Knoten nutzen und über den AST mit dem Visitor- oder dem Listener-Pattern iterieren.

Die Techniken sollen im Folgenden kurz vorgestellt werden.

6.3.5.1. ANTLR: Kontext-Objekte für Parser-Regeln

```
s    : expr         {List<EContext> x = $expr.ctx.e();} ;
expr : e '*' e ;
```



Jede Regel liefert ein passend zu dieser Regel generiertes Kontext-Objekt zurück. Darüber kann man das/die Kontextobjekt(e) der Sub-Regeln abfragen.

Die Regel `s()` liefert entsprechend ein `SContext`-Objekt und die Regel `expr()` liefert ein `ExprContext`-Objekt zurück.

In der Aktion fragt man das Kontextobjekt über `ctx` ab.

Für einfache Regel-Aufrufe liefert die parameterlose Methode nur ein einziges Kontextobjekt (statt einer Liste) zurück.

Anmerkung: ANTLR generiert nur dann Felder für die Regel-Elemente im Kontextobjekt, wenn diese in irgendeiner Form referenziert werden. Dies kann beispielsweise durch Benennung (Definition eines Labels, siehe nächste Folie) oder durch Nutzung in einer Aktion (siehe obiges Beispiel) geschehen.

6.3.5.2. ANTLR: Benannte Regel-Elemente oder Alternativen

```
1  stat  : 'return' value=e ';'    # Return
2        | 'break' ';'             # Break
3        ;
```

```
1  public static class StatContext extends ParserRuleContext { ... }
2  public static class ReturnContext extends StatContext {
3      public EContext value;
4      public EContext e() { ... }
5  }
6  public static class BreakContext extends StatContext { ... }
```

Mit `value=e` wird der Aufruf der Regel `e` mit dem Label `value` belegt, d.h. man kann mit `$e.text` oder `$value.text` auf das `text`-Attribut von `e` zugreifen. Falls es in einer Produktion mehrere Aufrufe einer anderen Regel gibt, **muss** man für den Zugriff auf die Attribute eindeutige Label vergeben.

Analog wird für die beiden Alternativen je ein eigener Kontext erzeugt.

6.3.5.3. ANTLR: Arbeiten mit dem Listener-Pattern

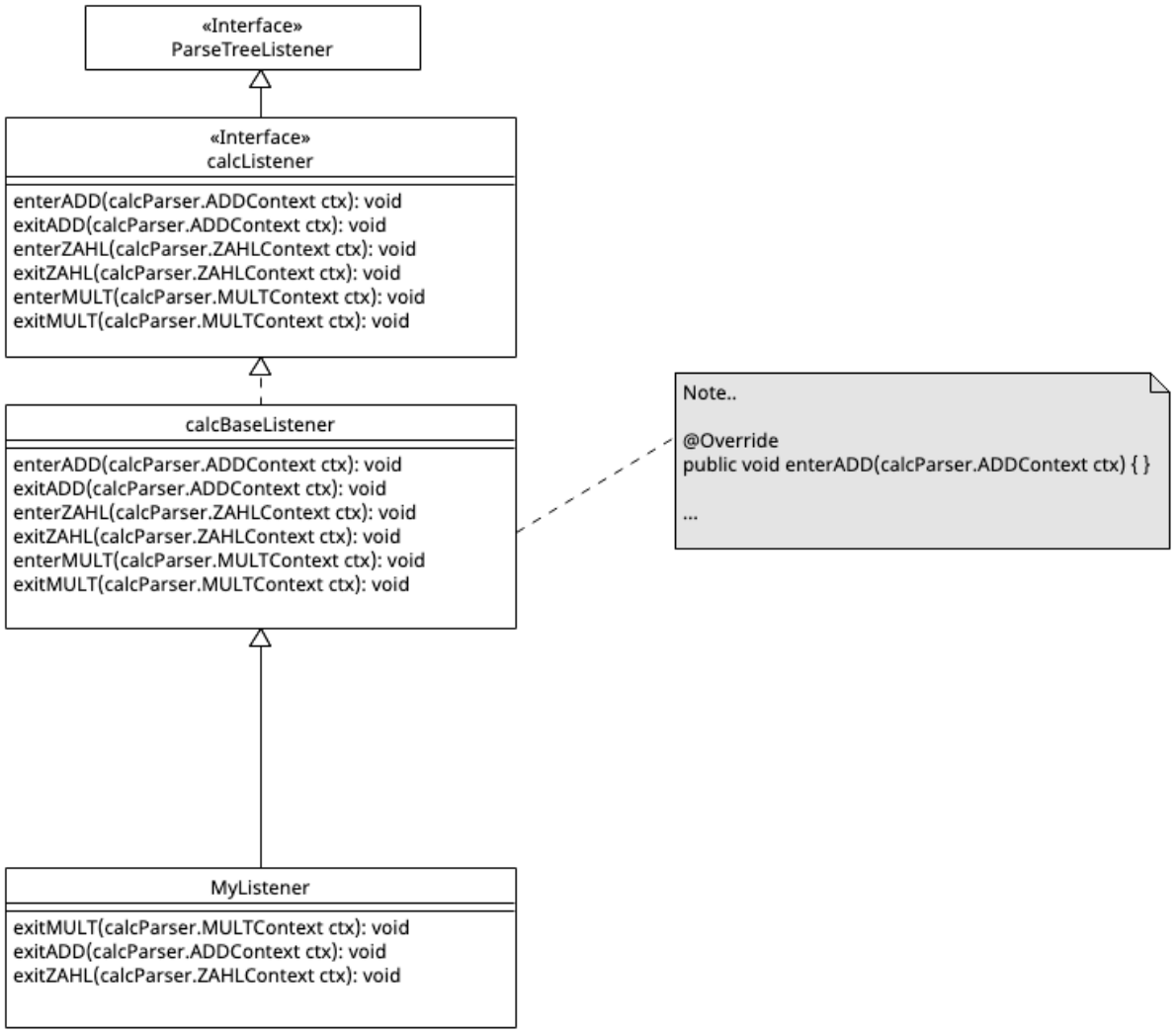
ANTLR (generiert auf Wunsch) zur Grammatik passende Listener (Interface und leere Basisimplementierung). Beim Traversieren mit dem Default-`ParseTreeWalker` wird der Parse-Tree mit Tiefensuche abgelaufen und

jeweils beim Eintritt in bzw. beim Austritt aus einen/m Knoten der passende Listener mit dem passenden Kontext-Objekt aufgerufen.

Damit kann man die Grammatik “für sich” halten, d.h. unabhängig von einer konkreten Zielsprache und die Aktionen über die Listener (oder Visitors, s.u.) ausführen.

```
1  expr : e1=expr '*' e2=expr      # MULT
2       | e1=expr '+' e2=expr      # ADD
3       | DIGIT                    # ZAHL
4       ;
```

ANTLR kann zu dieser Grammatik einen passenden Listener (Interface `calcListener`) generieren. Weiterhin generiert ANTLR eine leere Basisimplementierung (Klasse `calcBaseListener`):



Von dieser Basisklasse leitet man einen eigenen Listener ab und implementiert die Methoden, die man benötigt.

```java
public static class MyListener extends calcBaseListener {
    Stack<Integer> stack = new Stack<Integer>();

    public void exitMULT(calcParser.MULTContext ctx) {
        int right = stack.pop();
        int left = stack.pop();
        stack.push(left * right);   // {$v = $e1.v * $e2.v;}
    }
    public void exitADD(calcParser.ADDContext ctx) {
        int right = stack.pop();
        int left = stack.pop();
        stack.push(left + right);   // {$v = $e1.v + $e2.v;}
    }
    public void exitZAHL(calcParser.ZAHLContext ctx) {
        stack.push(Integer.valueOf(ctx.DIGIT().getText()));
    }
}
```

Anschließend baut man das alles in eine Traversierung des Parse-Trees ein:

```java
public class TestMyListener {
    public static class MyListener extends calcBaseListener {
        ...
    }

    public static void main(String[] args) throws Exception {
        calcLexer lexer = new calcLexer(CharStreams.fromStream(System.in));
        CommonTokenStream tokens = new CommonTokenStream(lexer);
        calcParser parser = new calcParser(tokens);

        ParseTree tree = parser.s();    // Start-Regel
        System.out.println(tree.toStringTree(parser));

        ParseTreeWalker walker = new ParseTreeWalker();
        MyListener eval = new MyListener();
        walker.walk(eval, tree);
        System.out.println(eval.stack.pop());
    }
}
```

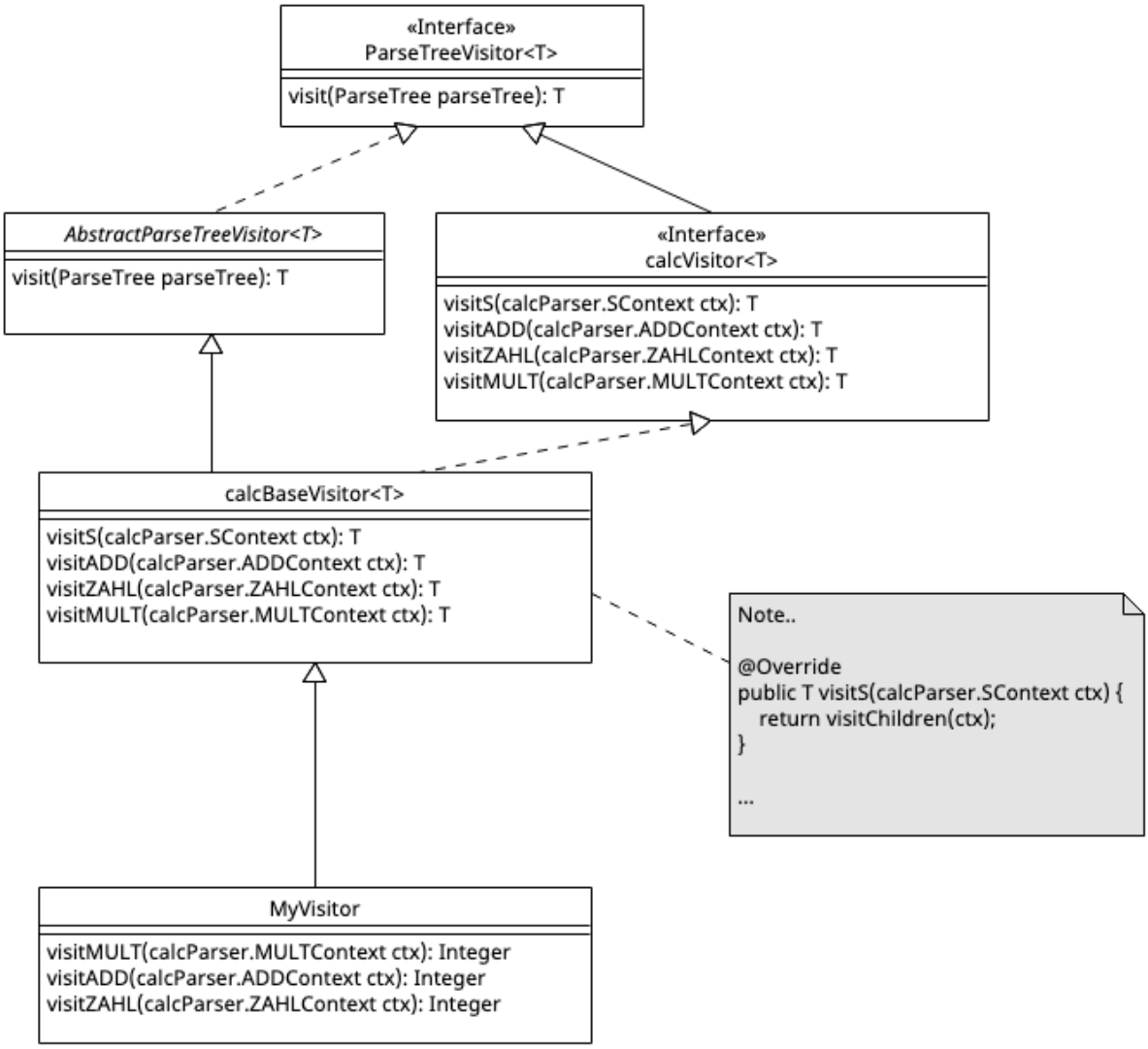
Beispiel: TestMyListener.java und calc.g4

6.3.5.4. ANTLR: Arbeiten mit dem Visitor-Pattern

ANTLR (generiert ebenfalls auf Wunsch) zur Grammatik passende Visitoren (Interface und leere Basisimplementierung). Hier muss man allerdings selbst für eine geeignete Traversierung des Parse-Trees sorgen. Dafür hat man mehr Freiheiten im Vergleich zum Listener-Pattern, insbesondere im Hinblick auf Rückgabewerte.

```
expr : e1=expr '*' e2=expr      # MULT
     | e1=expr '+' e2=expr      # ADD
     | DIGIT                    # ZAHL
     ;
```

ANTLR kann zu dieser Grammatik einen passenden Visitor (Interface `calcVisitor<T>`) generieren. Weiterhin generiert ANTLR eine leere Basisimplementierung (Klasse `calcBaseVisitor<T>`):



Von dieser Basisklasse leitet man einen eigenen Visitor ab und überschreibt die Methoden, die man benötigt. Wichtig ist, dass man selbst für das “Besuchen” der Kindknoten sorgen muss (rekursiver Aufruf der geerbten Methode `visit()`).

```
1  public static class MyVisitor extends calcBaseVisitor<Integer> {
2      public Integer visitMULT(calcParser.MULTContext ctx) {
3          return visit(ctx.e1) * visit(ctx.e2);   // {$v = $e1.v * $e2.v;}
4      }
5      public Integer visitADD(calcParser.ADDContext ctx) {
6          return visit(ctx.e1) + visit(ctx.e2);   // {$v = $e1.v + $e2.v;}
7      }
8      public Integer visitZAHL(calcParser.ZAHLContext ctx) {
```

```
9           return Integer.valueOf(ctx.DIGIT().getText());
10      }
11  }
```

Anschließend baut man das alles in eine manuelle Traversierung des Parse-Trees ein:

```
1   public class TestMyVisitor {
2       public static class MyVisitor extends calcBaseVisitor<Integer> {
3           ...
4       }
5
6       public static void main(String[] args) throws Exception {
7           calcLexer lexer = new calcLexer(CharStreams.fromStream(System.in));
8           CommonTokenStream tokens = new CommonTokenStream(lexer);
9           calcParser parser = new calcParser(tokens);
10
11          ParseTree tree = parser.s();    // Start-Regel
12          System.out.println(tree.toStringTree(parser));
13
14          MyVisitor eval = new MyVisitor();
15          System.out.println(eval.visit(tree));
16      }
17  }
```

Beispiel: TestMyVisitor.java und calc.g4

6.3.6. Wrap-Up

- Interpreter simulieren die Programmausführung

- Syntaxgesteuerter Interpreter (attributierte Grammatiken)
- Beispiel ANTLR: Eingebettete Aktionen, Kontextobjekte, Visitors/Listeners (AST-Traversierung)

Zum Nachlesen

- Nystrom (2021): Kapitel: A Tree-Walk Interpreter
- Levine (2009): Bison, Kapitel 6
- Parr (2014): Kapitel 6.4 und 8.4
- Parr (2010): Kapitel 8 und 9

Lernziele

- k3: Ich kann Attribute und eingebettete Aktionen in ANTLR einsetzen
- k3: Ich kann Parse-Trees traversieren und Aktionen implementieren mit Hilfe des Listener-Patterns
- k3: Ich kann Parse-Trees traversieren und Aktionen implementieren mit Hilfe des Visitor-Patterns

7. Optimierung

Üblicherweise finden sich die verschiedenen Optimierungsmöglichkeiten in der Backend-Phase. Man kann u.a. zwischen lokaler und globaler Optimierung oder beispielsweise JIT unterscheiden. Es kann sehr unterschiedliche Ziele für die Optimierung geben, beispielsweise möchte man den Code kleiner(kürzer, kompakter) gestalten oder die Laufzeit für das Programm verringern oder sogar Energieaspekte berücksichtigen.

7.1. Optimierung und Datenflussanalyse

Weitere Unterlagen

- Annotierte Folien: Optimierung und Datenflussanalyse

7.1.1. Motivation

7.1.1.1. Was geschieht hier?

```
01  {
02    var a;
03    var b = 2;
04    b = a;
05  }
```

7.1.2. Thema für heute: Optimierungen

7.1.2.1. Was ist Optimierung in Compilern?

Verändern von Quellcode, Zwischencode oder Maschinencode eines Programms mit dem Ziel,

- Laufzeit,
- Speicherplatz oder
- Energieverbrauch

zu verbessern.

7.1.2.2. Was ist machbar?

Manche Optimierungen machen den Code nur in bestimmten Fällen schneller, kleiner oder stromsparender.

Den optimalen Code zu finden, ist oft NP-vollständig oder sogar unentscheidbar.

- Heuristiken kommen zum Einsatz.
- Der Code wird verbessert, nicht in jedem Fall optimiert, manchmal auch verschlechtert.
- Der Einsatz eines Debuggers ist meist nicht mehr möglich.

7.1.2.3. Anforderungen an Optimierung

- sichere Transformationen durchführen
- möglichst keine nachteiligen Effekte erzeugen

7.1.2.4. Optimierung zur Übersetzungszeit vs. Optimierung zur Laufzeit

- Just-in-time-Compilierung (JIT), z. B. Java:

  Fast alle Optimierungsmaßnahmen finden in der virtuellen Maschine zur Laufzeit statt.
- Ahead-of-time-Compilierung (AOT), z. B. C:

  Der Compiler erzeugt Maschinencode, die Optimierung findet zur Übersetzungszeit statt.

Beide haben ihre eigenen Optimierungsmöglichkeiten, es gibt aber auch Methoden, die bei beiden einsetzbar sind.

7.1.2.5. Welcher Code wird optimiert?

- Algebraische Optimierung: Transformationen des Quellcodes
- Maschinenunabhängige Optimierung: Transformationen des Zwischencodes
- Maschinenabhängige Optimierung: Transformationen des Assemblercodes oder Maschinencodes

Viele Transformationen sind auf mehr als einer Ebene möglich. Wir wenden hier die meisten auf den Zwischencode an.

7.1.2.6. Welche Arten von Transformationen sind möglich?

- Eliminierung unnötiger Berechnungen
- Ersetzung von teuren Operationen durch kostengünstigere

7.1.2.7. Basisblöcke und Flussgraphen

Def.: Ein *Basisblock* ist eine Sequenz maximaler Länge von Anweisungen, die immer hintereinander ausgeführt werden.

Ein Sprungbefehl kann nur der letzte Befehl eines Basisblocks sein.

Def.: Ein *(Kontroll)Flussgraph* $G = (V, E)$ ist ein Graph mit

$V = \{B_i \mid B_i$ ist ein Basisblock des zu compilierenden Programms$\}$,

$E = \{(B_i, B_j) \mid$ es gibt einen Programmlauf, in dem B_j direkt hinter B_i ausgeführt wird$\}$

7.1.2.8. Beispiel

Hier entsteht ein Tafelbild.

7.1.2.9. Häufig benutzte Strategie: Peephole-Optimierung

Ein Fenster mit wenigen Zeilen Inhalt gleitet über den Quellcode, Zwischencode oder den Maschinencode. Der jeweils sichtbare Code wird mit Hilfe verschiedener Verfahren optimiert, wenn möglich.

Peephole-Optimierung ist zunächst ein lokales Verfahren, kann aber auch auf den gesamten Kontrollflussgraphen erweitert werden.

=> Anwendung von Graphalgorithmen!

7.1.3. Algebraische Optimierung

7.1.3.1. Ersetzen von Teilbäumen im AST durch andere Bäume

```
1      x = x*2          =>      x << 1
2
3      x = x + 0        // k.w.
4      x = x * 1        // k.w.
5
6      x = x*0          =>      x = 0
7      x = x*8          =>      x = x << 3
```

Sei $s = 2^a + 2^b$ die Summe zweier Zweierpotenzen:

```
1      x = n*s          =>      (n << a) + (n << b)
```

Diese Umformungen können zusätzlich mittels Peephole-Optimierung in späteren Optimierungsphasen durchgeführt werden.

7.1.4. Maschinenunabhängige Optimierung

7.1.4.1. Maschinenunabhängige Optimierung

- lokal (= innerhalb eines Basisblocks), z. B. Peephole-Optimierung

  Einige Strategien sind auch global einsetzbar (ohne die sog. Datenflussanalyse s. u.)

- global, braucht nicht-lokale Informationen
  - meist unter Zuhilfenahme der Datenflussanalyse
  - Schleifenoptimierung

7.1.4.2. Zwischencode (intermediate code); hier: Drei-Adress-Code

- registerbasiert
- Formen: `x = y op z`, `x = op z`, `x = y`
- temporäre Variablen für Zwischenergebnisse
- bedingte und unbedingte Sprünge
- Pointerarithmetik für Indizierung

```
i = 0
while(f[i] > 100)
    i = i + 1;
```

```
    i = 0
L1: t1 = i * 8
    t2 = f + t1
    if t2 <= 100 goto L2
    t3 = i + 1
    i = t3
    goto L1
L2: ...
```

7.1.5. Lokale Optimierung

7.1.5.1. Constant Folding und Common Subexpression elimination

- "*Constant Folding*": Auswerten von Konstanten zur Compile-Zeit

```
x = 6 * 7          =>      x = 42
if 2 > 0 jump L    =>      jump L
```

- "*Common Subexpression Elimination*"

```
x = y + z
...
a = y + z
```

ersetze mit (falls in `...` keine weiteren Zuweisungen an `x`, `y`, `z` erfolgen)

```
x = y + z
...
a = x
```

7.1.5.2. Elimination redundanter Berechnungen in einem Basisblock mitels DAGs

Hier werden sog. *DAG*s benötigt:

Ein DAG *directed acyclic graph* ist ein gerichteter, kreisfreier Graph.

DAGs werden für Berechnungen in Basisblöcken generiert, um gemeinsame Teilausdrücke zu erkennen.

Bsp.: a = (b + c) * (b + c) / 2

7.1.5.3. Copy propagation

- "*Copy Propagation*"

```
x = y + z
a = x
b = 2*a
```

ersetze mit

```
x = y + z
a = x
b = 2*x
```

Wenn auf *a* vor seiner nächsten Zuweisung nicht mehr lesend zugegriffen wird, kann *a* hier entfallen.

7.1.6. Globale Optimierung

7.1.6.1. Control Flow und Dead Code

- Kontrollfluss-Optimierungen

```
    if debug == 1 goto L1          if debug != 1 goto L2
    goto L2                        print debug info
L1: print debug info           L2: ...
L2: ...
```

- Elimination of unreachable code

```
     goto L1                       L1: a = b+c
     ...
 L1: a = b+c
```

7.1.6.2. Schleifenoptimierung

Loop unrolling:

```
        for i = 1 to 3             print("1")
            print(i)               print("2")
                                   print("3")
```

Code Hoisting:

- Invarianten vor die Schleife schieben

```
    x = 0                          x = 0
L: a = n*7                         a = n*7
    x = x + a                  L: x = x + a
    if x<42 jump L                 if x<42 jump L
```

7.1.6.3. Kombination zweier Verfahren

- Loop Unrolling (für eine Iteration), danach Common Subexpression Elimination

```
while (cond) {                 if (cond) {
    body                           body
}                                  while (cond) {
                                       body
                                   }
                               }
```

7.1.6.4. Datenflussanalyse

Die Datenflussanalyse (auf 3-Adress-Code) basiert auf dem Wissen der Verfügbarkeit von Variablen und Ausdrücken am Anfang oder Ende von Basisblöcken, und zwar für alle möglichen Programmläufe.

Man unterscheidet:

- Vorwärtsanalyse (in Richtung der Nachfolger eines Basisblocks)
- Rückwärtsanalyse (in Richtung der Vorgänger eines Basisblocks)

In beiden Fällen gibt es zwei Varianten:

- any analysis: Es wird die Vereinigung von Informationen benachbarter Block berücksichtigt.
- all analysis: Es wird die Schnittmenge von Informationen benachbarter Block berücksichtigt.

7.1.6.5. Forward-any-analysis

Diese Analyse wird zur Propagation von Konstanten und Variablen benutzt und bildet sukzessive Mengen von Zeilen mit Variablendefinitionen.

$$out(B_i) = gen(B_i) \cup (in(B_i) - kill(B_i))$$

$out(B_i)$: alle Zeilennummern von Variablendefinitionen, die am Ende von B_i gültig sind

$in(B_i)$: alle Zeilennummern von Variablendefinitionen, die am Ende von Vorgängerblöcken von B_i gültig sind

$gen(B_i)$: alle Zeilennummern von letzten Variablendefinitionen in B_i

$kill(B_i)$: alle Zeilennummern von Variablendefinitionen außerhalb von B_i, die in B_i überschrieben werden

Zunächst ist $in(B_1) = \emptyset$, danach ist $in(B_i) = \bigcup out(B_j)$ mit B_j ist Vorgänger von B_i.

7.1.6.6. Forward-all-analysis

Diese Analyse wird zur Berechnung verfügbarer Ausdrücke der Form $x = y\ op\ z$ für die Eliminierung redundanter Berechnungen benutzt und bildet sukzessive Mengen von Ausdrücken.

$$out(B_i) = gen(B_i) \cup (in(B_i) - kill(B_i))$$

$out(B_i)$: alle am Ende von B_i verfügbaren Ausdrücke

$in(B_i)$: alle Ausdrücke, die am Anfang von B_i verfügbar sind

$gen(B_i)$: alle in B_i berechneten Ausdrücke

$kill(B_i)$: alle Ausdrücke $x\ op\ y$ mit einer Definition von x oder y in B_i und $x\ op\ y$ ist nicht in B_i

Zunächst ist $gen(B_1) = \emptyset$, danach ist $in(B_i) = \bigcap out(B_j)$ mit B_j ist Vorgänger von B_i.

7.1.6.7. Backward-any-analysis

Diese Analyse dient der Ermittlung von lebenden und toten Variablen (für die Registerzuweisung) und bildet sukzessive Mengen von Variablen.

$$in(B_i) = gen(B_i) \cup (out(B_i) - kill(B_i))$$

$out(B_i)$: alle Variablen, die am Ende von B_i lebendig sind

$in(B_i)$: alle Variablen, die am Ende von Vorgängerblöcken von B_i lebendig sind

$gen(B_i)$: alle Variablen, deren erstes Vorkommen auf der echten Seite einer Zuweisung steht

$kill(B_i)$: alle Variablen, denen in B_i Werte zugewiesen werden.

Zunächst ist $out(B_n) = \emptyset$, danach ist $out(B_i) = \bigcup in(B_j)$ mit B_j ist Nachfolger von B_i.

7.1.6.8. Backward-all-analysis

Diese Analyse wird zur Berechnung von "very busy" Ausdrücken der Form $x = y\ op\ z$, die auf allen möglichen Wegen im Flussgraphen vom aktuellen Basisblock aus mindestens einmal benutzt werden. Ausdrücke sollten dort berechnet werden, wo sie very busy sind, um den Code kürzer zu machen.

$$in(B_i) = gen(B_i) \cup (out(B_i) - kill(B_i))$$

$out(B_i)$: alle Ausdrücke $x\ op\ y$, die am Ende von B_i very busy sind

$in(B_i)$: alle Ausdrücke, die am Anfang von B_i very busy sind

$gen(B_i)$: alle in B_i benutzen Ausdrücke

$kill(B_i)$: alle Ausdrücke $x\ op\ y$, deren Operanden in B_i nicht redefiniert werden.

Zunächst ist $out(B_n) = \emptyset$, danach ist $out(B_i) = \bigcap in(B_j)$ mit B_j ist Nachfolger von B_i.

7.1.7. Maschinenabhängige Optimierung

7.1.7.1. Elimination redundanter Lade-, Speicher- und Sprungoperationen

```
1     LD a, R0
2     ST R0, a        // k.w.
```

```
1         goto L1          goto L2
2         ...              ...
3     L1: goto L2      L1: goto L2
```

7.1.7.2. Register Allocation: Liveness Analysis

```
1 a = b + c
2 d = a + b
3 e = d - 1
```

a, d, e können auf **ein** Register abgebildet werden!

```
1 r1 = r2 + r3
2 r1 = r1 + r2
3 r1 = r1 - 1
```

=> a und d sind nach Gebrauch "*tot*"

7.1.7.3. Berechnung der minimal benötigten Anzahl von Registern

=> Liveness-Graph, Färbungsproblem für Graphen!

Es wird ein Graph $G = (V, E)$ erzeugt mit

$V = \{v \mid v \text{ ist eine benötigte Variable}\}$ und $E = \{(v_1, v_2) \mid v_1 \text{ und } v_2 \text{ sind zur selben Zeit "lebendig"}\}$

Heuristisch wird jetzt die minimale Anzahl von Farben für Knoten bestimmt, bei der benachbarte Knoten nicht dieselbe Farbe bekommen.

=> Das Ergebnis ist die Zahl der benötigten Register.

7.1.7.4. Und wenn man nicht so viele Register zur Verfügung hat?

Registerinhalte temporär in den Speicher auslagern ("*Spilling*").

Kandidaten dafür werden mit Heuristiken gefunden, z. B. Register mit vielen Konflikten (= Kanten) oder Register mit selten genutzten Variablen.

In Schleifen genutzte Variablen werden eher nicht ausgelagert.

7.1.8. Optimierung zur Reduzierung des Energieverbrauchs

7.1.8.1. Energieverbrauch verschiedener Maschinenbefehle

Maschinenoperationen, die nur auf Registern arbeiten, verbrauchen die wenigste Energie.

Operationen, die nur lesend auf Speicherzellen zugreifen, verbrauchen ca. ein Drittel mehr Energie.

Operationen, die Speicherzellen beschreiben, benötigen zwei Drittel mehr Energie als die Operationen ausschließlich auf Register.

7.1.8.2. Energieeinsparung durch laufzeitbezogene Optimierung

Kürzere Programmlaufzeiten führen in der Regel auch zu Energieeinsparungen.

gcc -O1 spart 2% bis 70% (durchschnittlich 20%) Energie

Umgekehrt: Energiebezogene Optimierung führt in der Regel zu kürzeren Laufzeiten.

7.1.8.3. Prozessorspannung variieren

Viele Prozessoren ermöglichen es, die Betriebsspannung per Maschinenbefehl zu verändern.

Eine höhere Spannung bewirkt eine proportionale Steigerung der Prozessorgeschwindigkeit und des fließenden Stroms, aber einen quadratischen Anstieg des Energieverbrauchs. $(P = U \times I, U = R \times I)$

Folgendes kann man ausnutzen:

Die Verringerung der Spannung um 20% führt zu einer um 20% geringeren Prozessorgeschwindigkeit, d. h. das Programm braucht 25% mehr Zeit, verbraucht aber 36% $(1 - (1 - 0,2)^2)$ weniger Energie.

=> Wenn das Programm durch Optimierung um 25% schneller wird und die Prozessorspannung um 20% verringert wird, verändert sich die Laufzeit des Programms nicht, man spart aber 36% Energie.

7.1.9. Wrap-Up

7.1.9.1. Wrap-Up

- Verschiedene Optimierungsverfahren auf verschiedenen Ebenen, Peephole
- Datenflussanalyse
- Senkung des Energieverbrauchs durch Optimierung

Zum Nachlesen

- Güting (1999): Kapitel 8
- Grune u. a. (2012): Kapitel 9.3
- Aho u. a. (2023): Kapitel 9
- Mogensen (2017): Kapitel 8, 10 und 11

Lernziele

- k1: Ich kenne verschiedene algebraische Optimierungen
- k1: Ich kenne verschiedene maschinenunabhängige Optimierungen
- k1: Ich kenne verschiedene maschinenabhängige Optimierungen
- k1: Ich kenne Datenflussanalyse auf 3-Adress-Code

Teil III.

Praktikum

Hier finden Sie die Übungsblätter.

8. Seminaristischer Unterricht: Vortragsthemen Kurzvortrag/Fachvortrag

8.1. Kurzvortrag “PL Feature” (ca. 20 Minuten, DE)

Ziel ist die Einführung in ein ausgewähltes Programmiersprachen-Thema. Das vortragende Team arbeitet die Kernideen heraus und demonstriert zentrale Konzepte anhand kleiner, prägnanter Beispiele und leitet die anschließende Diskussion.

Alle Teams haben die Literatur zu den Kurzvorträgen zumindest grob überflogen. Das vortragende Team leitet die an den Vortrag anschließende Diskussion und bereitet 2-4 gezielte Diskussionsfragen zur Aktivierung der Zuhörenden vor.

8.1.1. Verfügbare Themen

1. Object-Oriented Paradigm (OOP) ((Gabbrielli und Martini 2023), Kap. 10)
2. Functional Programming Paradigm (FP) ((Gabbrielli und Martini 2023), Kap. 11)
3. Logic Programming Paradigm (LP) ((Gabbrielli und Martini 2023), Kap. 12)
4. Constraint Programming Paradigm (CP) ((Gabbrielli und Martini 2023), Kap. 13)
5. Borrow Checking und Lifetimes am Beispiel Rust
6. Dependent Type Systems am Beispiel Idris
    - Why Dependent Types Matter
    - IDRIS —: systems programming meets full dependent types

8.1.2. Weitere mögliche Themen (nach Absprache)

- Algebraische Effekte und Effekt-Systeme (z.B. Koka), Exceptions vs. Effekte
- Pattern Matching und Algebraische Datentypen (OCaml/F#/Scala)
- Metaprogrammierung und Makros (Lisp/Clojure, Rust macro_rules! und proc-macros)
- Gradual Typing (TypeScript, Sorbet), Typklassen und Traits (Haskell/Rust)

8.1.3. Hinweise

- *Jede Person* bereitet sich vorab vor; das präsentierende Team moderiert die Diskussion

- Ziel ist Transferfähigkeit: Nach dem Vortrag sollen Zuhörende weiterführende Literatur verstehen und Ideen praktisch erproben können
- Empfohlen: Zwei bis drei gut kuratierte Codebeispiele (live oder als Snippets), Vorbereitung von einigen Diskussionsfragen
- Vortragssprache ist Deutsch

8.2. Fachvortrag “Compiler” (ca. 60 Minuten + 10 Minuten Q&A, DE)

Ziel ist die systematische Einführung in ein Compiler/VM-Thema mit genügend fachlicher Tiefe, so dass die Zuhörenden danach relevante Literatur lesen und das Thema praktisch anwenden können.

Bitte achten Sie auf eine präzise Begriffsbildung und den Einordnung in den Stand der Forschung/Praxis. Erstellen Sie eigene Beispiele und ggf. kurze Demos.

8.2.1. Verfügbare Themen

1. Parsergeneratoren (ANTLR, Tree-Sitter, Flex & Bison, …)
2. Fortgeschrittene Parsertechniken: LALR, PEG, Pratt, Parser-Combinators
3. Typen, Typsysteme, Type Checking
    - Hindley-Milner-Typsystem
    - Propositions as Types (Wadler)
    - Typechecker Zoo (Diehl)
    - On Understanding Types, Data Abstraction, and Polymorphism (Cardelli/Wegner)
4. VM & Bytecode
    - AST vs. Bytecode: Interpreters in the Age of Meta-Compilation
    - An Introduction to Interpreters and JIT Compilation
    - Optimizing the Order of Bytecode Handlers in Interpreters using a Genetic Algorithm
    - Beispiele: JVM oder WASM
5. Garbage Collection
    - (Nystrom 2021), Kap. 26
    - Unified Theory of Garbage Collection
    - Fast Conservative Garbage Collection
    - GC vs. explicit memory management
6. Just-in-Time Compilation (JIT)
    - An Introduction to Interpreters and JIT Compilation
    - AST vs. Bytecode: Interpreters in the Age of Meta-Compilation

8.2.2. Weitere mögliche Themen (nach Absprache)

- Fehlertolerantes Parsen und Diagnosequalität (Error Recovery, präzise Fehlermeldungen)
- Inkrementelle/Interaktive Compiler (IDE-Services, Language Server Protocol)
- Testen von Compilern
  - Finding and Understanding Bugs in C Compilers
  - Validating JIT Compilers via Compilation Space Exploration
  - A Survey of Compiler Testing
  - An empirical comparison of compiler testing techniques
  - Compiler Testing: A Systematic Literature Analysis
  - Snapshot Testing for Compilers
  - Tiny Unified Runner N' Tester (Turnt)
  - Testing Language Implementations

8.2.3. Hinweise

Planen Sie im Anschluss an den 60-minütigen Vortrag ca. 10 Minuten Q&A und Diskussion ein.

9. DSL-Projekt: Sprachen und Compiler

9.1. Ziel und Rahmen

In diesem Projekt beschäftigen Sie sich mit fortgeschrittenen Inhalten im Bereich Programmiersprachen und Compilerbau. Sie können eigenständige Ideen einbringen oder einen der nachfolgend genannten Vorschläge aufgreifen und verfeinern. Neben der praktischen Beschäftigung mit den Kurs-Themen recherchieren Sie selbstständig nach relevanter Fachliteratur und reflektieren Ihre Designentscheidungen.

> Für das MIF-1.5-Kursprojekt können Sie jede[1] Idee wählen, die einen Bezug zum Compilerbau hat.

9.2. Passende Themen (nicht abschließend)

Umfang und geeignete Themen (Beispiele, nicht abschließend)

- Sprachdesign (DSLs), Scanner/Parser (ANTLR, Tree-Sitter), Typ-/Effekt-Systeme
- IR-Design und -Transformation (Intermediate Representation), z.B. Bril, SSA, Dataflow-Analysen
- MLIR (Multi-Level IR): Dialektentwurf, Lowerings, Pass-Pipelines, Canonicalization
- Interpreter, VM/JIT, Codegenerierung (z.B. LLVM), Linken/Laden
- Statische Analysen, Verifikation, Symbolik/SMT, Transformationspipelines
- E-Graphs und Equality Saturation (z.B. egg/egglog) für optimierende Umformungen
- Tooling: Incremental Build/Hot-Reload, Profiling/Tracing, Debug-Infos, Fehlermeldungen

Bitte wählen Sie einen klar abgegrenzten Scope, der in Tiefe und Breite zum Semester passt. Definieren Sie mindestens:

- einen Minimalumfang, den Sie sicher erreichen wollen (und können), und
- ein oder zwei optionale Erweiterungen, die Sie angehen, wenn Zeit bleibt.

Der Minimalumfang muss einen Parser-/Analyse-/Interpreter-Kern enthalten, die Erweiterungen können z.B. IDE-Integration, Optimierungen oder empirische Evaluationen sein.

Potentiell interessante Literatur im Bereich DSL:

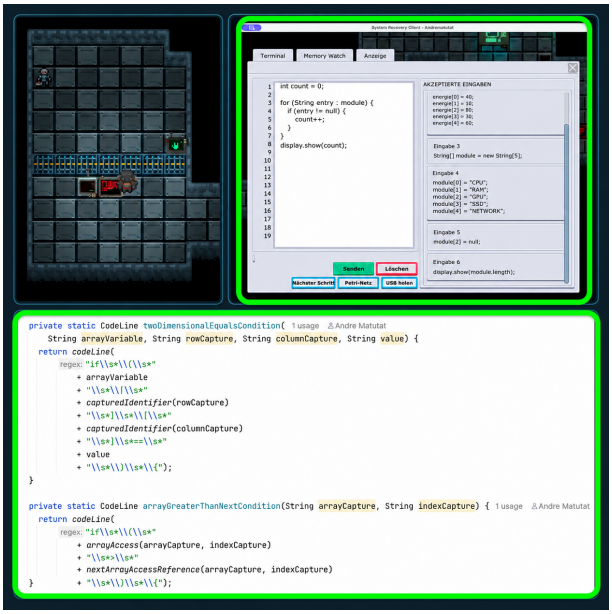
- Fowler (2010)
- Voelter (2013) (*Anmerkung*: Das Buch ist recht alt, aber die ersten drei Kapitel könnten hilfreich sein.)
- Mernik u. a. (2005)

[1] … for a given value of "*jede*" :) … Die Idee muss zum Thema passen und vom Anspruch und Umfang her einem 10 ECTS Master-Modul angemessen sein.

9.3. Projektanregungen

9.3.1. Escape-Rooms mit MiniJava-Auswertung

Parsen und Interpretieren von MiniJava-Code im Dungeon



Im Dungeon-Projekt wurden verschiedene Escape-Room-Rätsel realisiert. In einigen dieser Rätseln sollen die Spieler:innen Aufgaben lösen, indem sie MiniJava-Code (ein eingeschränkter Sub-Dialekt von Java) in ein Textfenster eingeben. Dieser Code muss syntaktisch und semantisch korrekt sein und soll geeignet evaluiert werden. Man sieht dabei live die Auswertung und die Auswirkung auf das Spiel. Diese Rätsel richten sich an Programmier-Einsteiger:innen, und zur Lösung der Rätsel sollen beispielsweise Kontrollstrukturen, Schleifen, Funktionen und ggf. auch Klassen definiert und eingesetzt werden.

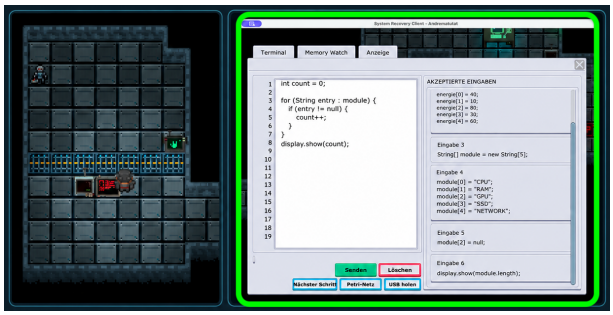
In der vorliegenden Implementierung wurden zu jeder Aufgabe ein oder mehrere teilweise komplexe reguläre Ausdrücke hinterlegt, mit der der eingegebene Code dann verglichen wird. Das ist nicht nur sehr umständlich und schlecht zu warten und zu erweitern, sondern erlaubt auch keine anderen Eingaben als die vorgesehenen, selbst wenn damit die Aufgabe inhaltlich korrekt gelöst werden würde.

Ziel dieses Projekt-Vorschlags ist zunächst die Erstellung eines inkrementellen Interpreters für die MiniJava-Eingaben. Der Interpreter muss dabei vorausgegangene Eingaben berücksichtigen (Session-Kontext oder Re-Initialisierung) und zu einer Eingabe ein Ergebnis liefern oder verständliche Fehlermeldungen. In einer zweiten Phase soll eine “Welt-Interaktion” (Interaktion des Interpreters mit dem Dungeon) realisiert werden: Über “eingebaute” Funktionen (“native functions”) sollen Spieler:innen direkt in ihrem MiniJava-Code Einfluss auf Elemente im Dungeon nehmen können. Es gibt beispielsweise einen kleinen Roboter, dessen Zustand im MiniJava-Code abgefragt und verändert werden kann. Dabei soll die Kopplung möglichst flexibel bleiben und die Spieler:innen sollen nicht der komplexen Dungeon-API ausgesetzt werden. Optional könnten statische Prüfungen (Schleifenbegrenzung, Laufzeitlimits, …) mit eingebaut werden und/oder untersucht werden, wie sich die Gestaltung der Fehlermeldungen auf die Lösungszeit/Fehlerrate auswirkt.

Dieses Projekt kann auch mit den nachfolgenden Projekt-Anregungen kombiniert werden.

9.3.2. Einsteiger-IDE für Escape-Rooms

Text-Widget mit LSP als Einsteiger-IDE



Für die Code-Eingabe im Dungeon-Projekt (vgl. voriger Projektvorschlag) wird ein Text-Widget benutzt. Dabei handelt es sich um die reine Eingabe von Text, es gibt kein Syntaxhighlighting oder Autocompletion oder eine Anzeige von Fehlern o.ä.

Ziel dieses Projekt-Vorschlags ist der Umbau des Text-Widgets zu einer “Mini-IDE” und Integration typischer IDE-Features wie Zeilennummern, Syntaxhighlighting, Autocompletion (Keywords, bekannte Namen aus aktuellem Scope, …), Anzeige von Fehlern oder sogar Tool-Tips beim Hovern mit der Maus und Anzeige der Javadoc (falls vorhanden). Hierzu soll eine geeignete technische Einbindung des interaktiven Interpreter aus dem vorigen Projektvorschlag erarbeitet und umgesetzt werden - beispielsweise losgelöst als LSP-Server vom Text-Widget.

Dieses Projekt kann auch mit der obigen Projekt-Anregung kombiniert werden.

9.3.3. Visualisierung von Programmabläufen

Visualisierung von Programmabläufen in einem interaktiven Interpreter im Dungeon-Framework

Sie entwickeln einen Interpreter, der z.B. einen Sub-Dialekt von Java (“MiniJava”) oder Python (“MiniPython”) zur Laufzeit in ein Computer-Spiel wie das Java-basierte Dungeon-Framework oder das von Ihnen parallel im Wahlmodul “Computer Games” entwickelte Spiel integriert.

Im Mittelpunkt steht die Frage, wie sich der Dungeon als **erweiterte Laufzeitumgebung** eines Interpreters nutzen lässt:

- Wie können Sie Zustände Ihrer Programmiersprache (Variablen, Objekte, Funktionsaufrufe) auf Elemente im Dungeon (z.B. NPCs, Türen, Schalter, Items) **abbilden und sichtbar machen**?
- Wie können Sie umgekehrt Dungeon-Elemente als **sprachseitige Objekte** oder Werte einblenden, sodass sie in der interpretierten Sprache direkt gelesen, verändert oder überwacht werden können?

Ziel ist es, Programmabläufe im Dungeon zu visualisieren: Zustandsänderungen (z.B. Variablenwerte, Objektzustände) sollen sich in der Spielwelt widerspiegeln, etwa durch Bewegung von NPCs, Animationen, Hervorhebungen oder Overlays. Gleichzeitig sollen Änderungen im Dungeon (Events, Kollisionen, Interaktionen) in der interpretierten Sprache beobachtbar und bearbeitbar sein (z.B. über Callbacks, Events, REPL).

Untersuchen Sie geeignete Mappings zwischen Programmzustand und Spielzustand, diskutieren Sie Designalternativen und erstellen Sie eine prototypische Laufzeitintegration mit visueller Darstellung von Programmlogik im Spiel.

9.3.4. Quest-DSL

Entwerfen Sie eine Sprache zur Beschreibung von Quests, Übungsaufgaben und Escape Rooms

Didaktische Inhalte können in Serious Games effizienter bereitgestellt werden, wenn Lehrende über eine fachnahe, deklarative Sprache arbeiten. Spiele(-frameworks) wie beispielsweise das Dungeon-Framework (ECS, Game-Loop) haben eine entsprechende API, die Nutzung erfordert jedoch aktuell dedizierte Kenntnisse der API und der jeweiligen Programmiersprache. Eine **Quest-DSL** schließt diese Lücke und eröffnet Raum für Forschung zu Sprachdesign, Laufzeitintegration, Event-Verarbeitung und Performanz in Game-Loops.

Sie entwickeln eine domänenspezifische Sprache (DSL), mit der sich:

- fachliche Unterrichtsaufgaben, Quests und ggf. komplette Escape Rooms **deklarativ beschreiben** lassen (z.B. Ziele, Bedingungen, Belohnungen, Abhängigkeiten, lineare/nicht-lineare Verläufe),
- sowie das Spiel zur Laufzeit **steuern und auf Spielereignisse reagieren** kann (z.B. bei “Quest erfüllt”, “Item eingesammelt”, “NPC erreicht Zielposition”).

Ihre DSL soll es erlauben, typische Quest-Logiken ohne direkten Kontakt zur Dungeon-API zu formulieren. Dazu gehört u.a.:

- Entwurf der Sprachkonzepte (Syntax, Grundbausteine, z.B. Zustände, Bedingungen, Trigger, Aktionen),
- Umsetzung eines Parsers und statischer Checks (Konsistenz, Vollständigkeit, Referenzierbarkeit von Objekten),
- eine Laufzeitintegration in ein Spiel (z.B. Dungeon-Framework), die Quest-Definitionen lädt, auswertet und mit Spielereignissen koppelt.

Untersuchen Sie existierende Quest-DSLs und Game-Scripting-Sprachen als Ausgangspunkt und reflektieren Sie, welche Designentscheidungen Sie für Ihre Sprache treffen.

Optional: Sie können zusätzlich eine explizite IR (Zwischendarstellung) für Ihre DSL entwerfen und diese als Zwischenziel der Übersetzung nutzen, z.B. zur besseren Analysierbarkeit oder Optimierung.

9.3.5. Compiler Bootstrapping

Eine kleine Sprache mit OOP- und FP-Features entwerfen und zwei Compiler bauen

Sie entwerfen eine kleine Sprache “X” mit OOP- und FP-Elementen.

Sie implementieren Compiler “A” in einer bestehenden Sprache Ihrer Wahl (Haskell, OCaml, Racket, Clojure, Java, zur Not auch Python oder Rust oder andere). Compiler A übersetzt nur ein Minimal-Subset von X: “X-min”. A parst X-min, prüft Typen in einfacher Form und erzeugt Code in der gewählten Zielsprache (“transpiling”).

Sie schreiben nun in X-min den Compiler “B”, der den vollen Sprachumfang von X übersetzen kann. Sie zeigen Bootstrapping: B wird mit A gebaut und kompiliert anschließend in X formulierte Programme. B parst und kompiliert die Vollsprache X. B darf intern ein Desugaring auf den Kern verwenden.

Bootstrapping: B wird mit A auf die Zielplattform transpiliert. Sie kompilieren die *Quellen* von B mit dem Compiler A. Sie erhalten daraus den Compiler B in der Zielsprache (Sourcecode für B in der Zielsprache, z.B. Haskell, Java, …). Sie bauen nun B mit dem passenden Systemwerkzeug (z.B. javac oder gcc). Verwenden sie den so entstehenden ausführbaren Compiler B, um Programme der Vollsprache X zu kompilieren.

Self-Hosting: Kann Ihr Compiler B sich selbst kompilieren, also kann B den in X geschriebenen Quellcode von B korrekt übersetzen?

Transpiling: Die Codegenerierung kann als Transpilation in eine bestehende Zielsprache (z.B. Java oder C) erfolgen. Sie brauchen also keine SSA-Phase und Assembler-Generierung o.ä. umsetzen. Der so generierte Code soll sich mit Standardwerkzeugen kompilieren oder ausführen lassen (z.B. javac, gcc, python, node). Sie sollten vermutlich eine kleine Laufzeitbibliothek in der Zielsprache mitliefern.

9.3.6. LaTeX Equation Language

Parsen und übersetzen eines LaTeX-Teilgrammatik (Arithmetik, cases, Funktionsdefinition) in ausführbaren LLVM-IR-Code oder Java-Bytecode o.ä. Ziel: Brücke von symbolischer Notation zu maschinennahem Code.

9.3.7. MLIR/E-Graphs-Pipeline

Entwurf eines kleinen MLIR-Dialekts (z.B. arithmetische Ausdrücke) mit Lowering nach LLVM; alternativ/ergänzend Optimierung via E-Graphs (Equality Saturation) und anschließendes Lowering.

Neben passenden Konzepten soll auch eine geeignete Umsetzung/Implementierung erstellt und empirisch untersucht werden.

Quellen

Aho, A. V., M. S. Lam, R. Sethi, J. D. Ullman, und S. Bansal. 2023. *Compilers: Principles, Techniques, and Tools, Updated 2nd Edition by Pearson*. Pearson India. https://learning.oreilly.com/library/view/compilers-principles-techniques/9789357054881/.

Fowler, M. 2010. *Domain Specific Languages*. Addison Wesley. https://learning.oreilly.com/library/view/domain-specific-languages/9780132107549/.

Gabbrielli, M., und S. Martini. 2023. *Programming Languages: Principles and Paradigms*. Springer Cham. https://doi.org/10.1007/978-3-031-34144-1.

Grune, D., K. van Reeuwijk, H. E. Bal, C. J. H. Jacobs, und K. Langendoen. 2012. *Modern Compiler Design*. Springer.

Güting, R. H. 1999. *Übersetzerbau: Techniken, Werkzeuge, Anwendungen*. Springer.

Hopcroft, J. E., R. Motwani, und J. D. Ullman. 2003. *Einführung in die Automatentheorie, formale Sprachen und Komplexitätstheorie*. I theoretische informatik. Pearson Education Deutschland GmbH.

Kunert, A. 2018. „LR(k)-Analyse für Pragmatiker“. http://amor.cms.hu-berlin.de/~kunert/papers/lr-analyse/lr.pdf.

Levine, J. 2009. *Flex & Bison*. O’Reilly.

Mernik, M., J. Heering, und A. M. Sloane. 2005. „When and How to Develop Domain-Specific Languages“. *ACM Computing Surveys (CSUR)* 37 (4): 316–44. https://john.cs.olemiss.edu/~hcc/csci658/notes/localcopy/WhenDSL.pdf.

Mogensen, T. 2017. *Introduction to Compiler Design*. Springer. https://doi.org/10.1007/978-3-319-66966-3.

Nystrom, R. 2021. *Crafting Interpreters*. Genever Benning. https://github.com/munificent/craftinginterpreters.

Parr, T. 2010. *Language Implementation Patterns*. Pragmatic Bookshelf. https://learning.oreilly.com/library/view/language-implementation-patterns/9781680500097/.

Parr, T. 2014. *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf. https://learning.oreilly.com/library/view/the-definitive-antlr/9781941222621/.

Torczon, L., und K. Cooper. 2012. *Engineering a Compiler*. Morgan Kaufmann. https://learning.oreilly.com/library/view/engineering-a-compiler/9780080916613/.

Voelter, M. 2013. *DSL Engineering: Designing, Implementing and Using Domain-Specific Languages*. CreateSpace Independent Publishing Platform. https://www.voelter.de/dslbook/markusvoelter-dslengineering-1.0.pdf.

Wagenknecht, C., und M. Hielscher. 2014. *Formale Sprachen, abstrakte Automaten und Compiler*. Springer Fachmedien Wiesbaden. https://doi.org/10.1007/978-3-658-02692-9.



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Exceptions:

- Screenshot of “Language Brainfuck” by Michal Wojciech Tarnowski on 99-bottles-of-beer.net
- “Language C” by Bill Wein on 99-bottles-of-beer.net
- Abzählreim “99 Bottles of Beer” nach “Lyrics of the song 99 Bottles of Beer” on 99-bottles-of-beer.net
- “Language Haskell” by Iavor on 99-bottles-of-beer.net
- “Language Prolog” by M@ on 99-bottles-of-beer.net
- A Map of the Territory (mountain.png) by Bob Nystrom on Github.com (MIT)
- “Language Java” by Sean Russell on 99-bottles-of-beer.net

> **Last modified:** 4f2e07a 2026-10-16 orga: improve readability